

Master in Connected Industry 4.0
2025–2026

Master Thesis

“Learning Binary Traversability from
Monocular RGB for Off-Road
Navigation”

Samer Awad

Tutor

Alberto Jiménez Hormeño
Madrid, July 2026

Abstract

Traversability perception estimates which parts of a visible scene a specific vehicle can negotiate. This thesis formulates that task as binary semantic segmentation from one RGB image using the vehicle-capability annotations of the CaT dataset. The released labels are converted into two documented policies. The principal blue+green policy treats the sedan and pickup regions as traversable while retaining background and off-road-only terrain as blocked. A deterministic split of the official training partition provides validation data, and all 544 released test images are reserved for evaluation.

PIDNet-S is first used to establish the data and evaluation pipeline and to test loss design, augmentation, class weighting, resolution, and decision threshold. Its separately retained reference checkpoint achieves 0.8939 mean Intersection over Union (mIoU), a 4.32% False-Safe Rate (FSR), and a 7.04% False-Block Rate (FBR). A separate CPU evaluation reproduces all four confusion counts over 133,693,440 pixel decisions. The model processes 21.16 frames per second (FPS) on an AMD Ryzen 5 5500 when the model forward pass and prediction are timed, and 106.43 FPS after TensorRT conversion on an NVIDIA RTX 5060 when preprocessing, inference, and mask return are timed.

The architecture study compares PIDNet-S with frozen-encoder ROD and seven other compact systems. An initial 15-epoch matrix supplies a matched fixed-budget comparison under blue-only and blue+green policies. Because the validation histories were still improving, a three-seed blue+green follow-up uses a 300-epoch ceiling, 60-epoch cosine decay, and validation-based early stopping. All nine models achieve higher mean test mIoU than under the fixed 15-epoch budget. FPN/EfficientNet-B0 obtains the largest mean test mIoU (0.9423 ± 0.0005), followed by U-Net/EfficientNet-B0 (0.9409 ± 0.0004); validation selects checkpoints between epochs 50 and 141. Runtime rankings differ: PIDNet-S leads the H100 measurements, while DDRNet-23-Slim leads the compiled CPU comparison.

Examples selected at several error percentiles, together with cases ranked by false-safe, false-block, and boundary error, connect the aggregate scores to the spatial form of the mistakes. Unlabelled ALICE sequences illustrate cross-domain output behaviour but provide no accuracy measurement. These findings support real-time pixel-level perception, but they do not certify that vehicle motion is safe: a single RGB image hides relevant physical properties, capability boundaries may be uncertain, and neither planning nor vehicle-level behaviour is evaluated.

Keywords: traversability estimation; semantic segmentation; PIDNet; ROD; CaT dataset; safety-oriented evaluation; real-time inference; TensorRT; reproducibility.

Resumen

La percepción de transitabilidad estima qué regiones visibles puede recorrer un vehículo determinado. Esta tesis formula la tarea como segmentación semántica binaria de una imagen RGB mediante las anotaciones de capacidad vehicular de CaT. Las etiquetas se convierten en dos políticas explícitas. La política principal azul+verde considera transitables las regiones anotadas como aptas para turismo o camionetas, y mantiene como no transitables el fondo y las regiones reservadas a vehículos todoterreno. Una división determinista de la partición oficial de entrenamiento proporciona los datos de validación, mientras que las 544 imágenes oficiales de prueba se reservan para la evaluación.

PIDNet-S establece el flujo experimental y permite estudiar la pérdida, el aumento de datos, la ponderación, la resolución y el umbral. El modelo seleccionado alcanza un mIoU de 0,8939, una tasa de falso seguro del 4,32% y una tasa de falso bloqueo del 7,04%. Una evaluación independiente en CPU reproduce los cuatro recuentos de la matriz de confusión sobre 133.693.440 decisiones de píxel. El modelo procesa 21,16 imágenes por segundo en un AMD Ryzen 5 5500 cuando se cronometran el pase hacia delante y la predicción, y 106,43 imágenes por segundo tras la conversión a TensorRT en una NVIDIA RTX 5060 cuando se cronometran el preprocesamiento, la inferencia y la devolución de la máscara.

El estudio compara PIDNet-S con ROD, cuyo codificador permanece congelado, y otros siete sistemas compactos. Una primera matriz de 15 épocas proporciona un presupuesto idéntico para las políticas azul y azul+verde. Como la validación seguía mejorando, un segundo experimento azul+verde con tres semillas utiliza un máximo de 300 épocas, decaimiento cosenoidal durante 60 épocas y parada temprana. Los nueve modelos obtienen un mIoU medio de prueba superior al alcanzado con el presupuesto fijo de 15 épocas. FPN/EfficientNet-B0 obtiene el mayor mIoU medio ($0,9423 \pm 0,0005$), seguido de U-Net/EfficientNet-B0 ($0,9409 \pm 0,0004$). Los puntos de control seleccionados mediante validación corresponden a épocas comprendidas entre 50 y 141. En cuanto a la velocidad, PIDNet-S alcanza el mayor rendimiento en la H100 y DDRNet-23-Slim encabeza la comparación compilada en CPU.

Los ejemplos seleccionados por percentiles de error y los casos ordenados por falso seguro, falso bloqueo y error de frontera relacionan las métricas agregadas con la forma espacial de los errores. Las secuencias de ALICE, al no estar anotadas, solo ilustran el comportamiento del modelo fuera del dominio de CaT. Los resultados muestran una percepción por píxel útil en tiempo real, pero no certifican que el movimiento del vehículo sea seguro.

Palabras clave: transitabilidad; segmentación semántica; PIDNet; ROD; CaT; métricas de error asimétricas; inferencia en tiempo real; reproducibilidad.

Notation and Acronyms

Term	Meaning in this thesis
ALICE	Unlabelled external mobile-robot sequences used only for qualitative transfer.
CaT	CAVS Traversability dataset.
CNN	Convolutional neural network.
CPU / GPU	Central processing unit / graphics processing unit.
FBR	False-Block Rate: fraction of truly traversable pixels predicted blocked.
FPN	Feature Pyramid Network.
FP, FN, TP, TN	False positive, false negative, true positive, and true negative, with traversable defined as positive.
FPS	Frames per second, always tied to a declared timing scope.
FSR	False-Safe Rate: fraction of truly untraversable pixels predicted traversable.
H100 MIG	The NVIDIA H100 NVL 3g.47GB multi-instance GPU partition used for common server timing.
IoU / mIoU	Intersection over Union for one class / unweighted mean across the two binary classes.
PIDNet-S	Small PIDNet variant used as the main lightweight baseline.
ROD	RGB-Only Fast and Efficient Off-road Freespace Detection model adapted to CaT.
RGB	Red, green, and blue image channels.
SMP	<code>segmentation-models-pytorch</code> , used for FPN, U-Net, and SegFormer implementations.
TensorRT	NVIDIA inference compiler/runtime used for the optimised RTX 5060 deployment measurements.
ViT-S	Small Vision Transformer encoder variant.

Contents

Abstract	iii
Resumen	iv
Notation and Acronyms	v
List of Figures	xi
List of Tables	xiii
1. Introduction	1
1.1. Motivation	1
1.2. Problem Definition and Research Questions.	2
1.3. Objectives.	2
1.4. Research Design	3
1.5. Contributions	4
1.6. Scope and Responsible Interpretation	5
1.7. Thesis Organisation	5
2. Background and Related Work	6
2.1. Traversability as Perception of an Affordance	6
2.2. Off-Road Perception Datasets.	6
2.3. Dense Semantic Segmentation	7
2.3.1 From Image Classification to Pixel-Level Prediction	7
2.3.2 The Encoder–Decoder Pattern	8
2.3.3 Learning Context at More Than One Scale	9
2.3.4 What the Network Produces in This Thesis	10
2.4. Compact Encoders and Real-Time Architectures	10
2.4.1 Why Compact Models Matter	10
2.4.2 MobileNetV2: Reducing the Cost of Convolution	11
2.4.3 EfficientNet-B0: Balancing Width, Depth, and Image Resolution .	12
2.4.4 Networks Designed Specifically for Real-Time Segmentation . .	13
2.5. PIDNet-S	14
2.5.1 Why the Name Refers to a PID Controller	14

2.5.2	The Three Processing Branches	14
2.5.3	How Information Is Combined	15
2.5.4	Why PIDNet-S Is the Starting Point	15
2.6.	ROD and Frozen-Encoder Transfer.	16
2.6.1	Why Pretraining Is Useful on a Small Dataset	16
2.6.2	From Vision Transformers to EfficientSAM	17
2.6.3	What ROD Changes	17
2.6.4	How ROD Is Used in This Thesis	18
2.7.	CaT Literature and Comparability	18
2.7.1	Why Two Results From CaT May Still Measure Different Tasks	18
2.7.2	The Original Four-Class CaT Evaluation	19
2.7.3	EfficientViT Results and Split Independence	19
2.7.4	Binary CaT With a Different Positive Class	20
2.7.5	The Comparison Rule Used Here	21
2.8.	Research Gap	21
2.8.1	What Does “Traversable” Mean?	21
2.8.2	What Kind of Mistake Did the Model Make?	21
2.8.3	Is a Model Fast on the Hardware That Will Run It?	21
2.8.4	Can the Result Be Traced and Repeated?	22
3.	CaT Binary Task and Common Pipeline	23
3.1.	Dataset Release and Audit.	23
3.2.	Vehicle-Capability Labels and Binary Policies	24
3.3.	Train, Validation, and Test Separation	26
3.4.	Image and Mask Preparation	27
3.5.	Common Software Workflow	28
3.6.	Annotation Ambiguity and Observable Evidence	29
4.	PIDNet-S Model Development.	32
4.1.	Selection Rationale.	32
4.2.	Implemented Architecture.	32
4.3.	Objective, Optimiser, and Checkpoint Selection	35
4.4.	Exploratory Development	37

4.5. Controlled Configuration Ablation	37
4.6. Threshold as an Operating Control	39
4.7. Reference Checkpoint	40
5. ROD and Comparative Architecture Study	41
5.1. Motivation for Pretrained Feature Transfer	41
5.2. Implemented ROD ViT-S Model	41
5.3. Controlled PIDNet-ROD Comparison	43
5.4. Reconstructed ROD Training Regime	43
5.5. Architecture Set	44
5.6. Fixed-Budget Experimental Scope	45
5.7. Convergence-Aware Blue+Green Follow-Up	46
6. Experimental Protocols and Evaluation Methods.	47
6.1. Experimental Phases	47
6.2. Training Procedure	48
6.3. Confusion Matrix Convention.	49
6.4. Accuracy and Asymmetric Error Metrics	50
6.5. Multi-Seed Reporting	51
6.6. Qualitative Review Protocol.	52
6.7. CPU and H100 Timing Protocols.	53
6.8. RTX 5060 TensorRT Protocol.	53
6.9. Artifact Provenance and Reproduction	54
7. Accuracy, Asymmetric Errors, and Qualitative Results	56
7.1. Reference PIDNet-S Result	56
7.2. PIDNet-ROD Comparison	57
7.3. Fixed-Budget Blue+Green Architecture Comparison.	58
7.4. Convergence-Aware Blue+Green Follow-Up	59
7.5. Blue-Only Architecture Comparison	62
7.6. High-Performing PIDNet Cases	63
7.7. Performance Across the Test Distribution	64
7.8. False-Safe, False-Block, and Boundary Failures	67
7.9. Qualitative Transfer to ALICE	68

8. Hardware and Inference-Speed Analysis	70
8.1. Why One FPS Number Is Insufficient	70
8.2. Ryzen 5 5500 CPU Results	70
8.3. Common H100 Results	71
8.4. Accuracy–Speed Relationship.	72
8.5. RTX 5060 TensorRT Results	73
8.6. Mixed-Precision Accuracy Verification	74
8.7. Deployment Meaning	74
9. Discussion and Research Outlook	76
9.1. Synthesis of Findings	76
9.2. Interpreting the Architecture Results	77
9.3. Meaning of the Asymmetric Error Metrics	77
9.4. Annotation and Task Uncertainty	78
9.5. Limitations	79
9.6. Future Work	80
10. Conclusions.	82
Bibliography.	84
A. Configuration Reference	87
A.1. PIDNet-S Configuration	87
A.2. Reconstructed ROD Configuration.	87
A.3. Common Fixed-Budget Architecture-Comparison Template	88
A.4. Convergence-Aware Follow-Up Template.	89
B. Reference Evaluation Record	90
C. Reproduction Commands	91
C.1. Audit and Preprocessing	91
C.2. PIDNet-S Evaluation	91
C.3. Ranked Qualitative Review	91
C.4. TensorRT Benchmarks	92
C.5. Architecture Ledgers	92
D. Artifact Identities	93
E. Implementation Map	94

F. Development Time and Work Plan.	96
--	----

List of Figures

1.1	Experimental structure.	4
2.1	From an RGB image to a binary segmentation mask.	9
2.2	Standard and depthwise-separable convolution.	11
2.3	Frozen-encoder transfer learning used by ROD.	17
3.1	Conversion from CaT raw classes to the main binary policy.	24
3.2	Example of the two binary conversions.	25
3.3	CaT train, validation, and test separation.	26
3.4	Common implementation workflow.	29
3.5	Examples of ambiguity in the available RGB evidence.	30
4.1	Conceptual PIDNet-S three-branch architecture.	33
4.2	Implemented PIDNet-S binary pipeline.	34
4.3	False-safe and false-block trade-off under post-hoc threshold variation. A threshold is an operating choice; it does not improve both error directions simultaneously.	39
5.1	Implemented ROD ViT-S pipeline.	42
6.1	Experimental phases and controls.	47
6.2	Visual interpretation of the binary confusion matrix.	50
6.3	Timing scopes used for the RTX 5060 benchmark.	54
7.1	Fixed-budget and convergence-aware test mIoU.	60
7.2	Validation-selected epochs in the convergence suite.	61
7.3	Validation histories for the convergence-aware runs.	62
7.4	High-performing reference PIDNet-S examples.	64
7.5	PIDNet-S predictions across the test performance range.	65
7.6	Detailed successful winding-track example.	66
7.7	Asymmetric-error examples for reference PIDNet-S.	67
7.8	Unscored PIDNet-S transfer to ALICE.	68

8.1	Blue+green accuracy versus H100 throughput.	72
8.2	TensorRT latency within a 30-FPS frame budget.	73
9.1	From pixel metrics to vehicle-level evidence.	78

List of Tables

2.1	Representative off-road datasets and their relation to this work.	7
2.2	Architecture families, implemented systems, and the principal design contrast represented in the comparative experiment.	14
2.3	Original four-class CaT test results reported by Sharma et al. [1]; values are percentages.	19
2.4	Why headline CaT results require protocol alignment before comparison.	20
3.1	Audited CaT environment counts and stored RGB resolutions.	23
3.2	CaT split and the role of each subset.	27
4.1	Principal implemented PIDNet-S feature shapes at 640×384	34
4.2	Training configuration of the reference PIDNet-S model.	36
4.3	Exploratory PIDNet-S development runs. Values are test results; the runs are confounded and do not estimate individual causal effects.	37
4.4	Single-seed controlled PIDNet-S ablation. Bold identifies the configuration retained for subsequent analysis. Lower FSR and FBR are preferable. . .	38
4.5	Post-hoc threshold sweep on the earlier blue_green_best checkpoint. .	39
5.1	Training choices held in the controlled architecture comparison and their counterparts in the closest documented ROD recipe.	44
5.2	Exact model systems, parameter counts, and initialisation in the comparative study.	45
6.1	Operational interpretation of the four binary outcomes.	50
7.1	Verified reference PIDNet-S result for blue+green traversability. Lower FSR and FBR are preferable; the other quality metrics are higher-is-better.	56
7.2	Aggregated pixel confusion matrix for the reference PIDNet-S checkpoint. Rows are ground truth and columns are prediction.	56
7.3	Closest documented ROD recipe transferred to CaT. This is a single-seed approximation because the publication omits the epoch budget and augmentation policy.	57

7.4	Fixed-15-epoch blue+green results over seeds 1337, 2027, and 4242 (mean $\pm$ population standard deviation). Lower FSR and FBR are preferable. . .	58
7.5	Convergence-aware blue+green architecture results.	59
7.6	Controlled blue-only results over seeds 1337, 2027, and 4242 (mean $\pm$ population standard deviation). Lower FSR and FBR are preferable. . . .	63
8.1	Ryzen 5 5500 batch-one throughput at 640×384 . Values are forward and prediction only; compiled timing excludes compilation warm-up.	70
8.2	Parameter count and common H100 NVL 3g.47GB MIG batch-one forward throughput. External input is 640×384 ; execution is eager FP32. . .	71
8.3	TensorRT batch-one timing on the RTX 5060.	73
8.4	Full 544-image comparison of each source PyTorch checkpoint with its mixed-precision TensorRT conversion.	74
A.1	Configuration model names used to construct each architecture-comparison system.	89
D.1	SHA-256 identities for the PIDNet evidence chain.	93
E.1	Principal repository files and their role in the experimental record.	94
F.1	Development-time breakdown.	96

1. Introduction

1.1. Motivation

Autonomous driving on a paved road is supported by a set of regularities: lanes have conventional widths, curbs provide visible boundaries, signs encode rules, and the road surface is designed for ordinary vehicles. An off-road vehicle encounters none of these guarantees. A pale strip in an image may be compacted gravel, loose soil, shallow water, or a patch of dry vegetation. The border between a track and its surroundings can be gradual, interrupted, or hidden by shadow. Even if the surface category were recognised perfectly, the answer to “can the vehicle cross it?” would still depend on clearance, traction, wheelbase, mass, slope, and current conditions.

Traversability is therefore an *affordance*: it describes what a particular platform can do in a scene. The same region may be accessible to a pickup, outside a sedan’s capability, and trivial for a dedicated off-road vehicle. This vehicle dependence makes traversability more operational than ordinary terrain classification. It also creates a demanding perception problem, because some of the relevant physical properties are not directly observable from one RGB frame [3, 4].

The practical output considered in this thesis is a dense binary mask. Every image pixel is labelled either traversable or untraversable for a declared vehicle-capability policy. A mask is convenient for downstream processing: a planner can intersect it with a projected footprint, inflate blocked regions, or combine it with geometric and temporal estimates. Dense prediction also exposes the spatial structure of errors. A connected false-safe region directly in front of the vehicle has a different operational meaning from a handful of isolated pixels at the top of the image, even if both contribute the same number of errors to a global score.

Two competing engineering requirements motivate the study. First, the mask must be accurate enough to represent narrow corridors and irregular boundaries. Second, it must be produced fast enough for an onboard navigation loop. A large pretrained transformer may supply strong visual features, while a purpose-built real-time network may offer far lower latency. The thesis therefore studies accuracy, asymmetric error rates, reproducibility, and hardware speed together rather than treating any one of them as a complete measure of success.

1.2. Problem Definition and Research Questions

For an RGB image

$$x \in \mathbb{R}^{H \times W \times 3}, \quad (1.1)$$

where x is the input tensor, H and W are its height and width in pixels, the final dimension contains the red, green, and blue channels, and $\mathbb{R}$ denotes real-valued samples. The system estimates a traversable probability for each pixel and produces

$$\hat{y}_{ij} = \mathbb{I}[p(y_{ij} = 1 \mid x) \geq \tau], \quad \hat{y} \in \{0, 1\}^{H \times W}, \quad (1.2)$$

Here, $i \in \{1, \dots, H\}$ and $j \in \{1, \dots, W\}$ index the pixel row and column; y_{ij} is its target class; $p(y_{ij} = 1 \mid x)$ is the model's conditional probability of the traversable class; $\mathbb{I}[\cdot]$ is one when its condition is true and zero otherwise; and $\hat{y}_{ij}$ is the resulting binary prediction. Class 1 is traversable, class 0 is untraversable, and τ is the fixed decision threshold. The complete predicted mask $\hat{y}$ therefore has the same $H \times W$ spatial dimensions as the image. The main PIDNet-S development uses $\tau = 0.50$ and the CaT blue+green policy: sedan- and pickup-traversable pixels are positive, while background and off-road-vehicle-only pixels are negative. The architecture experiment also evaluates the more conservative blue-only policy.

This definition makes the two error directions explicit. A false positive opens a pixel that the annotation declares blocked; it is called a *false safe*. A false negative rejects a pixel that the annotation declares traversable; it is called a *false block*. The first can expose a planner to terrain outside the selected capability envelope. The second can remove a feasible route or cause unnecessary stopping. The costs are asymmetric, but they are not completely described by their pixel counts: location, connectedness, vehicle speed, and the planner all affect the consequence.

The investigation is organised around four research questions:

1. Can PIDNet-S provide a reproducible, real-time baseline for the CaT binary task using only the official training partition?
2. Which of the tested training and postprocessing choices materially change its accuracy, false-safe behaviour, and false-block behaviour?
3. How does frozen-encoder transfer in ROD compare with PIDNet-S under matched seeds, and where do both systems lie among other compact segmentation architectures?
4. How do the accuracy rankings change when inference latency, target hardware, execution runtime, and timing scope are included?

1.3. Objectives

The investigation has four methodological objectives:

1. define a reproducible binary task from the released CaT files, including explicit vehicle policies and a development split that does not draw from the official test partition;
2. determine how a compact PIDNet-S implementation responds to changes in training objective, augmentation, class weighting, resolution, and threshold, while preserving the complete record of the selected model;
3. compare that model with frozen-encoder ROD and a representative group of compact convolutional and transformer systems under repeatable training conditions;
4. relate overlap quality and asymmetric errors to observed latency on CPU, H100, and TensorRT execution rather than using parameter count as a proxy for deployability.

1.4. Research Design

The experiments were organised in two linked stages. The first stage treated PIDNet-S as an engineering reference. Building that model required decisions about file pairing, target conversion, augmentation, loss terms, validation, error reporting, and export. Exploratory runs exposed interactions between several of those choices; a subsequent single-factor study separated them. The selected checkpoint was retained unchanged so that its numerical, qualitative, and runtime measurements all refer to the same parameters.

The second stage asked whether the conclusion depended on that initial architecture. ROD introduced a pretrained transformer whose encoder remained fixed during CaT training. A matched PIDNet-ROD experiment measured the difference over three seeds, after which seven additional systems supplied context for the pairwise result. The first matrix used a common 15-epoch budget under both label policies. Its validation histories showed that this was useful as a fixed-budget comparison but too early to represent the models' attainable performance. A convergence-aware follow-up therefore repeated the principal blue+green matrix with a longer ceiling and validation-based stopping. This order separates three questions that are often conflated: how one model was engineered into a reproducible system, how complete model recipes compare under equal training exposure, and how that ranking changes when each run is allowed to progress beyond the early fixed budget.

Figure 1.1 summarises this progression from a fixed task definition to model development, comparison, and hardware evaluation.

The validation-selected PIDNet-S checkpoint and the three-seed comparison are therefore reported separately. The former is an artifact-level result backed by exact reproduction and deployment measurements. The latter estimates between-run variation under a common recipe. Combining them into one average would erase the experimental conditions that give each number its meaning.

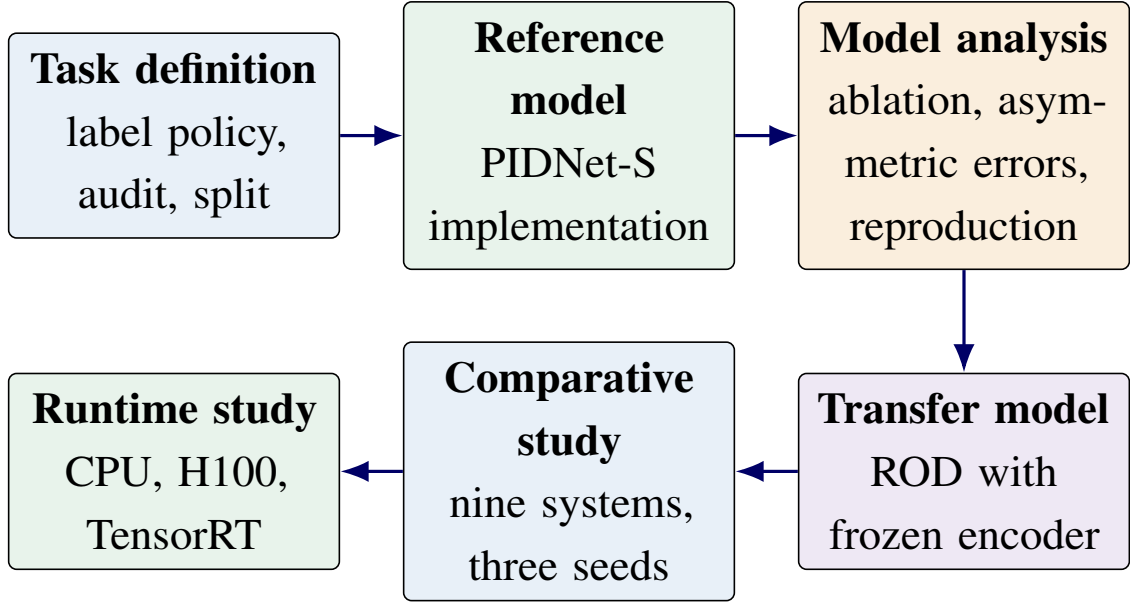


Fig. 1.1. Experimental structure. A fixed task definition connects model development, repeatability analysis, architectural comparison, and hardware measurement.

1.5. Contributions

The resulting contributions are:

- an audited conversion of 1,812 underlying CaT samples to binary traversability, with an unchanged 544-image official test partition excluded from training and validation splitting and a deterministic 1,002/266 train–validation split;
- a detailed implementation of PIDNet-S for two-class prediction, including its tensor shapes, objective, optimisation, decision threshold, and controlled ablation history;
- a 7,623,522-parameter PIDNet-S checkpoint with 0.893941 mIoU, 4.3176% FSR, 7.0369% FBR, and an exactly reproduced confusion matrix covering 133,693,440 pixel decisions;
- percentile-based and failure-ranked qualitative evidence that connects the aggregate asymmetric error rates to visible spatial behaviour;
- a CaT implementation of ROD with a frozen EfficientSAM ViT-S encoder and an explicit reconstruction of the published training regime;
- a nine-system, three-seed comparison under both blue-only and blue+green policies, including a 2×2 FPN/U-Net by MobileNetV2/EfficientNet-B0 design;
- a convergence-aware three-seed follow-up for the principal blue+green policy, with 27 validation-selected checkpoints, complete test records, and checkpoint identities;
- hardware measurements that distinguish forward-only CPU and H100 timing from a larger TensorRT preprocessing-to-returned-mask scope.

1.6. Scope and Responsible Interpretation

This thesis studies monocular RGB segmentation. It does not estimate depth, slope, soil strength, wheel slip, water depth, or a dynamically feasible trajectory. The term *safe* is retained in the metric name “False Safe Rate” because it describes the binary label convention, not because the network output certifies safe vehicle motion. A deployed system would require sensor fusion, uncertainty handling, planner constraints, and vehicle-level validation.

The quantitative claims are restricted to CaT, the two stated binary policies, the declared seeds, and the saved protocols. Frames from the external ALICE mobile-robot sequences are used only for qualitative transfer because no ground truth is available. Hardware rates are not interchangeable: changing the processor, runtime, numerical precision, batch size, input resolution, or the start and end of the timer changes the meaning of FPS.

The CaT labels are treated as the reference for scoring, but not as a perfect measurement of physical reality. Some boundaries are difficult to justify from RGB appearance alone and may depend on physical information or subjective vehicle-capability judgement unavailable to the model and thesis author. Specific examples and the limits of that observation are discussed in Chapters 3, 7, and 9.

1.7. Thesis Organisation

Chapter 2 introduces traversability perception, relevant datasets, segmentation architectures, and CaT literature. Chapter 3 explains the CaT release, binary label policies, split, preprocessing, and common software pipeline. Chapter 4 presents the PIDNet-S implementation, exploratory experiments, and controlled ablation. Chapter 5 introduces ROD and defines the comparative architecture study. Chapter 6 defines training, evaluation, qualitative, reproducibility, and timing protocols. Chapters 7 and 8 report accuracy, safety-oriented error metrics, qualitative behaviour, and hardware speed. Chapter 9 analyses the trade-offs, limitations, and future work; Chapter 10 summarises the evidence in relation to the research questions. Detailed configurations and reproduction commands are provided in the appendices.

2. Background and Related Work

2.1. Traversability as Perception of an Affordance

Traversability estimation asks whether a mobile platform can move through a region of the environment. This differs from semantic terrain recognition. The class “grass”, for example, does not determine whether the surface is load-bearing, whether vegetation hides a ditch, or whether the slope exceeds the vehicle’s limits. Conversely, several semantic materials can form one continuous traversable corridor. Surveys therefore describe traversability as a combination of geometric, appearance-based, semantic, and vehicle-interaction evidence [3, 4].

Geometric approaches use LiDAR, stereo, or structured depth to estimate slope, roughness, steps, and obstacle height. They offer direct evidence for some physical constraints, but flat vegetation, water, deformable soil, and sparse returns remain difficult. Appearance-based systems use colour, texture, shape, and learned context from RGB images. They are inexpensive and information-rich but cannot reliably infer every physical property. A multimodal system can combine both strengths at the cost of calibration, synchronisation, sensor weight, and computation.

Another distinction concerns the source of supervision. Human annotation can describe regions that appear suitable for a vehicle. Self-supervised systems can learn from wheel slip, suspension response, vehicle tracks, or a geometric teacher. Interaction-derived labels are platform-specific, but they cover only terrain the robot visits and inherit the assumptions of the teacher signal. The present study uses a human-annotated dataset because it enables every model to receive exactly the same dense target. This makes architectural comparison possible, while leaving the physical validity of the labels as an explicit limitation.

2.2. Off-Road Perception Datasets

Off-road datasets vary in their task, sensors, geography, and label semantics. RUGD supplies dense outdoor semantic labels in unstructured scenes [5]. RELIS-3D combines RGB and LiDAR at an off-road proving ground [6]. TartanDrive emphasises multimodal sequences and vehicle dynamics [7]. ORFD targets off-road freespace detection with RGB and LiDAR-derived information [8]. WildScenes provides repeated natural-environment traversals with calibrated image and LiDAR data, enabling 2D, 3D, and domain-adaptation studies [9].

Table 2.1 summarises the sensing modalities and label scope of these datasets relative to the monocular binary task used here.

TABLE 2.1. Representative off-road datasets and their relation to this work.

Dataset	Year	Images	Primary data	Main label or research focus
RUGD [5]	2019	7,453	RGB	24 semantic outdoor classes
RELLIS-3D [6]	2021	6,235	RGB + LiDAR	Multimodal off-road semantics
CaT [1]	2022	1,812	RGB	Nested vehicle-capability traversability
TartanDrive [7]	2022	—	Multimodal	Terrain interaction and vehicle dynamics
ORFD [8]	2022	12,198	RGB + LiDAR	Off-road freespace detection
WildScenes [9]	2025	sequences	RGB + LiDAR	Natural-scene semantics and domain change

CaT is smaller than several alternatives, but its label semantics are particularly relevant to the research question. Instead of assigning material names, it represents nested vehicle capability: sedan-traversable terrain is contained within pickup-traversable terrain, which is contained within off-road-vehicle-traversable terrain [1]. This hierarchy can be converted directly into different binary operating policies. Its small size also makes data efficiency, pretraining, and overfitting central concerns.

2.3. Dense Semantic Segmentation

2.3.1. From Image Classification to Pixel-Level Prediction

A conventional image classifier answers one question about an entire image: for example, whether the image contains a road vehicle. Semantic segmentation answers a different and more detailed question. It assigns a class to every pixel, so its output has the same spatial meaning as the input image. In the binary task used in this thesis, each output pixel represents one of two decisions: *traversable* or *untraversable*. The result can therefore be displayed as a mask laid directly over the camera image.

This distinction changes the design of the network. A classifier is allowed to discard the exact position of an object as long as it identifies the object correctly. A segmentation model must retain enough position information to reconstruct boundaries. If a trail occupies the centre of an image, predicting “trail present” is not sufficient; the model must indicate where the trail begins and where the vegetation, ditch, or obstacle beside it starts.

The input to the models in this work is an RGB image. RGB means that each pixel is represented by three numbers describing its red, green, and blue intensity. The first layers of a convolutional neural network (CNN) apply small learned filters across this image. A filter can respond to a simple local pattern such as an edge, a colour transition, or a texture. Later layers combine many such responses into more abstract features. A feature is not a manually assigned label; it is an internal numerical representation learned because it helps the network reduce its training error.

The region of the input that can influence one feature is called its *receptive field*. Early features see only a small neighbourhood. Deeper features can see a much larger part of the scene. This larger view is useful: a brown patch may be recognised as part of a track only after the network also sees its continuation through the image. However, repeatedly reducing the feature-map resolution to obtain this wider view also removes fine spatial detail. Semantic segmentation architectures are largely different solutions to this trade-off between broad context and precise location.

2.3.2. The Encoder–Decoder Pattern

The development of fully convolutional networks made it possible to train a network end to end for dense prediction [10]. “Fully convolutional” means that the network does not end with the fixed-size fully-connected classification layers used by older image classifiers. Instead, it preserves a spatial grid of features and converts that grid back into a pixel-level prediction.

Most modern segmentation systems can be explained using two components:

- The **encoder** reads the image and progressively builds stronger semantic features. It commonly reduces width and height while increasing the number of feature channels. This makes processing cheaper and increases the receptive field, but it also produces a coarse representation.
- The **decoder** converts the coarse representation into a dense mask. It increases the spatial resolution and combines high-level meaning with earlier features that still contain local detail.

Increasing resolution is often called *upsampling*. Upsampling alone cannot recreate information that the encoder discarded. For this reason, decoders frequently use *skip connections*: direct routes that carry features from an earlier, higher-resolution encoder stage to a corresponding decoder stage. The decoder then has both kinds of evidence available. Deep features help decide *what* a region is, while shallow features help decide *where* its edge lies.

Figure 2.1 summarises this complete change of representation. The image is compressed into progressively coarser feature grids, then expanded into a mask. The skip connections carry location detail around the narrowest part of the network instead of asking the decoder to invent that detail from the coarse representation.

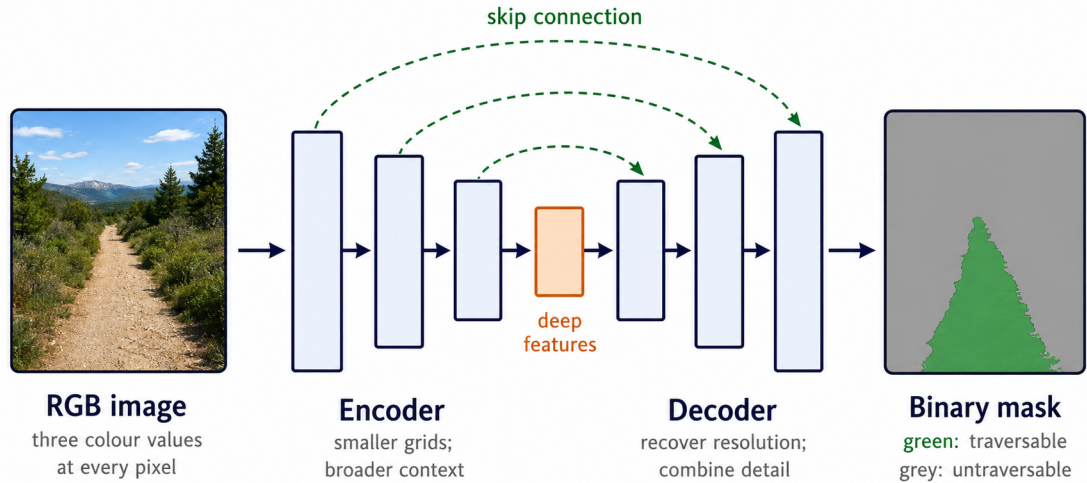


Fig. 2.1. From an RGB image to a binary segmentation mask. The encoder gathers increasingly broad context, the decoder restores spatial resolution, and skip connections preserve local detail. The landscape and mask are conceptual rather than dataset samples. Original schematic drawn for this thesis.

U-Net is a clear example of this pattern [11]. Its encoder contracts the image representation in several stages, and its approximately symmetric decoder expands it again. At each scale, a skip connection provides the decoder with encoder features of the same resolution. U-Net was introduced for biomedical images, where accurate boundaries are important, but the same design principle is useful for roads, terrain, and many other segmentation tasks.

Feature Pyramid Networks (FPNs) organise the same multi-scale idea differently [12]. The encoder first produces a hierarchy of features: fine features near the input and coarser, more semantic features deeper in the network. A top-down path then carries semantic information back towards the finer resolutions. Lateral connections join the top-down signal with the encoder feature at each scale. The result is a pyramid in which several resolutions contain useful semantic information. Although FPN was introduced for object detection, this pyramid is also a practical decoder for segmentation.

2.3.3. Learning Context at More Than One Scale

An object or terrain region does not have one fixed size in the image. Nearby stones may occupy many pixels; the same stones in the distance may occupy only a few. Segmentation systems therefore need to combine information gathered at different spatial scales.

PSPNet does this using pyramid pooling [14]. It pools a deep feature map into several grid sizes. A very coarse grid summarises almost the whole scene, while finer grids retain more local variation. The pooled results are resized and combined, giving the prediction access to both global and regional context. DeepLabv3+ uses another method, known as

atrous or *dilated* convolution [13]. A dilated filter samples positions that are spaced apart, allowing it to cover a wider area without reducing the feature-map resolution by the same amount. DeepLabv3+ then adds a decoder to refine boundaries.

These methods are not merely interchangeable names. They express different engineering choices. Pooling summarises regions explicitly. Dilated convolution widens the sampled neighbourhood. Skip-connected decoders recover detail from earlier layers. Multi-resolution networks, discussed below, keep more than one resolution active throughout much of the model. Each choice changes memory use, computation, and the type of spatial evidence that reaches the final classifier.

2.3.4. What the Network Produces in This Thesis

The final layer produces one numerical score, or *logit*, for each class at each pixel. A logit is an unnormalised confidence value. During training, the logits are compared with the annotated mask by a loss function. The loss is a single number that measures how incompatible the prediction is with the target; optimisation changes the network weights to reduce it. During evaluation, the class with the stronger output is selected, or an equivalent probability threshold is applied for the binary decision.

For CaT, the target is not a material map. The positive class is defined by a vehicle-capability policy described in Chapter 3. A pixel showing grass can be positive in one place and negative in another if the annotation indicates different vehicle access. The network is thus learning a visual approximation of the released capability labels, not a universal law that grass, soil, or gravel is always safe.

The context–resolution trade-off has a direct meaning here. The model should recognise that a partly shadowed surface continues as one track, which requires scene context. At the same time, it should place the border between that track and adjacent vegetation accurately, which requires local detail. A large open region contributes many pixels to an average score, while a thin but safety-relevant boundary contributes relatively few. This is why the thesis examines not only overall overlap but also false-safe, false-block, and boundary errors.

2.4. Compact Encoders and Real-Time Architectures

2.4.1. Why Compact Models Matter

An autonomous vehicle receives a continuous stream of camera frames. A useful model must finish processing one frame before too many new frames arrive. Otherwise the perception result describes an increasingly old scene. Large models may achieve high accuracy on a server but be unsuitable for a vehicle with limited processing power, memory, electrical power, or cooling.

The word *compact* can describe several different properties. A model may have few learned parameters, require few arithmetic operations, use little temporary memory, or run quickly on a particular processor. These properties are related but not identical. Parameters are the learned numerical weights stored by the model. Operation counts estimate arithmetic work. Latency is the time actually required to process a frame. A model with fewer parameters can still be slower if its operations are poorly supported by the hardware or if it repeatedly moves large feature maps through memory.

This thesis therefore treats compact architecture design and measured runtime as separate questions. Architecture explains why a model might be efficient. Timing experiments later determine whether that expectation holds on the CPU, H100 partition, and RTX 5060 systems that were actually used.

2.4.2. MobileNetV2: Reducing the Cost of Convolution

A standard convolution learns filters that jointly combine spatial positions and input channels. This is powerful but can be expensive. A *depthwise-separable convolution* divides the work into two steps. First, a small spatial filter is applied independently to each channel. Second, a 1×1 convolution mixes information across channels. The factorisation usually requires much less computation than one dense convolution with a similar input and output size.

Figure 2.2 shows the factorisation without relying on operation-count notation. A standard convolution performs spatial filtering and channel mixing together. The depthwise-separable alternative first asks where a pattern occurs inside each channel and then uses a 1×1 operation to combine the channel responses.

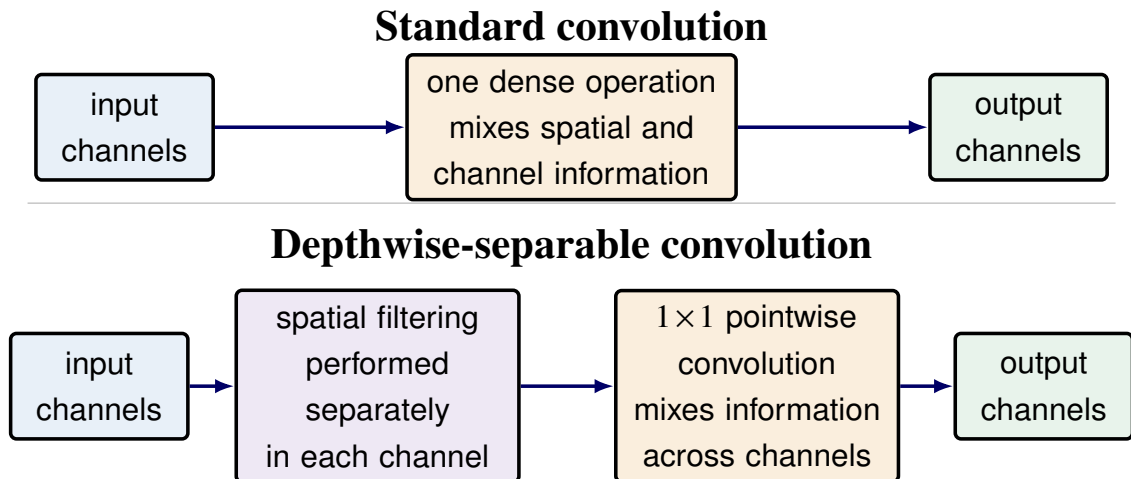


Fig. 2.2. Conceptual difference between a standard convolution and a depthwise-separable convolution. Splitting spatial filtering from channel mixing is the main efficiency mechanism used by MobileNetV2. Original schematic drawn for this thesis.

MobileNetV2 was designed around this idea for resource-constrained devices [16].

Its characteristic unit is the *inverted residual block*. The block first expands a narrow input into more channels, applies an inexpensive depthwise spatial convolution, and then projects the result back into a narrow representation. It is “inverted” because the wider representation exists inside the block rather than between blocks, which is the opposite of the bottleneck pattern used in many earlier residual networks.

The final projection is linear: it does not apply the usual nonlinear activation after compressing the feature. MobileNetV2 calls this a *linear bottleneck*. The motivation is that forcing a nonlinear operation onto a very narrow representation can destroy useful information. When the input and output shapes match, a residual connection also lets the block add its input to its output. This gives information and gradients a shorter path through the network.

In this thesis, MobileNetV2 is used as an encoder rather than as a complete image classifier. Intermediate features from several resolutions are passed to an FPN or U-Net decoder. This pairing asks a concrete question: when the decoder is held fixed, how well does a highly economical encoder preserve the evidence needed for CaT segmentation?

2.4.3. EfficientNet-B0: Balancing Width, Depth, and Image Resolution

There are three common ways to make a CNN larger. It can be made *deeper* by adding layers, *wider* by adding channels to each layer, or it can process a higher-resolution image. Increasing only one dimension can produce an unbalanced model. For example, a very deep but narrow network may have limited feature capacity, while a wide network operating on a small image cannot recover detail that the resizing step removed.

EfficientNet introduced a compound scaling method that increases depth, width, and input resolution together using a coordinated rule [17]. EfficientNet-B0 is the baseline architecture from which the larger EfficientNet variants are scaled. Its building blocks also use efficient depthwise operations, but the overall channel sizes and stage layout differ from MobileNetV2.

EfficientNet-B0 is relevant here for two reasons. First, it is still small enough to be a plausible compact encoder. Second, pretrained weights are widely available, so the model begins with visual features learned from a much larger image collection than CaT. Pretraining does not mean that the network already understands traversability. It means that the early and middle layers already respond to broadly useful visual structures such as contours, textures, and object parts. Training on CaT then adapts those features to the binary target.

MobileNetV2 and EfficientNet-B0 are each paired with FPN and U-Net. This 2×2 arrangement avoids changing the encoder and decoder at the same time. MobileNetV2/FPN can be compared with EfficientNet-B0/FPN to isolate the encoder choice approximately. MobileNetV2/FPN can be compared with MobileNetV2/U-Net to examine the decoder choice while retaining the same encoder. It is a limited comparison rather than a universal

proof, because training details and implementation quality can still interact with each architecture.

2.4.4. Networks Designed Specifically for Real-Time Segmentation

General encoder–decoder models reduce an image and reconstruct it afterwards. Real-time segmentation research has also explored architectures that preserve a high-resolution route for longer.

BiSeNetV2 uses two branches [20]. The detail branch remains comparatively wide and shallow so that it retains edges and fine spatial structure. The semantic branch reduces resolution more aggressively so that it can recognise scene-level meaning at lower computational cost. A guided aggregation layer combines the branches rather than simply adding their outputs. In plain terms, one route concentrates on *where* boundaries are, the other on *what* the regions mean, and the fusion stage learns how much to use from each.

DDRNet, or Deep Dual-Resolution Network, also keeps two resolutions active [19]. A low-resolution branch can be deep and gather broad context efficiently. A high-resolution branch preserves spatial precision. Repeated bilateral exchanges pass information in both directions: coarse semantic information is resized and supplied to the detailed branch, while detailed information is reduced and supplied to the coarse branch. The 23-Slim variant included in this thesis is the smaller, throughput-oriented member of the family.

SegFormer provides a different comparison because its encoder is based on a vision transformer rather than only on convolution [18]. A transformer represents an image as a sequence of feature tokens and uses *attention* to let one token gather information from other locations. This can connect distant parts of the image without relying only on a long chain of local convolutional filters. SegFormer’s Mix Transformer is hierarchical: it progressively reduces resolution and produces features at several scales, much like a CNN pyramid. A lightweight multilayer-perceptron (MLP) decoder projects and combines these features. Here, an MLP is a small set of learned channel-mixing layers, not a second large image-processing network.

PIDNet is the third multi-resolution family in the comparison. It retains separate routes for detail, context, and boundaries. Because PIDNet-S is the initial model developed in this thesis, its design is explained separately in Section 2.5.

None of these descriptions establishes which network is fastest. Attention, depthwise convolution, multi-branch exchange, and high-resolution features produce different patterns of arithmetic and memory access. CPUs, general GPUs, and TensorRT may optimise those patterns differently. Consequently, the later hardware chapter reports measured latency under a stated timing scope instead of converting parameter counts into assumed speed.

Table 2.2 groups the model families later compared and identifies the principal design contrast represented by each one.

TABLE 2.2. Architecture families, implemented systems, and the principal design contrast represented in the comparative experiment.

Family	Systems in this thesis	Main design question represented
General encoder–decoder	FPN and U-Net with MobileNetV2 or EfficientNet-B0	How encoder capacity and decoder organisation affect the common task
Multi-resolution real-time CNN	PIDNet-S, DDRNet-23-Slim, BiSeNetV2	Whether explicit detail/context streams provide a better speed–accuracy balance
Compact task-specific transformer	SegFormer-B0	Whether hierarchical attention with a light decoder improves context efficiently
Frozen foundation-model transfer	ROD ViT-S	Whether pretrained EfficientSAM features transfer to CaT with only the decoder trained

2.5. PIDNet-S

2.5.1. Why the Name Refers to a PID Controller

PIDNet takes its name from proportional–integral–derivative (PID) control [2]. A traditional PID controller repeatedly compares a desired value with a measured value. Its proportional term reacts to the current error, its integral term accumulates error over time, and its derivative term reacts to how quickly the error is changing. PIDNet does *not* place such a controller inside the segmentation model and it does not operate over time in this thesis. The authors use the three roles as an analogy for organising spatial image information.

The analogy addresses a known problem in segmentation. A deep, low-resolution route is effective at understanding the overall scene but can produce smooth, imprecise borders after it is enlarged. A high-resolution route preserves location but may lack enough context to distinguish visually similar regions. PIDNet adds a third, boundary-focused route instead of expecting one feature stream to perform all three jobs.

2.5.2. The Three Processing Branches

PIDNet calls its branches P, I, and D:

- The **P branch** is the proportional branch. It keeps a relatively high feature resolution and responds strongly to local appearance. Its main role is to preserve spatial detail: corners, narrow structures, and the position of region boundaries.

- The **I branch** is the integral branch. It follows a deeper, lower-resolution path and obtains a larger receptive field. Its main role is semantic context. It can use a wider view to decide whether a local texture belongs to a continuous track, vegetation, or background.
- The **D branch** is the derivative branch. It is trained to emphasise rapid spatial changes associated with semantic boundaries. Here, *high frequency* means that feature values change sharply across nearby pixels, as they often do at an edge. The branch supplies an explicit cue about where one predicted region should end and another should begin.

The branches are not independent models. They exchange information at several stages and are fused before the final mask is produced. This matters because a boundary cue without semantic context cannot say which side is traversable, while a semantic region without boundary evidence may spread too far.

2.5.3. How Information Is Combined

The Pixel-attention-guided Fusion Module (PagFM) transfers information from the context-rich I branch to the detailed P branch. *Attention* here means a learned spatial weighting. The module compares features and computes where the semantic information should be trusted. It can then add useful context without replacing the high-resolution representation everywhere.

Near the end of PIDNet-S, the Parallel Aggregation Pyramid Pooling Module (PAPPM) collects context at several scales. Like other pyramid-pooling methods, it creates summaries over differently sized regions. The parallel, lightweight construction is intended to provide a broad scene view without turning the small model into a heavy decoder.

The final combination uses Light-Bag, the lightweight form of the Boundary-attention-guided fusion module. Boundary information from the D branch guides the combination of detailed and semantic features. Away from a boundary, the model can favour stable region-level context. Near a boundary, it can give more importance to the representation that preserves local structure. The exact weights are learned during training rather than defined by a hand-written rule.

2.5.4. Why PIDNet-S Is the Starting Point

The suffix “S” identifies the small PIDNet variant. The implementation used here has 7.62 million parameters. It was selected as the first model because the CaT task contains the three visual demands represented by its branches: broad trail context, small irregular details, and capability boundaries that matter to the safety interpretation. Its size also made repeated development experiments practical and provided a plausible route to CPU and embedded-GPU deployment.

This motivation is a hypothesis, not a result. A specialised boundary branch does not guarantee the best CaT score, and a low parameter count does not guarantee the lowest latency. Chapters 4 and 7 therefore separate the architectural rationale from the measured outcome. PIDNet-S first establishes the dataset, loss, checkpoint, threshold, and evaluation pipeline. The later controlled study then compares it with conventional encoder–decoder networks, other real-time CNNs, a compact transformer, and ROD.

2.6. ROD and Frozen-Encoder Transfer

2.6.1. Why Pretraining Is Useful on a Small Dataset

Training a neural network begins by adjusting its weights from an initial state. If the initial weights are random, the model must learn both basic visual patterns and the final task from the available training images. CaT is small compared with the large datasets commonly used in computer vision. A high-capacity network can therefore memorise details of the training set instead of learning features that transfer to new scenes.

Pretraining addresses this problem by starting from weights learned on a larger source task. The source task need not use the same labels. Edges, repeated textures, object contours, and relationships between image regions can remain useful when the final label changes from an object category to traversability. The final model then performs *transfer learning*: it transfers a general visual representation and learns a smaller task-specific mapping for CaT.

There are two broad ways to use a pretrained encoder. Fine-tuning continues to update some or all encoder weights on the new task. Freezing keeps those weights fixed and trains only newly attached layers. Fine-tuning gives the encoder more freedom to adapt but also increases the risk of overfitting or damaging useful pretrained features. Freezing reduces that risk and lowers the memory required to store gradients during training, but it prevents the encoder from correcting a representation that is poorly matched to the new domain.

The two-stage process is illustrated in Figure 2.3. Pretraining learns the encoder on a large source corpus. For the CaT task, ROD copies those weights, keeps them fixed, and updates only the decoder that converts the retained features into a traversability mask.

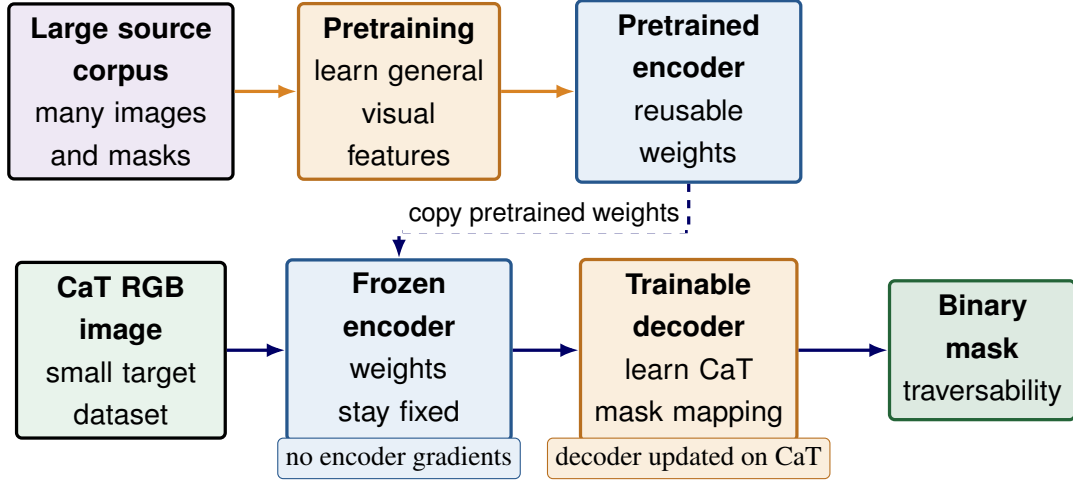


Fig. 2.3. Frozen-encoder transfer learning used by ROD. Blue blocks retain pretrained weights; the orange decoder is the part trained for the CaT target. Freezing changes training, but the encoder still executes for every image. Original schematic drawn for this thesis.

2.6.2. From Vision Transformers to EfficientSAM

A vision transformer divides an image representation into a grid of *tokens*. A token is a feature vector describing one image patch or one location in an intermediate feature map. Attention allows each token to build a weighted combination of information from other tokens. In principle, a location on one side of a track can therefore use evidence from a distant continuation of that track directly [23].

The benefit comes with a cost. Comparing many token pairs can be expensive, especially when the spatial grid is large. Efficient transformer designs reduce token resolution, use hierarchical stages, or otherwise simplify the attention computation. EfficientSAM is an efficient segmentation model whose encoder is trained to retain useful representations for mask prediction [22]. The ViT-S name used here means the small vision transformer variant; “small” is relative to larger transformer variants and does not mean that its deployment cost is negligible.

EfficientSAM belongs to the broader family of promptable segmentation models popularised by the Segment Anything Model [24]. Such models are designed to produce masks using guidance such as points or boxes. ROD does not use EfficientSAM as an interactive tool. It reuses the pretrained image encoder as a source of multi-scale visual features and learns a decoder for automatic off-road freespace prediction.

2.6.3. What ROD Changes

ROD stands for RGB-Only Fast and Efficient Off-road Freespace Detection [21]. Earlier multimodal freespace approaches can use RGB appearance together with geometry derived from LiDAR. LiDAR measures distance using emitted light and can provide a three-

dimensional point cloud. Surface normals derived from that geometry describe local surface orientation, which helps identify slopes and obstacles. The cost is an additional sensor, calibration between camera and LiDAR, synchronisation, and a more complex processing chain.

ROD removes the LiDAR-derived branch and predicts from RGB alone. Its EfficientSAM ViT-S encoder remains frozen. A convolutional decoder reads features from more than one encoder scale, combines them, and learns the target mask. “RGB-only” therefore describes the sensor input, not a claim that the model is a small CNN. Every image still passes through the complete transformer encoder.

2.6.4. How ROD Is Used in This Thesis

ROD is relevant because it represents a different strategy from PIDNet-S. PIDNet-S trains a purpose-built compact segmentation network on CaT. ROD retains a larger pretrained representation and trains only the decoder. The comparison asks whether strong frozen features compensate for the small dataset, and what runtime cost accompanies that transfer.

Freezing reduces the number of *trainable* parameters, but not necessarily the number of operations at inference. Gradients are unnecessary during deployment, yet the encoder must still calculate its feature maps for every frame. A frozen transformer can therefore be economical to train and still be slower than a fully trainable compact CNN at runtime.

The published ROD result and RTX A6000 timing cannot be copied directly into a CaT comparison. The dataset, target definition, hardware, software, and timer scope differ. In addition, the publication does not specify every item needed to recreate its training run, including the epoch budget, augmentation policy, and complete timing procedure. Chapter 5 consequently describes the implementation in this thesis as a reconstructed ROD configuration. Documented elements are retained, missing choices are stated, and the model is evaluated through the same CaT split and metric code as the other architectures.

2.7. CaT Literature and Comparability

2.7.1. Why Two Results From CaT May Still Measure Different Tasks

Using the same dataset name does not make two headline scores directly comparable. The studies must also agree on which pixels belong to each class, which images are used for training and testing, how predictions are resized, and how the final metric is calculated.

Intersection over Union (IoU) measures the overlap between a predicted region and its target. The intersection contains pixels included in both regions. The union contains pixels included in either region. Mean IoU (mIoU) averages class IoUs. If four classes are replaced by two merged classes, both the regions and the terms being averaged change. A four-class mIoU is therefore not a harder or easier version of exactly the same number; it

is a metric for a different label space.

The distinction is especially important in CaT. Its colour-coded regions represent nested vehicle capability. A blue-only binary policy asks whether a sedan can traverse a pixel. A blue+green policy also accepts the pickup region. A policy that merges blue, green, and red asks whether any represented vehicle can traverse the pixel. All three policies use CaT, but a positive prediction has a different physical meaning in each one.

2.7.2. The Original Four-Class CaT Evaluation

The original CaT study evaluates the four released classes separately using PSPNet with several ResNet encoders [1, 14, 15]. ResNet introduced residual connections that let a block learn a change relative to its input rather than an entirely new representation. The numbered variants contain different depths; a deeper backbone generally has more capacity and computation.

PSPNet then adds multi-scale pooled context to the backbone features before producing the segmentation. Table 2.3 reproduces the reported test IoUs. The table is useful for showing that the released images support strong pixel-level prediction and for identifying the difficulty of individual capability bands. It is not a binary leaderboard for the experiments in this thesis.

TABLE 2.3. Original four-class CaT test results reported by Sharma et al. [1]; values are percentages.

Network	Backbone	Background	Sedan	Pickup	Off-road	mIoU
PSPNet	ResNet-18	94.60	90.44	66.62	79.71	82.84
PSPNet	ResNet-34	94.79	91.22	68.65	80.52	83.79
PSPNet	ResNet-50	94.63	90.70	67.40	80.00	83.18
PSPNet	ResNet-101	95.00	91.65	69.08	81.00	84.18

2.7.3. EfficientViT Results and Split Independence

Faykus et al. evaluate an EfficientViT architecture on CaT and report a 98.22% sedan IoU and a 94.44% mean over the three traversability classes [25]. EfficientViT is a vision-transformer family designed to reduce the cost of attention. The architecture is relevant to compact off-road perception, but the published CaT data accounting prevents a direct ranking against this thesis.

The paper describes 3,624 images. The public release audited here contains 1,812 underlying images, with each sample accessible twice: once through its environment folder and once through the consolidated mixed folder. Thus, counting file paths instead of underlying samples also produces 3,624. The reported train and test counts do not sum to the stated total, and no list of sample identifiers is published for either partition.

This does not prove that training and test images overlap. It means that independence cannot be checked. If two paths to the same underlying image were placed in different partitions, the model could be evaluated on pixels it had already seen during training. Because the identifiers are unavailable, this thesis records the uncertainty and does not assign a cause or reconstruct the authors’ split.

2.7.4. Binary CaT With a Different Positive Class

Ramtekkar et al. formulate CaT as a binary road-detection task [26]. However, they merge the sedan, pickup, and off-road-only regions into the positive road class. Their positive region is therefore broader than either binary policy studied here.

Their system uses the MiT-B3 encoder from the SegFormer family and contains approximately 64 million parameters. It is trained using a generated multi-source corpus of 2.7 million images. This additional data can expose the model to far more appearance variation than the CaT training partition alone. The work answers a useful question about robust road-region detection, but it does not isolate learning from the public CaT training set under a sedan-oriented or pickup-inclusive policy.

The reported road IoU should consequently not be placed beside the mIoU values in Chapter 7 as though the models took the same examination. The class definition, training corpus, architecture scale, and metric differ.

Table 2.4 collects these protocol differences and states which kinds of comparison remain valid.

TABLE 2.4. Why headline CaT results require protocol alignment before comparison.

Study	Target	Principal comparability issue
Sharma et al. [1]	Four mutually exclusive raw labels encoding vehicle-capability bands	Four-class optimisation and mIoU differ from merged binary evaluation
Faykus et al. [25]	Three traversability-class mean	Published sample counts and absent manifest do not establish an independent split
Ramtekkar et al. [26]	Blue+green+red as road	Different positive class and much larger generated training corpus
This thesis	Blue-only or blue+green binary	Official test retained; exact binary policy, split, seeds, and artifacts declared

2.7.5. The Comparison Rule Used Here

Only the sedan IoU from the original four-class study uses the same positive pixel set as the blue-only traversable IoU. Even that number is not fully equivalent: the original network must distinguish all four classes during training, whereas the binary network only separates traversable from untraversable.

The literature is therefore used in two ways. First, it explains how CaT has been interpreted by previous systems. Second, it identifies protocol choices that must be stated for a reproducible comparison. Numerical ranking is reserved for models evaluated here with the same binary mask policy, split manifest, evaluator, and timing definition. This narrower rule produces fewer headline comparisons, but the remaining comparisons have a clear meaning.

2.8. Research Gap

The background above leads to four practical gaps. Each gap is expressed here as a question that the experimental chapters can answer.

2.8.1. What Does “Traversable” Mean?

CaT does not provide one universal road class. It provides nested regions for vehicles with different capabilities. A result has little operational meaning unless it states which of those regions are treated as positive. This thesis therefore defines two explicit binary policies and makes the blue+green policy the principal one. Every reported experiment names its policy.

2.8.2. What Kind of Mistake Did the Model Make?

One mIoU value combines several outcomes. For navigation, predicting an untraversable pixel as traversable is qualitatively different from blocking a pixel that was actually traversable. The first error can make terrain labelled untraversable available to the planner; the second can remove a valid route and make the vehicle unnecessarily conservative. This thesis reports False-Safe Rate and False-Block Rate alongside overlap metrics, and it inspects where those errors occur in the image.

2.8.3. Is a Model Fast on the Hardware That Will Run It?

Published frames-per-second values often use different processors, input sizes, numeric precision, batch sizes, and timer boundaries. One measurement may time only the neural-network forward pass, while another includes image preparation and transfer to the GPU.

The resulting numbers are not interchangeable. This thesis measures the selected models on declared hardware and states exactly which operations each timer includes.

2.8.4. Can the Result Be Traced and Repeated?

A checkpoint alone does not describe an experiment. Reproduction also needs the file partition, label conversion, image size, preprocessing, training configuration, selection rule, threshold, and evaluation code. The work therefore saves manifests, configurations, checkpoints, metric outputs, and timing records as linked artifacts.

These gaps explain why the thesis does more than introduce another segmentation network. Its contribution is a common, inspectable experiment in which compact architectures are trained for the same vehicle-level task, evaluated with both accuracy and safety-oriented measures, and timed under declared deployment conditions. Chapter 3 next defines the data and label pipeline on which that comparison depends.

3. CaT Binary Task and Common Pipeline

3.1. Dataset Release and Audit

The first implementation task was not model training but establishing what the dataset actually contains. The CaT release [1] exposes each underlying sample in an environment-specific directory and again in a consolidated mixed directory. Treating both directory views as independent images would double the nominal sample count and could place identical scenes in different partitions. The pipeline therefore consumes only the consolidated tree and treats the environment folders as alternate views useful for audit.

The release contains Brown Field, Main Trail, and Power Line environments. Their cameras and stored image resolutions are not uniform. Brown Field images are 1024×644 , Main Trail images are 960×604 , and Power Line contains both 720×452 and 1920×1208 images. Table 3.1 gives the exact release counts. The Power Line environment contributes 78.5% of the data, so an aggregate test score is influenced more strongly by its appearance distribution than by the two smaller environments.

TABLE 3.1. Audited CaT environment counts and stored RGB resolutions.

Environment	Train release	Test release	Total	Stored resolution(s)
Brown Field	140	60	200	1024×644
Main Trail	133	57	190	960×604
Power Line	995	427	1,422	720×452 ; 1920×1208
Consolidated unique total	1,268	544	1,812	all of the above

For every consolidated sample, the audit pairs three files by numeric sample identifier:

1. the RGB PNG in `imgs`;
2. the coloured palette mask in `masks`;
3. the integer label map in `annos/int_maps`.

The audit validates that the expected files are present, readable, and spatially aligned. All 1,812 selected samples have an exact integer-map pair; the palette-mask fallback exists in the implementation but is not used for the processed datasets reported here. The audit confirms that the label maps contain only the expected raw IDs $\{0, 1, 2, 3\}$ and that the palette masks use only the four expected colours. It writes both a machine-readable report and the discovered mapping configuration before any binary mask is generated.

This procedure was chosen over directly reading the colour masks during training. Integer IDs are unambiguous and avoid dependence on PNG palette interpretation. Retaining

a validated colour fallback makes the tool robust to a sample for which the integer map is absent, but silently mixing an unaligned map with an image is forbidden.

3.2. Vehicle-Capability Labels and Binary Policies

CaT defines four nested capability regions:

- raw ID 0, black (0, 0, 0): background or untraversable;
- raw ID 1, blue (27, 122, 235): sedan-traversable;
- raw ID 2, green (56, 162, 4): additional pickup-traversable terrain;
- raw ID 3, red (245, 34, 45): additional terrain reserved for the off-road vehicle.

The colours are annotation colours, not terrain appearances. “Blue” and “green” are therefore convenient policy names rather than physical surface categories.

The main development policy merges sedan and pickup capability:

$$y_{BG} = \begin{cases} 1, & r \in \{1, 2\}, \\ 0, & r \in \{0, 3\}, \end{cases} \quad (3.1)$$

Here, $r \in \{0, 1, 2, 3\}$ is the raw CaT class ID and $y_{BG} \in \{0, 1\}$ is the resulting binary label, with 1 denoting traversable and 0 denoting untraversable. This policy asks whether a pixel lies inside the combined sedan/pickup envelope. Raw off-road-only pixels are deliberately blocked; calling every red pixel “road” would represent a more capable vehicle and a different task.

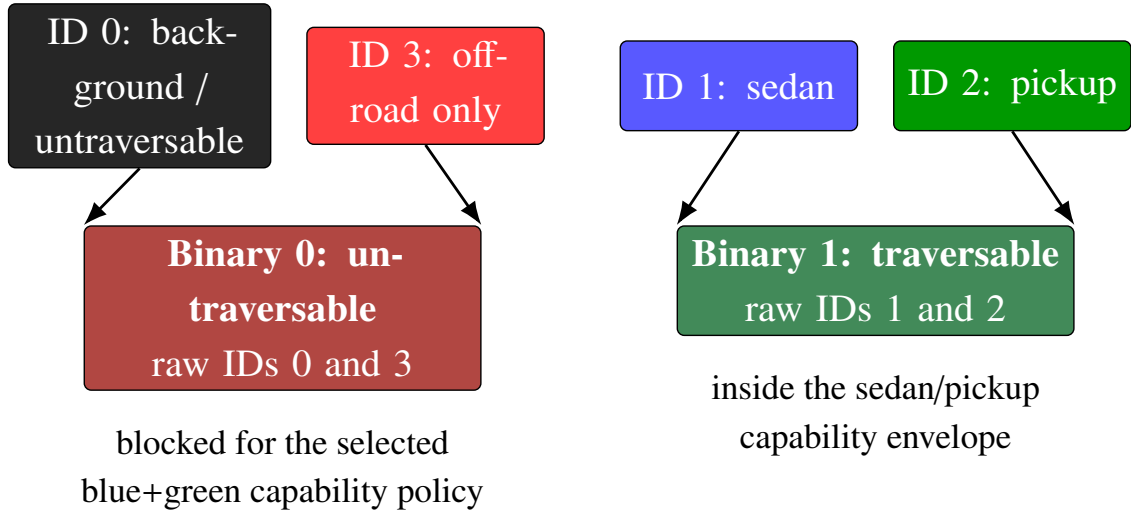


Fig. 3.1. Conversion from the four raw CaT classes to the main blue+green binary policy. The class merge is a vehicle-capability decision, not a claim that the two merged raw classes are visually or physically identical. Raw classes are grouped by binary destination rather than numerical ID.

Figure 3.1 visualises how the four raw IDs enter the two destination classes defined by Equation 3.1.

The architecture experiment also evaluates a blue-only policy:

$$y_B = \begin{cases} 1, & r = 1, \\ 0, & r \in \{0, 2, 3\}. \end{cases} \quad (3.2)$$

Here, $y_B \in \{0, 1\}$ is the blue-only binary label, with 1 denoting traversable and 0 denoting untraversable; r retains the raw-ID definition above. Blue-only is more conservative because pickup-only terrain is reassigned to the blocked class. Its scores answer a different question from the blue+green scores because the positive mask, class balance, and assumed vehicle all change.

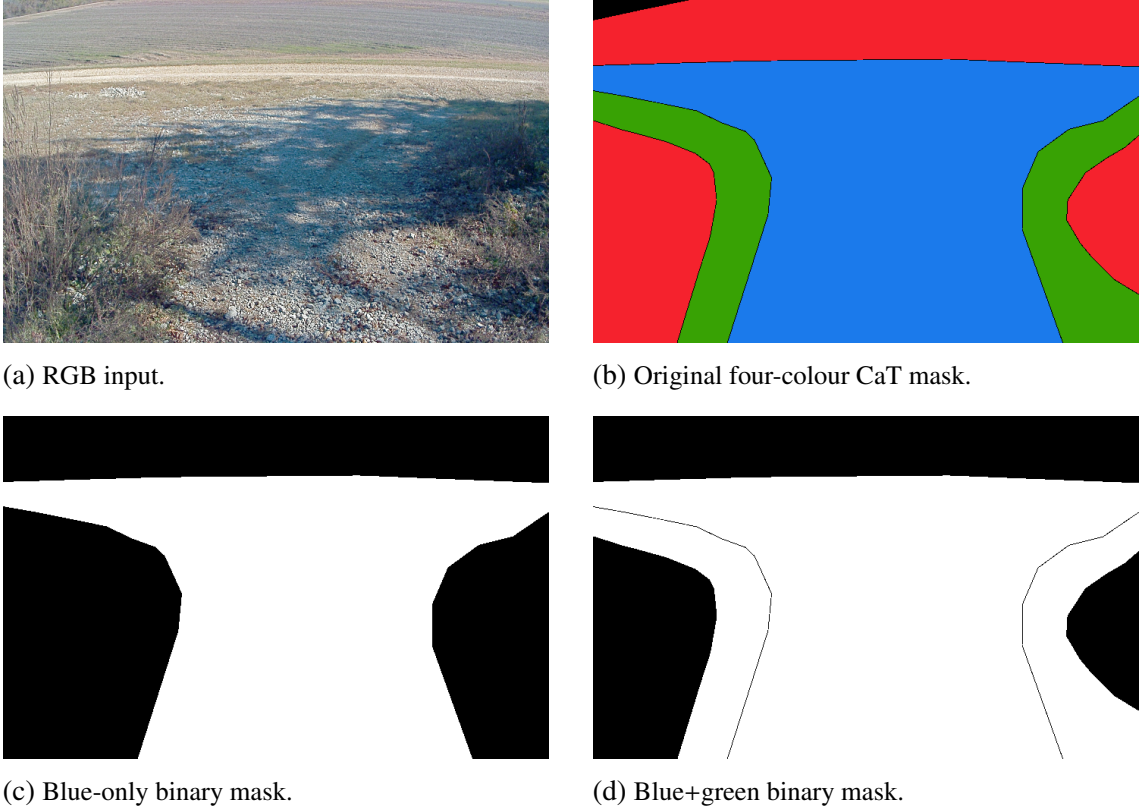


Fig. 3.2. Example of the two binary conversions for `mixed_pln_1278`. In the lower masks, white denotes traversable and black denotes untraversable. Blue-only retains the central raw blue region; blue+green also admits the raw green shoulders. The upper raw red field remains blocked in both policies.

Figure 3.2 also illustrates why the mapping must be reported with every result. A binary visual can look plausible without revealing whether green or red source regions were merged. The preprocessing configuration therefore stores the traversable palette names and raw IDs, and the processed summary records the mapping actually applied.

3.3. Train, Validation, and Test Separation

The original CaT test partition of 544 images is preserved intact and excluded from gradient updates and automatic checkpoint selection. It is used for read-only benchmark evaluation across several experiments, so it is not a single-use blinded test set; Chapter 6 states this limitation explicitly. The original 1,268-image training partition is divided into train and validation subsets by a deterministic hash:

$$b(s) = \frac{\text{int}(\text{SHA256}(\text{join}(\text{seed}, \text{dataset}, s))_{0:12}, 16)}{16^{12}}, \quad (3.3)$$

Here, s is the complete sample identifier and $b(s)$ is its normalised hash bucket, for which $0 \leq b(s) < 1$. The SHA-256 input is the UTF-8 string $\text{join}(\text{seed}, \text{dataset}, s)$, where join concatenates the split seed, dataset name, and complete sample identifier with colons. The subscript $0:12$ selects the first 12 hexadecimal characters; $\text{int}(\cdot, 16)$ converts that substring from base 16; division by 16^{12} normalises the integer to $[0, 1)$. The sample is assigned to validation if $b(s) < 0.2$. Split seed 1337 yields 1,002 training and 266 validation images. Hash bucketing makes the assignment stable across machines and independent of filesystem ordering. It also explains why the realised validation fraction is close to, but not exactly, 20%.

Figure 3.3 makes the information boundary explicit. The validation set is created only from the official training part and is used to select checkpoints. The official test branch remains outside optimisation and automatic checkpoint selection, while repeated read-only evaluation is shown as a separate benchmark path.

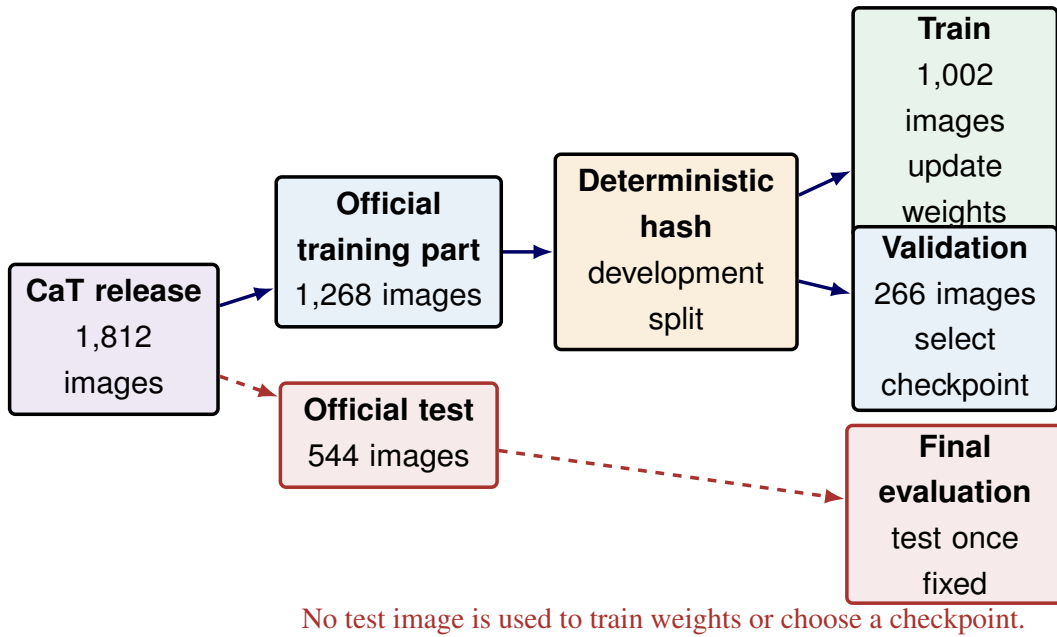


Fig. 3.3. CaT data separation used throughout the thesis. Solid blue arrows describe development data flow; dashed red arrows mark the preserved official test branch. Original schematic drawn for this thesis.

TABLE 3.2. CaT split and the role of each subset.

Split	Images	Share	Permitted use
Train	1,002	55.30%	Gradient-based weight optimisation
Validation	266	14.68%	Epoch checkpoint selection by mIoU
Test	544	30.02%	Read-only benchmark scoring; no training or checkpoint selection
Total	1,812	100.00%	

Table 3.2 records the exact image counts and prevents the roles of validation and test data from being conflated later in the thesis.

Every processed sample is entered in a JSON manifest with its split, sample identifier, processed RGB path, processed mask path, label source, and raw label path. Training and evaluation read this manifest rather than rediscovering files. The manifest therefore acts as the contract between preprocessing and all subsequent experiments.

3.4. Image and Mask Preparation

The training loader applies a small, explicit sequence of operations:

1. open the processed RGB file and convert it to three-channel RGB;
2. open the binary mask without colour conversion;
3. resize the image to the configured width and height with bilinear interpolation;
4. resize the mask with nearest-neighbour interpolation;
5. if training augmentation is enabled, apply a matched horizontal flip and image-only photometric transforms;
6. convert RGB values to floating point in $[0, 1]$, transpose to channel-first format, and convert mask values to integer class IDs.

Nearest-neighbour mask interpolation is essential. Bilinear interpolation would produce fractional values at class boundaries and turn a categorical label map into an unintended soft target. All controlled experiments use a resolution of 640×384 . This converts the different native image sizes to one tensor shape, supports batching, and is divisible by the main architectures' downsampling factors. Resizing changes the aspect ratio slightly because the native ratios are not identical. Crop-and-pad preprocessing was considered but not used, so this small distortion is part of the declared protocol.

PIDNet-S and the other ordinary controlled models consume the $[0, 1]$ tensor directly; no ImageNet mean or standard-deviation normalisation is applied by the common loader. ROD receives the same tensor, then internally resizes it to 1024×1024 and applies the

ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225] expected by EfficientSAM. This model-specific operation is described separately because it changes ROD’s computation even though its external input remains 640×384 . The mean and standard-deviation entries are ordered red, green, blue.

When enabled, augmentation uses deterministic per-sample randomness derived from the seed, sample index, and epoch. The tested recipe consists of:

- horizontal flip with probability 0.5;
- brightness, contrast, and saturation factors sampled independently from [0.9, 1.1];
- sharpness adjustment with probability 0.25 and factor [0.7, 1.3];
- Gaussian blur with probability 0.2 and radius [0.2, 1.1].

Bracketed augmentation intervals are uniform sampling ranges. Colour and sharpness factors are dimensionless multipliers, whereas blur radius is measured in pixels. Only the horizontal flip changes the mask. The reference PIDNet checkpoint disables this entire recipe after the controlled ablation showed lower validation-selected test performance when it was enabled. This result applies to the stated recipe, not to augmentation in general.

3.5. Common Software Workflow

The repository separates dataset audit, preprocessing, training, evaluation, qualitative review, inference export, and benchmarking into configuration-driven entry points. Figure 3.4 shows the lifecycle. The separation was chosen so that evaluating a saved checkpoint does not implicitly retrain it and so that a benchmark can name the exact model and processed manifest it consumes.

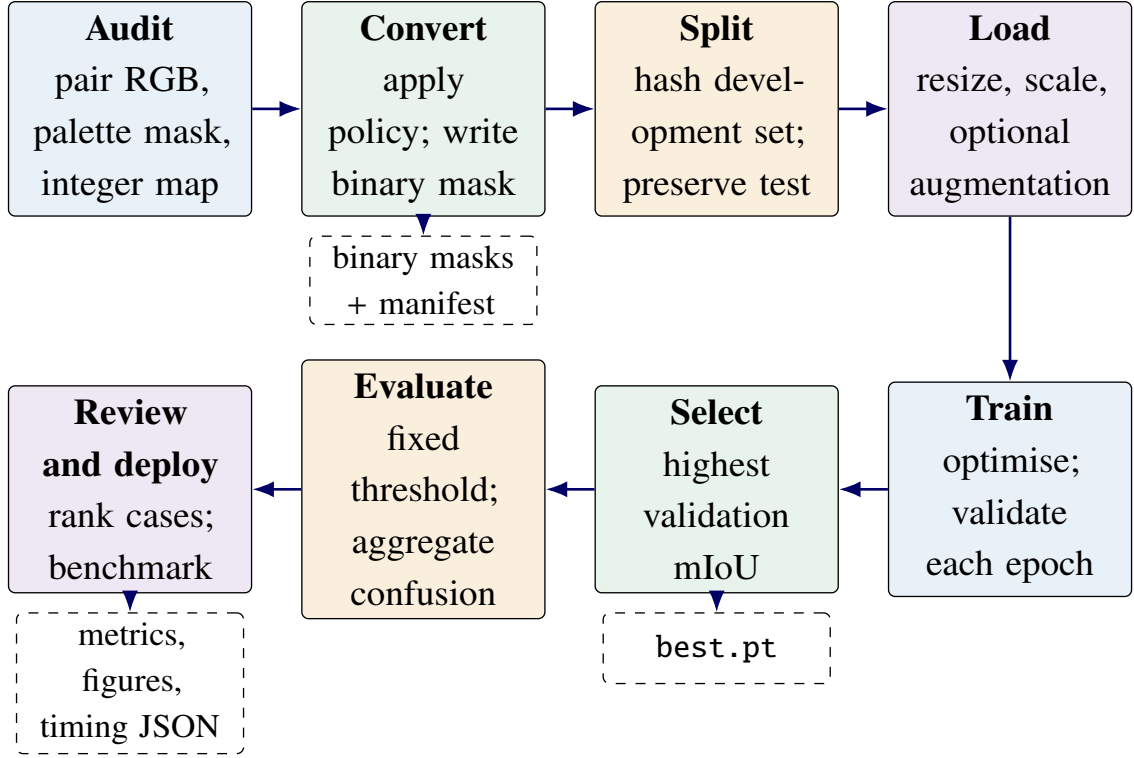


Fig. 3.4. Common implementation workflow from raw CaT files to saved evidence. Dashed boxes are persistent artifacts that allow a reported result to be traced to its data, selected checkpoint, and evaluation outputs.

During training, the data loader shuffles only the training subset. Validation is deterministic and unaugmented. At the end of each epoch the model is evaluated on all 266 validation images; `best.pt` is replaced only if validation mIoU improves. The checkpoint stores the model state, optimiser state, epoch, complete configuration, and class weights. The test evaluator loads that checkpoint, switches the model to evaluation mode, disables gradients, iterates through the test manifest without shuffling, and accumulates one 2×2 confusion matrix.

The output directory for each experiment contains the training history, `best.pt`, `last.pt`, validation visualisations, test metrics, and selected review panels. Generated repeat-seed configurations are retained instead of being assembled only in memory. These decisions make the experimental procedure inspectable after training hardware is no longer available.

3.6. Annotation Ambiguity and Observable Evidence

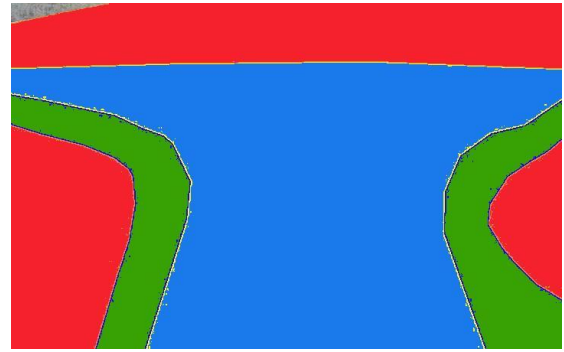
For metric computation the CaT mask is ground truth: every disagreement is counted as model error. For scientific interpretation, however, the annotation is a human judgement about vehicle capability from a scene. Several cases show why disagreement cannot always be diagnosed from RGB appearance alone.

In `mixed_pln_1278`, the annotated blue corridor continues through a pale open area, while the cultivated field at the top of the image is off-road-only and therefore blocked by the blue+green policy. The field and track share colour and texture cues in the RGB image. The label may depend on surface preparation, soil support, crop rows, or the intended route—physical information that a single frame does not establish.

In `mixed_118`, the RGB frame shows dense woodland and a leaf-covered floor with almost no visually distinct road edge. The annotation nevertheless assigns a broad, irregular foreground region to the sedan-capable class. Some of that region may correspond to a faint route visible to the annotator or in adjacent frames, but its extent is difficult to reconstruct from this image alone.



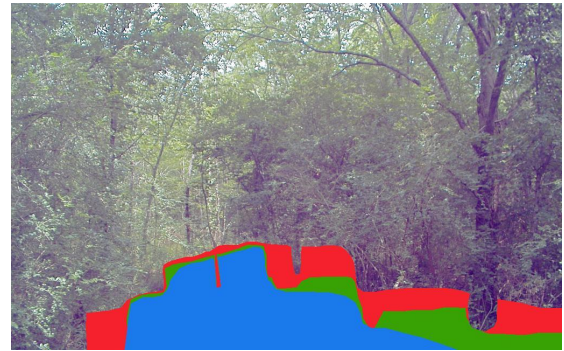
(a) `mixed_pln_1278`: RGB image.



(b) `mixed_pln_1278`: released annotation.



(c) `mixed_118`: RGB image.



(d) `mixed_118`: released annotation.

Fig. 3.5. RGB frames and released CaT annotations for the two cases discussed in the text. In `mixed_pln_1278`, the pale track and cultivated field have similar appearance despite different capability labels. In `mixed_118`, dense vegetation and leaf-covered ground provide few clear cues for the broad accepted foreground region. The annotation colours retain the original four-class CaT vehicle-capability hierarchy.

Figure 3.5 places the two discussed RGB frames beside their released annotations so that the ambiguity is visible rather than asserted only in prose.

In `mixed_459`, low vegetation and the track meet along an irregular foreground edge. Moving the line by several pixels changes the boundary-band score substantially even though more than one contour may appear plausible from RGB alone.

These examples do *not* demonstrate that the annotations are wrong. The annotator may have had route knowledge, higher-resolution context, or a vehicle-capability convention unavailable in the displayed frame. Conversely, visual plausibility does not prove physical traversability. CaT provides neither multiple independent annotations nor inter-annotator agreement, so the amount of label uncertainty cannot be estimated from the released data. The thesis can identify cases where RGB evidence is insufficient to justify a precise boundary, but it cannot calculate a noise rate or replace the supplied label with a preferred alternative.

This limitation affects both training and evaluation. The model is optimised to reproduce the annotations, and the reported FSR/FBR describe disagreement with that policy. They are not direct measurements of real vehicle incidents. The qualitative chapter therefore distinguishes clear spatial model failures from cases whose physical status would require additional evidence.

4. PIDNet-S Model Development

4.1. Selection Rationale

The initial goal was to establish a model that could be trained, evaluated, and plausibly deployed without requiring a high-end GPU. PIDNet-S was selected because its architecture explicitly separates semantic context, spatial detail, and boundary information while remaining compact [2]. This matched the main visual structure of CaT: large connected trail regions require context, small rocks and vegetation edges require detail, and capability boundaries are safety-relevant.

A conventional encoder–decoder such as U-Net offered a simpler alternative, whereas a large transformer offered greater representation capacity. Neither addressed the initial latency constraint as directly. Starting with a purpose-built real-time network made it possible to develop the complete pipeline around a compact deployment candidate. The comparative experiments in Chapter 5 subsequently test whether this choice sacrifices accuracy relative to pretrained encoders and other decoder families.

The model is trained from scratch. No Cityscapes or ImageNet checkpoint is loaded. This keeps the baseline self-contained, but it also means later comparisons with pretrained encoders measure complete model recipes rather than architecture alone.

4.2. Implemented Architecture

PIDNet’s control-system terminology is an analogy. The network does not calculate a proportional–integral–derivative control command. Its three branches instead divide the segmentation problem:

- the **P branch** preserves local appearance and spatial resolution;
- the **I branch** accumulates semantic context on progressively smaller feature grids;
- the **D branch** retains high-frequency differences that help locate region boundaries.

After a shared stem, the P and D paths remain at one eighth of the input resolution. The I path reaches one sixteenth, one thirty-second, and one sixty-fourth resolution. PagFM modules compare P- and I-path features and selectively inject semantic context into the detail path. Difference features from the semantic stages update the D branch. PAPPm pools context at multiple scales at the end of the I path. Light-Bag then uses boundary attention from the D branch to fuse the spatial and semantic representations [2].

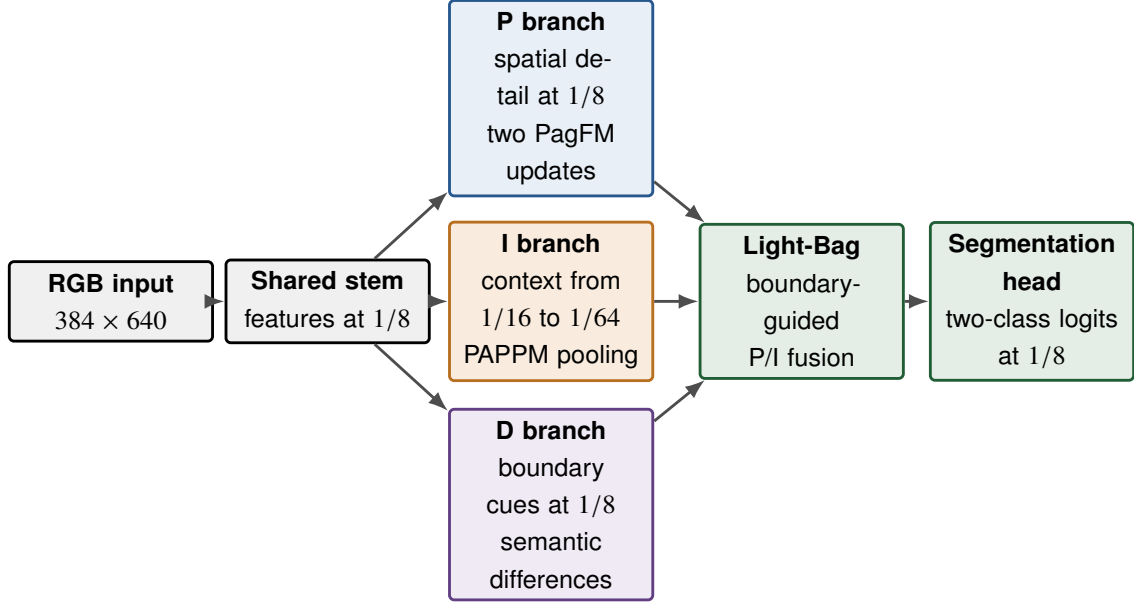


Fig. 4.1. Conceptual PIDNet-S flow. The P branch retains spatial detail, the I branch extracts context, and the D branch supplies boundary cues. PagFM, PAPPM, and Light-Bag exchange and fuse those representations before the segmentation head. Original schematic drawn for this thesis and adapted from Xu et al. [2].

Figure 4.1 shows the three processing branches and the points at which they exchange and fuse features.

The implementation uses the PIDNet-S schedule $(m, n, C) = (2, 3, 32)$, with 96 pyramid-pooling channels and 128 head channels. C is the base width; m and n control residual-stage repetition. Table 4.1 records the principal shapes for the 640×384 input. Listing these shapes is useful both for implementation verification and for understanding where computation is spent.

TABLE 4.1. Principal implemented PIDNet-S feature shapes at 640×384 .

Component	Spatial size	Channels	Purpose
Shared stem output	80×48 (1/8)	64	Common input to the three paths
P branch	80×48 (1/8)	$64 \rightarrow 128$	Preserve detail; receive two PagFM updates
I stage 3	40×24 (1/16)	128	First deeper semantic stage
I stage 4	20×12 (1/32)	256	Broader contextual representation
I stage 5	10×6 (1/64)	512	Deepest semantic features before PAPP
PAPP output	80×48 (1/8)	128	Multi-scale context returned for fusion
D branch	80×48 (1/8)	$32 \rightarrow 64 \rightarrow 128$	Boundary-sensitive representation
Light-Bag output	80×48 (1/8)	128	Boundary-guided P/I fusion
Binary head	80×48 (1/8)	2	Native untraversable/traversable logits
Wrapper output	640×384	2	Bilinearly upsampled full-resolution logits

Auxiliary P- and D-head outputs are disabled, so the network exposes one final two-class logits tensor and has 7,623,522 trainable parameters. In the vendored PIDNet implementation this architectural choice is named `augment=False`. It is unrelated to the data-loader `augment` flag. Both happen to be false for the retained checkpoint, but they control different operations.

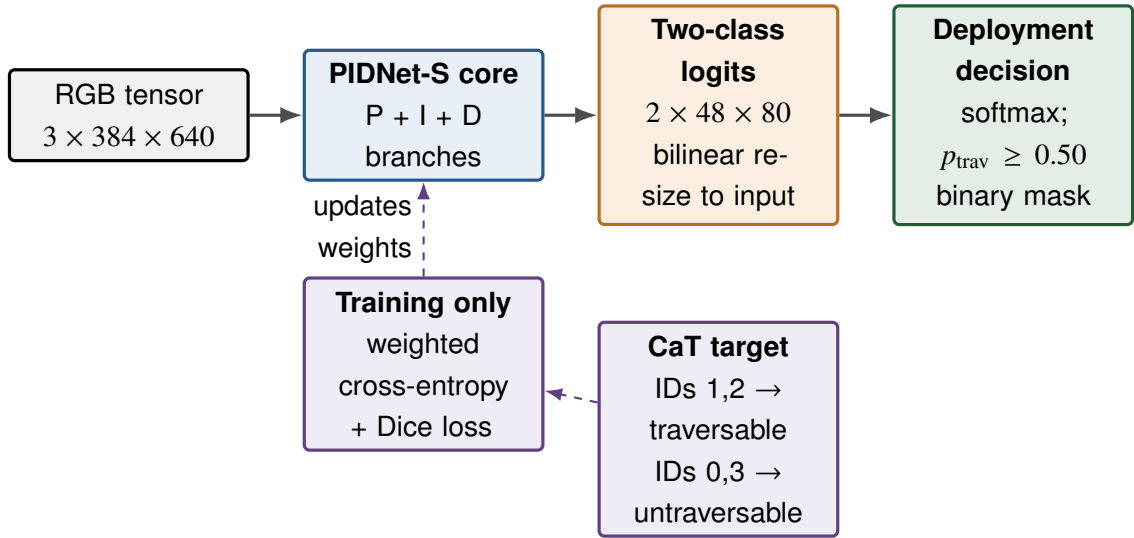


Fig. 4.2. Implemented PIDNet-S binary pipeline. The one-eighth-resolution logits are returned to the input resolution before the training loss or deployment decision is calculated. The lower dashed path is active only during training. Original schematic drawn for this thesis; the core network follows Xu et al. [2].

Figure 4.2 then connects that core network to the two-class training and thresholding operations implemented here.

For logits z_{i0} and z_{i1} at pixel i , the traversable probability is

$$p_i = \frac{\exp(z_{i1})}{\exp(z_{i0}) + \exp(z_{i1})}. \quad (4.1)$$

Here, i indexes a pixel, z_{i0} and z_{i1} are its untraversable and traversable logits, $\exp(\cdot)$ is the exponential function, and p_i is the normalised probability assigned to the traversable class. The other softmax output, $1 - p_i$, is the untraversable probability. The reference binary prediction is $\hat{y}_i = \mathbb{I}[p_i \geq 0.50]$. The constant 0.50 is the fixed probability threshold, and equality is assigned to the traversable class. No morphological cleanup, connected-component filtering, or temporal smoothing is applied. The output therefore represents the model and threshold directly.

4.3. Objective, Optimiser, and Checkpoint Selection

The main PIDNet objective combines weighted cross-entropy and soft Dice loss. For classes $c \in \{0, 1\}$ and their corresponding weights w_c , the cross-entropy term is

$$\mathcal{L}_{\text{WCE}} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} \log p(y_i | x). \quad (4.2)$$

In this expression, $\mathcal{L}_{\text{WCE}}$ is weighted cross-entropy, N is the number of pixels included in the loss, i indexes those pixels, $y_i \in \{0, 1\}$ is the target class, w_{y_i} is the weight of that class, and $p(y_i | x)$ is the probability assigned to the correct class for input x . The natural logarithm is denoted by $\log$. The implemented Dice term operates on the traversable probability and binary target. For one image,

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + \epsilon}, \quad (4.3)$$

Here, p_i is the traversable probability from Equation 4.1, y_i is one for a traversable target and zero otherwise, each sum covers the pixels of one image, and $\epsilon = 10^{-6}$ prevents division by zero. The resulting $\mathcal{L}_{\text{Dice}}$ is zero for perfect overlap and approaches one as overlap is lost. The constant factor 2 is the conventional Dice normalisation: when prediction and target coincide, the doubled intersection equals the sum of their two sizes. The batch mean is used. Dice complements per-pixel cross-entropy with a region-overlap signal [27]. The default total is

$$\mathcal{L} = \mathcal{L}_{\text{WCE}} + \mathcal{L}_{\text{Dice}}. \quad (4.4)$$

Thus, $\mathcal{L}$ is the scalar objective differentiated during the default training run, and both component losses enter with unit coefficient.

Class weights are computed from inverse training-pixel frequency. Before normalisation, the untraversable weight is multiplied by 2.5 and the traversable weight by 0.75. The two final weights are rescaled to sum to two. For the fixed split they are

[1.2606075, 0.7393925]. The intent was to discourage false-safe predictions by making errors on the blocked class more expensive. Because loss weighting changes optimisation rather than directly optimising FSR, its actual effect must be established experimentally.

An optional false-safe probability penalty was also implemented:

$$\mathcal{L}_{\text{FS}} = \frac{\sum_i p_i \mathbb{I}[y_i = 0]}{\sum_i \mathbb{I}[y_i = 0]}. \quad (4.5)$$

i indexes the same N loss pixels as above, and both sums run over those pixels. The indicator $\mathbb{I}[y_i = 0]$ selects annotated untraversable pixels, so the numerator sums their predicted traversable probabilities and the denominator counts them. Consequently, $\mathcal{L}_{\text{FS}}$ is the mean traversable probability over the blocked class. With non-negative coefficient λ_{FS} , the tested objective becomes $\mathcal{L} + \lambda_{\text{FS}}\mathcal{L}_{\text{FS}}$; the ablation uses $\lambda_{\text{FS}} = 0.35$. This term pushes traversable probability down over all annotated negative pixels. It was a plausible alternative to class weighting, but the controlled experiment later showed that the tested coefficient did not improve the desired rate.

The retained training settings are given in Table 4.2. AdamW supplies decoupled weight decay [28]. Cosine annealing reduces the learning rate from 2.5×10^{-4} toward 10^{-5} over 15 epochs. Bfloat16 automatic mixed precision is used on CUDA when supported, while model parameters and optimiser state remain stored in FP32.

TABLE 4.2. Training configuration of the reference PIDNet-S model.

Setting	Implemented value and function
Initialisation	Random PIDNet-S weights; no pretrained checkpoint
Input and batch	640×384 RGB; batch size 4
Epochs	15 complete passes over 1,002 training images
Optimiser	AdamW, initial learning rate 2.5×10^{-4} , weight decay 10^{-4}
Scheduler	Cosine annealing by epoch to minimum learning rate 10^{-5}
Objective	weighted cross-entropy + traversable soft Dice
Class weights	[1.2606075, 0.7393925] for untraversable/traversable
Data augmentation	disabled
Mixed precision	bfloat16 autocast on supported CUDA hardware
Seed and workers	seed 1337; zero loader worker processes in the portable config
Selection	save the epoch with maximum validation mIoU
Decision threshold	0.50, fixed before official test evaluation

After each epoch, all 266 validation images are evaluated without augmentation or shuffling. A new `best.pt` is written only if validation mIoU increases. The retained checkpoint comes from epoch 15, where validation mIoU is 0.88834. This rule prevents the 544-image test score from participating in model selection.

4.4. Exploratory Development

The earliest experiments were iterative engineering runs. They are useful because they show how the system changed, but they are not single-factor ablations: several settings change together. Table 4.3 lists both configuration and test outcome so that the later controlled study is not confused with this exploration.

TABLE 4.3. Exploratory PIDNet-S development runs. Values are test results; the runs are confounded and do not estimate individual causal effects.

Run	Main change(s)	Policy	mIoU	FSR	FBR	Thr.
default	Initial pipeline, five epochs	Blue	0.8374	7.24%	7.92%	0.50
conservative	15 epochs and biased class weights	Blue	0.8735	3.89%	9.67%	0.50
third_run	720 × 448, false-safe term, 18 epochs	Blue	0.8219	2.32%	21.42%	0.58
fourth_run	Remove false-safe term; retain higher resolution	Blue	0.8544	4.08%	12.30%	0.53
blue_green_best	Change policy; 640 × 384, 15 epochs	Blue+green	0.8780	4.87%	8.34%	0.50

The five-epoch run established that the complete pipeline functioned but left clear room for learning. Extending training and biasing the loss reduced FSR while increasing FBR, suggesting a more conservative operating point. The third run combined higher resolution, an explicit false-safe penalty, and a 0.58 threshold. Its low FSR was accompanied by a very high 21.42% FBR and lower mIoU. Because three factors changed, it was impossible to identify the cause. The fourth run removed the explicit penalty but retained the higher resolution and a threshold above 0.50; it recovered part of the accuracy but remained overly conservative.

The blue+green run changed the target vehicle and returned to 640 × 384. Its result motivated using blue+green as the main operational policy, but it does not prove that merging green causes an accuracy improvement: target definition, training history, and class balance differ from the earlier blue-only runs. This interpretive constraint is retained throughout the thesis.

The exploratory phase produced two lessons. First, a low FSR can be obtained by simply blocking more pixels, so FSR must always be read with FBR. Second, multi-factor iteration can guide engineering but cannot support a claim about which setting caused the change. Those lessons led to the controlled ablation.

4.5. Controlled Configuration Ablation

The controlled reference uses the blue+green policy, PIDNet-S, 640 × 384, batch size 4, 15 epochs, seed 1337, augmentation enabled, biased class weights, CE+Dice, and threshold 0.50. Four variants each change one factor:

1. add the false-safe penalty with weight 0.35;
2. disable the complete tested augmentation recipe;
3. increase input to 720×448 and reduce batch size to 2 for memory;
4. replace the computed class weights with neutral $[1, 1]$.

The alternatives were chosen to answer practical questions arising from the exploratory work. The penalty tests whether FSR can be reduced inside the loss rather than at the threshold. Disabling augmentation tests whether the synthetic appearance variation matches CaT. Higher resolution tests whether thin boundaries are limited by input detail. Neutral weights test whether the manually biased imbalance correction is necessary.

TABLE 4.4. Single-seed controlled PIDNet-S ablation. Bold identifies the configuration retained for subsequent analysis. Lower FSR and FBR are preferable.

Run	One changed factor	mIoU	FSR	FBR
Reference	None	0.8667	5.89%	8.48%
False-safe loss	Penalty coefficient 0.35	0.8177	7.45%	12.95%
Augmentation off	Disable tested flip and photometric recipe	0.8939	4.32%	7.04%
Higher resolution	720×448 , batch size 2	0.8632	5.62%	9.28%
Neutral weights	Class weights $[1, 1]$	0.8848	5.28%	6.95%

Table 4.4 supplies the test result for each one-factor change and marks the retained configuration.

Disabling the tested augmentation improves mIoU by 2.72 percentage points and reduces both asymmetric error rates relative to the reference. The result suggests that the colour, blur, and sharpness variations used here do not represent helpful additional CaT variation. It cannot establish that a different geometric, weather, or sensor-specific augmentation would also be harmful.

The false-safe penalty is discarded because it worsens mIoU, FSR, and FBR. Its gradient pushes traversable probability downward on every negative pixel, but that global pressure does not guarantee a better thresholded operating point after optimisation. Higher resolution is also discarded: it increases pixel computation by 31.25%, halves the batch size, and does not improve any headline metric. Neutral weights produce the lowest FBR but a higher FSR and lower mIoU than augmentation-off. The augmentation-off checkpoint is selected by validation mIoU. Settings from different rows are not combined after examining their test results.

Because each row has one seed, the differences provide engineering evidence rather than estimates of population effects. The separate three-seed architecture experiment measures repeatability for complete model recipes; it does not change the statistical scope of this ablation.

4.6. Threshold as an Operating Control

A threshold sweep was conducted on the earlier `blue_green_best` checkpoint from 0.35 to 0.65. It was not used to alter the reference checkpoint’s 0.50 threshold. Table 4.5 shows the expected monotonic trade: increasing the threshold reduces false-safe openings and increases false blocks.

TABLE 4.5. Post-hoc threshold sweep on the earlier `blue_green_best` checkpoint.

Threshold	mIoU	FSR	FBR
0.35	0.8774	6.14%	6.78%
0.40	0.8781	5.68%	7.29%
0.45	0.8782	5.26%	7.81%
0.50	0.8780	4.87%	8.34%
0.55	0.8774	4.50%	8.89%
0.60	0.8763	4.15%	9.49%
0.65	0.8747	3.80%	10.14%

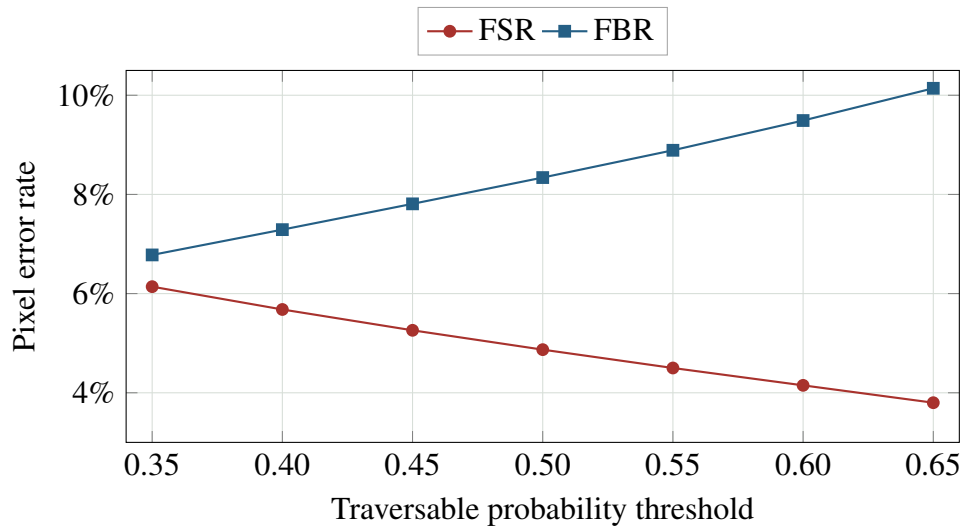


Fig. 4.3. False-safe and false-block trade-off under post-hoc threshold variation. A threshold is an operating choice; it does not improve both error directions simultaneously.

Figure 4.3 makes the opposing movement of FSR and FBR across the tested thresholds directly visible.

The maximum observed mIoU occurs at 0.45, but the difference from 0.50 is only 0.0002. More importantly, the sweep uses the test set for characterisation. Choosing 0.45 from this table and reporting it as an untouched test result would leak test information into model selection. The final 0.50 operating point is retained. A deployed threshold should instead be selected on validation data using an application-defined error cost or risk constraint, then evaluated once on an independent test set.

4.7. Reference Checkpoint

All subsequent PIDNet-S error analysis, reproduction, CPU, and TensorRT measurements use the following checkpoint:

```
outputs/controlled_ablation_augment_off/checkpoints/best.pt
```

Keeping the parameters fixed preserves a direct link between the configuration, manifest, decision threshold, pixel metrics, qualitative panels, and exported engines. The checkpoint is the strongest PIDNet-S configuration observed in the controlled study, not an estimate of the architecture’s maximum possible performance. Other augmentation families, pretrained weights, boundary supervision, longer schedules, or different seed sets could produce a different result. Its value is that it identifies one model whose behaviour can be reproduced across evaluation and deployment settings.

5. ROD and Comparative Architecture Study

5.1. Motivation for Pretrained Feature Transfer

The PIDNet-S analysis exposed a recurring pattern: low-contrast corridors were sometimes removed, broad fields could be opened incorrectly, and small boundary displacements survived even in otherwise accurate masks. With only 1,002 training images, one possible remedy is to reuse visual structure learned from a much larger corpus rather than asking every feature extractor to learn from CaT alone. ROD provides a concrete instance of this strategy. It connects a trainable off-road segmentation decoder to a pretrained EfficientSAM ViT-S encoder whose weights remain fixed during task training [21, 22].

This design occupies a markedly different operating point from PIDNet-S. It introduces substantially more representation capacity and transfers pretrained features, but the full transformer still executes for every frame. Fixed weights reduce optimisation cost; they do not make inference inexpensive. Accuracy and latency must therefore be measured together.

The comparison proceeds from controlled pairwise evidence to architectural context. PIDNet-S and ROD first share the same split, seeds, epoch budget, objective, and external resolution. Seven additional systems are then trained under the same protocol. This distinguishes a gain over one reference model from a gain over the available compact alternatives.

5.2. Implemented ROD ViT-S Model

The ROD implementation contains 29,106,050 parameters. Of these, 22.35 million belong to the EfficientSAM ViT-S encoder and are frozen; approximately 6.75 million belong to the decoder and are trainable. Freezing is enforced in three ways: encoder parameters have `requires_grad=False`, the encoder is kept in evaluation mode even when the outer model enters training mode, and its forward pass is executed under `no_grad`. Batch-normalisation in the decoder remains trainable.

The external input is the same $3 \times 384 \times 640$ tensor used by the other models. Inside ROD it is bilinearly resized to 1024×1024 and normalised with ImageNet statistics. A 16×16 patch embedding produces a 64×64 grid of 384-dimensional tokens. The encoder has 12 transformer blocks with six attention heads. The learned position embedding was pretrained on a 14×14 patch grid and is bicubically interpolated to the 64×64 grid.

The released implementation is more specific than the paper’s high-level description. Rather than summing every transformer output, the decoder uses the outputs of blocks 2 through 12. Each selected feature is rearranged to channel-first form and projected from

384 to 128 channels by a 1×1 convolution followed by a 3×3 convolution. The projected features are fused sequentially through residual convolutional blocks. A post-fusion block refines the result and expands it to 256 channels.

Two residual upsampling stages create progressively larger feature maps. The decoder then resizes four 256-channel sources—the two upsampled maps, the pre-upsample map, and the encoder’s final image embedding—to one common resolution, concatenates them, and reduces the result to 256 channels. A 1×1 prediction layer produces two logits. They are finally resized to the original 640×384 external input size.

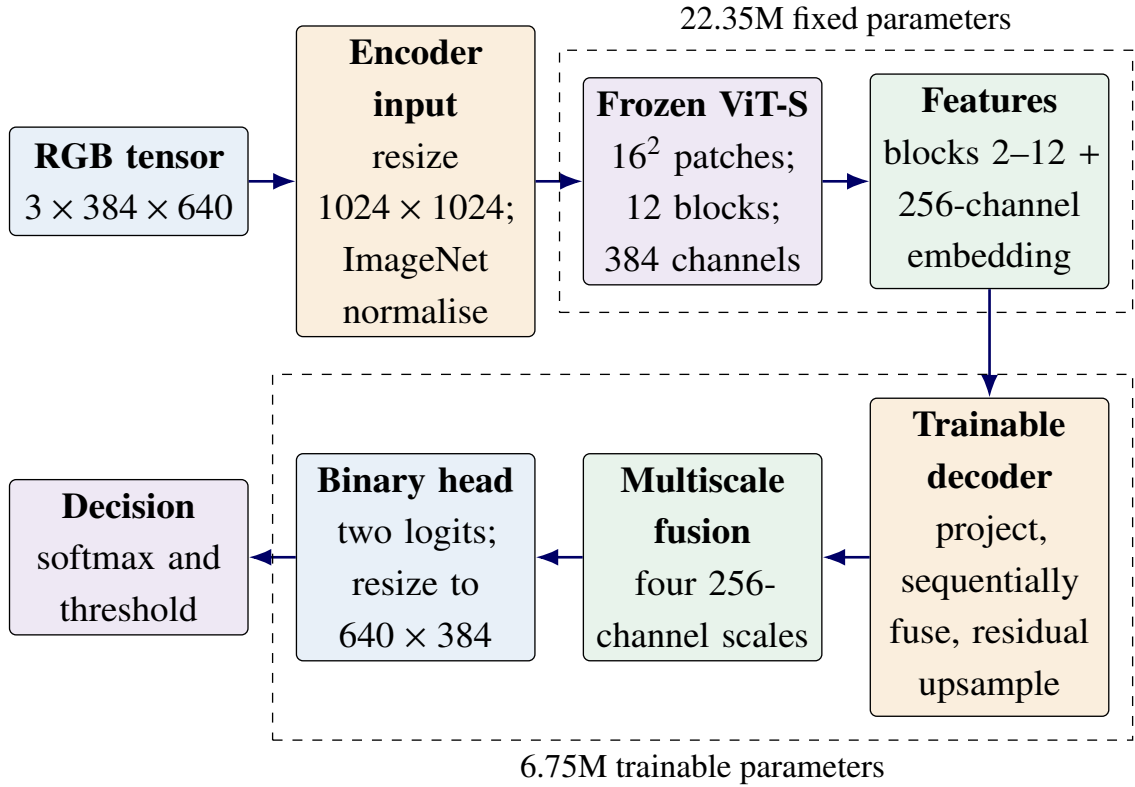


Fig. 5.1. Implemented ROD ViT-S pipeline. The pretrained encoder is frozen during CaT training but remains part of every inference. The decoder uses intermediate blocks 2–12 and multiscale convolutional fusion before the two-class head.

Figure 5.1 distinguishes the frozen feature extractor from the decoder that is updated on CaT.

Full encoder fine-tuning was an available alternative. It was not adopted because ROD’s defining transfer strategy holds the encoder fixed, the CaT training set is small, and fine-tuning would greatly increase training memory and the number of task-specific degrees of freedom. The experiment consequently does not determine whether end-to-end EfficientSAM adaptation would improve the result.

5.3. Controlled PIDNet–ROD Comparison

The matched comparison uses seeds 1337, 2027, and 4242 under both blue-only and blue+green label policies. Within a policy, PIDNet-S and ROD receive:

- the same 1,002/266/544 manifest assignment;
- the same 640×384 external tensors and batch size 4;
- 15 epochs and validation-mIoU checkpoint selection;
- AdamW at 2.5×10^{-4} with weight decay 10^{-4} ;
- epoch-level cosine annealing to 10^{-5} ;
- the same deterministic augmentation recipe;
- biased inverse-frequency class weights, CE+Dice, and threshold 0.50.

Matching these decisions prevents a favourable ROD optimiser, seed, or split from explaining the comparison. It does not make the two networks identical experiments in every respect: ROD uses pretrained frozen encoder weights, model-specific normalisation, and an internal 1024×1024 transformer input, whereas PIDNet-S is randomly initialised and operates directly on the external resolution. The result therefore compares the complete model pipelines under common task-level controls.

Population mean and population standard deviation are reported:

$$\bar{m} = \frac{1}{3} \sum_{k=1}^3 m_k, \quad (5.1)$$

$$\sigma_{\text{pop}} = \sqrt{\frac{1}{3} \sum_{k=1}^3 (m_k - \bar{m})^2}. \quad (5.2)$$

Here, m_k is the value of the reported metric in run k ; the index $k \in \{1, 2, 3\}$ corresponds to seeds 1337, 2027, and 4242; $\bar{m}$ is their arithmetic mean; and σ_{pop} is their population standard deviation. The divisor is three because the equations describe the complete declared set of runs rather than estimating variance from a random sample. With only three selected seeds, the standard deviation describes observed repeatability; it is not a confidence interval over all possible training runs.

5.4. Reconstructed ROD Training Regime

The controlled recipe answers a fair architecture-comparison question, but it does not use the optimisation choices documented for ROD. A second experiment therefore transfers the published settings as closely as the available description permits. Table 5.1 contrasts the two recipes.

TABLE 5.1. Training choices held in the controlled architecture comparison and their counterparts in the closest documented ROD recipe.

Setting	Controlled comparison	ROD paper approximation
Objective	weighted cross-entropy + Dice	cross-entropy only
Class weights	inverse frequency with 2.5/0.75 biases	neutral [1, 1]
AdamW learning rate	2.5×10^{-4}	10^{-3}
Weight decay	10^{-4}	0.01
Batch size	4	8 (validation 4)
Schedule	epoch-level cosine to 10^{-5}	iteration-level polynomial, power 0.9
Augmentation	common tested recipe enabled	disabled because none is documented
Epoch budget	15	15 retained because publication does not specify one
Threshold	0.50	0.50

The polynomial scheduler updates after every optimiser step:

$$\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^{0.9}, \quad (5.3)$$

where η_t is the learning rate after optimiser step t , $\eta_0 = 10^{-3}$ is the initial learning rate, T is the total number of optimiser steps in 15 epochs, and 0.9 is the fixed polynomial-decay exponent transferred from the documented recipe. The step index runs from $t = 0$ to $t = T$. The run uses bfloat16 mixed precision, the official EfficientSAM ViT-S checkpoint, and a frozen encoder. Because the ROD paper does not disclose its epoch budget, augmentation policy, or all timing details, this experiment is called the *closest documented recipe*. It is evidence about transfer of the reported settings to CaT, not an exact reproduction of a published CaT result.

5.5. Architecture Set

After the PIDNet–ROD comparison, seven additional compact systems were added: FPN/MobileNetV2, FPN/EfficientNet-B0, U-Net/MobileNetV2, U-Net/EfficientNet-B0, SegFormer-B0, DDRNet-23-Slim, and BiSeNetV2. With PIDNet-S and ROD this yields nine model systems. Each is trained for the same three seeds under both label policies, producing 54 scored runs.

FPN and U-Net are paired with both encoders to separate, within this limited factorial subset, encoder and decoder effects. MobileNetV2 provides the lighter convolutional encoder; EfficientNet-B0 provides a somewhat higher-capacity compact encoder. SegFormer-B0 supplies a hierarchical transformer comparison. DDRNet-23-Slim and BiSeNetV2

represent architectures designed specifically for real-time segmentation.

Table 5.2 records exact parameter counts and initialisation. FPN, U-Net, and SegFormer are created through `segmentation-models-pytorch`. DDRNet-23-Slim and BiSeNetV2 are adapted from released PyTorch implementations under their licences. Every wrapper returns one full-resolution two-class logits tensor so that the common loss and metrics see the same interface.

TABLE 5.2. Exact model systems, parameter counts, and initialisation in the comparative study.

System	Parameters	Initialisation	Implementation note
PIDNet-S	7,623,522	Random	One final head; PAPP and Light-Bag
ROD ViT-S	29,106,050	EfficientSAM ViT-S	22.35M encoder parameters frozen
FPN/MobileNetV2	4,215,554	ImageNet encoder	SMP FPN with mobilenet_v2
FPN/EfficientNet-B0	5,759,614	ImageNet encoder	SMP FPN with efficientnet-b0
U-Net/MobileNetV2	6,629,090	ImageNet encoder	SMP U-Net with mobilenet_v2
U-Net/EfficientNet-B0	6,251,614	ImageNet encoder	SMP U-Net with efficientnet-b0
SegFormer-B0	3,714,658	ImageNet MiT-B0	SMP SegFormer decoder
DDRNet-23-Slim	5,694,882	Random	Two-resolution network; no stable pretrained checkpoint used
BiSeNetV2	3,341,202	Released pretrained backbone	Auxiliary heads disabled for the common loss

Initialisation is not hidden because it is a major experimental factor on a small dataset. PIDNet-S and DDRNet start from random weights; the other seven systems use some form of pretraining, although ROD freezes its pretrained encoder while the standard ImageNet encoders are trainable. The benchmark therefore ranks the implemented recipes. It cannot isolate the causal effect of architecture from pretraining without additional within-family pretrained-versus-random experiments.

5.6. Fixed-Budget Experimental Scope

For every comparative run, the external input is 640×384 , the batch size is 4, the epoch budget is 15, and the model is selected by validation mIoU. AdamW, the learning rate, cosine schedule, common CE+Dice objective, policy-specific class weights, augmentation,

0.50 threshold, and three seeds are fixed. Blue-only and blue+green experiments write to distinct processed roots and output directories so that one policy cannot overwrite the other.

These controls support a restricted statement: under the tested CaT recipe, model A has a higher three-seed mean than model B. They do not prove that the ranking would remain under model-specific hyperparameter tuning, longer training, more seeds, another dataset, or an embedded runtime. In particular, the common 15-epoch budget may favour pretrained systems that converge quickly and disadvantage randomly initialised models.

The nine-system experiment places the pairwise result in context. ROD’s advantage over PIDNet-S remains clear, but several pretrained encoder–decoder systems achieve higher CaT overlap under the common recipe. The results are therefore interpreted as a ranking of complete training systems, with small differences read against their observed between-seed variation.

5.7. Convergence-Aware Blue+Green Follow-Up

The 15-epoch matrix answers a matched-compute question, but the saved validation histories show that it stops while several models are still improving. A second experiment therefore repeats all nine architectures and three seeds for the principal blue+green policy. It retains the same data, external resolution, augmentation, objective, class weighting, optimiser, initial learning rate, weight decay, threshold, and seed identities. Only the training horizon and stopping rule change.

The follow-up sets 300 epochs as a ceiling, not as a presumed optimum. Cosine decay runs from 2.5×10^{-4} to 10^{-5} over the first 60 epochs; the learning rate then remains at 10^{-5} . Training cannot stop before epoch 60. After that point it stops when validation mIoU has failed to improve by an absolute increase of at least 10^{-4} on the 0-to-1 mIoU scale for 25 epochs. The checkpoint writer remains slightly more sensitive than the stopping counter: it stores any strict validation-mIoU increase, even if the increase is smaller than 10^{-4} . Consequently, a run may save its numerically highest checkpoint near the stopping epoch without resetting the early-stopping counter.

This design gives every run time to pass the early learning phase while still bounding unnecessary work after the validation curve has flattened. The selected epoch is therefore called the *validation-selected checkpoint epoch*; it is not described as the universally optimal epoch for that architecture. Model-specific objectives and schedules could still change both the selected epoch and the final ranking.

6. Experimental Protocols and Evaluation Methods

6.1. Experimental Phases

The complete experimental campaign is separated into seven phases so that an exploratory observation is not mistaken for controlled evidence.

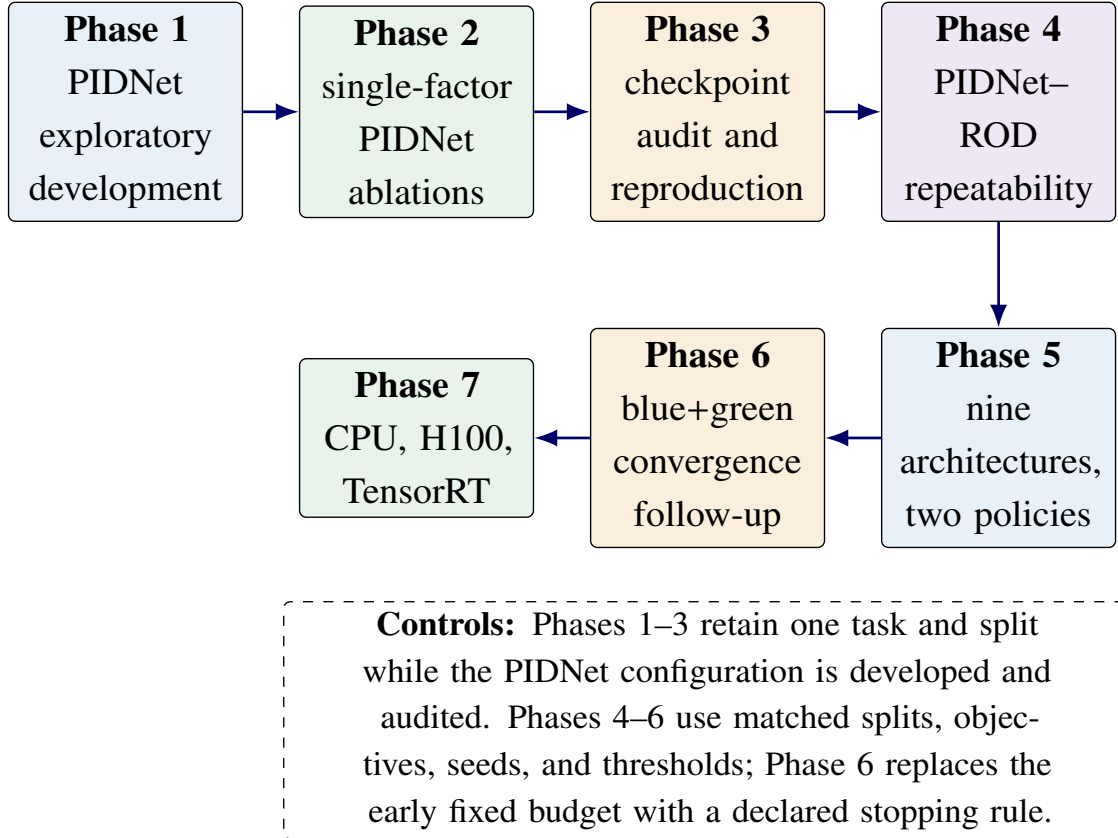


Fig. 6.1. Experimental phases and their principal controls. Dataset identity, target policy, and metric definitions remain fixed as the question changes from configuration selection to model comparison and runtime measurement.

Figure 6.1 provides the overview; the paragraphs below state the purpose and controls of each phase.

Phase 1: exploratory PIDNet development. Five runs changed multiple settings to obtain a working pipeline and reveal promising or problematic directions. These runs guide questions; they do not isolate causal factors.

Phase 2: controlled PIDNet ablation. A reference configuration and four one-factor variants test augmentation, false-safe penalty, resolution, and class weights. All other recorded settings are held fixed. One seed is used, so this is a controlled engineering comparison without a variance estimate.

Phase 3: checkpoint audit. The validation-selected augmentation-off PIDNet checkpoint is identified, reevaluated, qualitatively ranked, and benchmarked. Subsequent measurements refer to these saved parameters.

Phase 4: PIDNet-ROD repeatability. Both architectures are trained for seeds 1337, 2027, and 4242 under blue-only and blue+green policies with one controlled recipe. Separate ROD runs use the optimisation settings that can be reconstructed from the publication.

Phase 5: architecture matrix. Seven additional systems complete a nine-model, three-seed, two-policy matrix. Every policy contains 27 scored model-seed records; together the two ledgers contain 54.

Phase 6: convergence-aware follow-up. The nine-model, three-seed matrix is repeated for the principal blue+green policy with a 300-epoch ceiling, 60-epoch cosine decay, a 60-epoch minimum, and validation-based early stopping. This phase contains another 27 scored model-seed records.

Phase 7: runtime evaluation. Models are timed on a Ryzen 5 5500 CPU and a common H100 MIG server protocol. The reference PIDNet checkpoint and the reconstructed ROD checkpoint are also exported to TensorRT and evaluated on an RTX 5060.

All experiments exclude the official CaT test images from gradient updates and automatic checkpoint selection. The test set is nevertheless evaluated for multiple exploratory, ablation, and architecture runs over the lifetime of the project. It should therefore be understood as a consistently held-out benchmark partition, not as a single-use blinded set whose scores were hidden until every engineering decision was complete. Model checkpoints are selected by validation mIoU, but repeated human inspection of test reports remains a possible source of research-level selection bias.

6.2. Training Procedure

Each scored run follows the same operational sequence:

1. load one complete YAML configuration and set the Python, NumPy, and PyTorch random seeds;
2. construct training and validation datasets from the processed manifest, enabling augmentation only for training when configured;
3. compute class weights from training masks only, or load explicit neutral weights;
4. build the named two-class model and AdamW optimiser;
5. for every epoch, shuffle the training loader, compute the configured loss, backpropagate, update weights, and advance the scheduler;
6. run an unaugmented validation pass, accumulating its confusion matrix;

7. write the epoch history and `last.pt`; replace `best.pt` if validation mIoU exceeds the previous maximum;
8. after training, load `best.pt` in the evaluator and score the complete test split without gradients or shuffling.

Non-finite training loss or validation metrics raise an error rather than silently saving a checkpoint. Mixed precision uses `bfloat16` when CUDA supports it and falls back to `float16` otherwise. The ordinary 15-epoch experiments do not use early stopping; every model receives the same maximum number of training passes. This fixed budget improves protocol matching but is not necessarily optimal for every architecture.

The convergence-aware blue+green follow-up uses the same operational sequence with three changes. The epoch ceiling is 300, cosine decay reaches the minimum learning rate at epoch 60 and then holds it constant, and early stopping is enabled only after epoch 60. The stopping counter requires an improvement of at least 10^{-4} in validation mIoU and has patience 25. Checkpoint saving uses any strict improvement, so it can record a slightly higher value that is too small to reset the patience counter. Every reported test result still comes from `best.pt`, never from the final in-memory epoch.

The test evaluator computes weighted cross-entropy loss using the class weights stored in the checkpoint. Quality metrics are derived independently from thresholded predictions, so the test loss and mIoU need not rank models in the same order. The prediction interface is normalised to one full-resolution two-logit tensor even for third-party architectures that can expose auxiliary outputs.

6.3. Confusion Matrix Convention

Traversable is always the positive class. The aggregated confusion matrix is

$$\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}, \quad (6.1)$$

Here, $\mathbf{C}$ is the test-wide count matrix; TN, FP, FN, and TP denote true-negative, false-positive, false-negative, and true-positive pixel counts. Rows are ground truth and columns are prediction. This ordering is published because false-safe and false-block terminology can otherwise obscure which denominator is used.

Figure 6.2 reads the same matrix as two operational questions. The upper ground-truth row contains all pixels that should be blocked and supplies the denominator of FSR. The lower row contains all pixels that should be traversable and supplies the denominator of FBR.

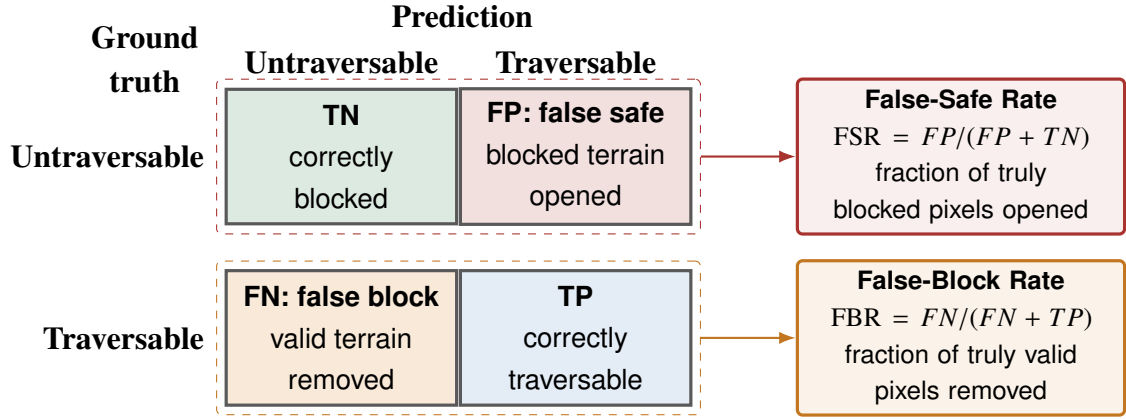


Fig. 6.2. Visual interpretation of the binary confusion matrix and the two asymmetric error rates. A false safe opens annotated blocked terrain; a false block removes annotated traversable terrain. Original schematic drawn for this thesis.

TABLE 6.1. Operational interpretation of the four binary outcomes.

Ground truth	Prediction	Count	Interpretation
Untraversable	Untraversable	TN	Correctly preserves a blocked pixel
Untraversable	Traversable	FP	False safe: opens a pixel outside the annotated capability envelope
Traversable	Untraversable	FN	False block: removes an annotated feasible pixel
Traversable	Traversable	TP	Correctly preserves a feasible pixel

Table 6.1 states the corresponding operational meaning of each matrix entry.

All principal test metrics use one matrix accumulated over every pixel in the split. They are therefore micro-aggregated by pixel. An image with more pixels would contribute more decisions, although fixed evaluation resizing makes all 544 test images equal in pixel count. Per-image metrics are used only for qualitative ranking.

6.4. Accuracy and Asymmetric Error Metrics

For class c , Intersection over Union is

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}. \quad (6.2)$$

Here, c denotes one semantic class, and TP_c , FP_c , and FN_c count pixels treated as true positives, false positives, and false negatives when c is considered the positive class. The operator IoU_c is therefore the intersection of prediction and target for c divided by their

union. For the binary convention this becomes

$$\text{IoU}_{\text{trav}} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}, \quad (6.3)$$

$$\text{IoU}_{\text{untrav}} = \frac{\text{TN}}{\text{TN} + \text{FP} + \text{FN}}, \quad (6.4)$$

$$\text{mIoU} = \frac{\text{IoU}_{\text{trav}} + \text{IoU}_{\text{untrav}}}{2}. \quad (6.5)$$

The subscripts *trav* and *untrav* denote traversable and untraversable. In the latter equation, *TN* becomes the intersection because the negative class in the global convention is treated as the class of interest. The unweighted mean prevents the more frequent class from completely determining the headline score.

Traversable precision, recall, and *F1* are

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F1 = \frac{2PR}{P + R}. \quad (6.6)$$

Here, *P* is traversable precision, *R* is traversable recall, and *F1* is their harmonic mean. *F1* describes positive-class overlap through precision and recall but does not describe true-negative performance. It is therefore reported beside rather than instead of *mIoU*.

The two safety-oriented rates are

$$\text{FSR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (6.7)$$

$$\text{FBR} = \frac{\text{FN}}{\text{FN} + \text{TP}}. \quad (6.8)$$

In these equations, *FSR* is False-Safe Rate and *FBR* is False-Block Rate; the four count symbols retain the definitions in Table 6.1. *FSR* conditions on truly untraversable pixels and asks what fraction the model opens. *FBR* conditions on truly traversable pixels and asks what fraction the model blocks. *FBR* is equal to one minus traversable recall; it is retained as a named metric because its operational meaning is clearer in this task.

The two errors matter differently. A connected false-safe area can allow a planner to consider terrain outside the vehicle’s annotated capability, while a false block can make a feasible route disappear or cause a conservative stop. Reducing one by moving the threshold usually increases the other. A useful system must therefore report both. Neither rate includes distance from the vehicle, trajectory intersection, connected-component size, severity of terrain, prediction confidence, or actual vehicle outcomes. They are safety-oriented diagnostic metrics, not safety probabilities.

6.5. Multi-Seed Reporting

For the nine-model comparison, the metric for each model and policy is reported as mean $\pm$ population standard deviation over three fixed seeds. Population standard deviation is used because the purpose is to describe those three executed runs, not to estimate an unknown variance from a random sample.

The seed set controls random initialisation, loader order, and deterministic augmentation sequences. It does not capture dataset-split uncertainty because all seeds use the same manifest assignment. It also does not capture hyperparameter uncertainty. A difference smaller than the observed run-to-run standard deviation is treated cautiously. In the 15-epoch blue+green matrix, the top FPN/EfficientNet-B0 and U-Net/EfficientNet-B0 means differ by only 0.02 percentage points. In the convergence-aware follow-up, FPN/EfficientNet-B0 is higher in all three executed seeds and its mean lead is 0.13 points. This is a consistent observed ordering for the declared suite, but three selected seeds still do not establish universal superiority or provide a confidence interval. The matched pairwise comparison shows the same complete consistency: ROD records higher test mIoU than PIDNet-S in all six of its policy–seed runs.

6.6. Qualitative Review Protocol

Aggregate metrics reveal how many pixels are wrong but not how those errors are arranged. The reference PIDNet checkpoint is therefore applied to all 544 test images and each image is assigned:

- per-image FSR;
- per-image FBR;
- combined error FSR + FBR for distributional ranking;
- boundary-band error.

The ground-truth boundary band is formed from the morphological difference between a two-pixel dilation and a two-pixel erosion. Boundary error is the fraction of pixels inside that band whose predicted class differs from the target. The band emphasises local contour placement but can be unstable for very thin regions and visually ambiguous annotations; it is a diagnostic, not an alternative headline metric.

The representative-range panel uses mechanically selected images near the 10th, 30th, 50th, 70th, and 90th percentiles of combined per-image error. Separate ranked panels select the maximum FSR, maximum FBR, and a severe boundary error. This procedure avoids selecting only visually attractive successes. The captions report identifiers and rates so the relationship between the visible mask and the numerical metric can be checked.

The checkpoint is also applied to 130 frames from the external, unlabelled ALICE mobile-robot sequences (scenario 02, sequences 01–05). Four frames are displayed to span several path and illumination conditions. ALICE is not used for training, and no ground-truth masks are available in this study. The panel can show whether predictions are connected, responsive to scene change, or visibly implausible; it cannot measure accuracy, FSR, FBR, calibration, or domain generalisation.

6.7. CPU and H100 Timing Protocols

The CPU platform is an AMD Ryzen 5 5500 with six cores and twelve hardware threads. PyTorch uses six inference threads, batch size 1, and 640×384 input. Thirty-two test tensors are preloaded. Two complete warm-up rounds are excluded, followed by five measured repeats, for 160 timed inferences. Eager execution and `torch.compile` execution are measured for all nine comparison models. Compilation occurs during warm-up and is excluded from the timed samples.

The CPU timer covers model forward, full-resolution logits, softmax, and thresholded prediction. It excludes image decoding, camera acquisition, visualisation, and disk output. The reference PIDNet and ROD results are tied to PyTorch 2.13.0, Python 3.12.12, and their exact checkpoints. ROD retains the common 640×384 external input but resizes it internally to 1024×1024 before its transformer encoder.

The common server measurement uses one NVIDIA H100 NVL 3g.47GB MIG partition, eager PyTorch FP32, batch size 1, and 640×384 external input. The 128 test tensors are preloaded. Five warm-up passes are excluded; ten repeats are measured with explicit CUDA synchronisation around every forward pass. The same partition and procedure are used for all nine models. These values control hardware within the architecture comparison, but an H100 MIG partition is not an embedded target.

The convergence experiment changes learned parameter values but not the model graphs, external tensor shape, or prediction operations. Its accuracy values are therefore paired with the existing architecture-level CPU and H100 timing measurements rather than rerunning identical inference graphs. Training wall-clock fields from the convergence histories are excluded: workers were paused and resumed while sharing the MIG partition, which preserves model and optimiser state but contaminates elapsed-time measurements.

6.8. RTX 5060 TensorRT Protocol

The reference PIDNet checkpoint and the reconstructed blue+green ROD checkpoint are exported to the Open Neural Network Exchange (ONNX) format and converted to fixed-shape, batch-one TensorRT 10.16.1 engines. The builder enables FP16 for eligible operations and retains FP32 for numerically sensitive reduction, softmax, normalisation, and selected unary operations. The result is described as FP16-enabled mixed precision rather than all-FP16 inference.

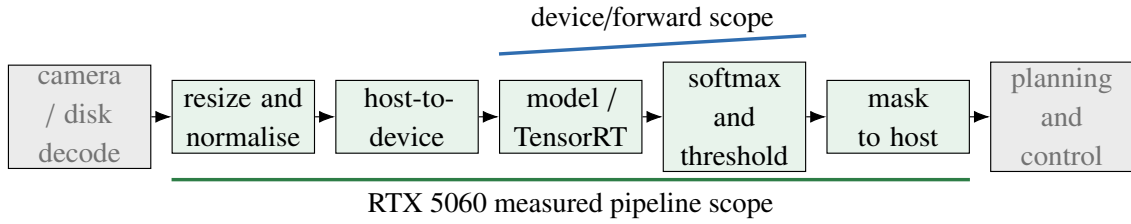


Fig. 6.3. Timing scopes used for the RTX 5060 benchmark. Grey stages are excluded. The device/forward scope starts from GPU-resident input; the measured pipeline starts from an already decoded host RGB array and returns a binary mask to host memory.

Figure 6.3 makes the two reported timing boundaries explicit before their sample counts are introduced.

Timing uses 64 preloaded native-resolution test frames, ten excluded warm-up passes, and five measured repeats, yielding 320 samples per model and scope. The *device scope* includes a device-to-device buffer copy, TensorRT execution, softmax, and thresholding. It excludes host-to-device input and device-to-host mask transfer. The *measured pipeline* includes CPU resize and normalisation, input transfer, TensorRT inference, softmax, thresholding, and return of the mask to CPU memory. Both exclude disk decode, camera-driver latency, visualisation, planning, actuation, and one-time engine construction.

After timing, each engine is evaluated on all 544 official test images. This full accuracy pass is mandatory because mixed precision can move probabilities across the 0.50 threshold even when average numerical differences are small. TensorRT metrics are reported separately from the original PyTorch metrics.

6.9. Artifact Provenance and Reproduction

Reproducibility repository:

<https://github.com/hax2/cat-pidnet-thesis-reproducibility>. This public repository is the companion artifact for the thesis. It contains the source code, exact configurations, split and label-conversion records, experiment launchers, checkpoint and metric evidence, audit scripts, and instructions needed to reproduce the reported workflows. The appendix provides direct commands and expected outputs so that the link is usable without having to infer the execution order from the source tree.

A headline result is considered identified only when the following artifacts are named:

1. the processed split manifest and binary mapping;
2. the complete YAML configuration;
3. the selected `best.pt` checkpoint;
4. the evaluator and metric implementation;

5. the saved JSON result and, where applicable, hardware report.

The reference PIDNet chain includes SHA-256 identities for all five elements and for the qualitative review, CPU benchmark, ONNX model, TensorRT engine, and TensorRT report. The saved test confusion matrix was independently recomputed in a fresh CPU evaluation. Exact equality of the counts establishes *evaluation reproducibility*: the same artifact and evaluator produce the same score. It is different from *training repeatability*, which asks how new stochastic training runs vary and is addressed only by the separate three-seed studies.

The blue-only fixed-budget architecture ledger contains all 27 configurations, metric records, and checkpoint hashes. The corresponding fixed-budget blue+green ledger contains all 27 configuration and metric chains and all confusion-derived integrity checks pass; 21 locally retained checkpoints have hashes. The six earlier controlled PIDNet-S and ROD blue+green checkpoints were not included in that download bundle.

The newer convergence-aware ledger is complete at the identity level: it contains 27 configurations, histories, train summaries, test records, and server-computed checkpoint SHA-256 values. All 27 confusion matrices reproduce their reported metrics and each contains 133,693,440 evaluated pixels. The checkpoint binaries themselves were not copied locally, so the ledger proves their recorded server identity but does not permit local reevaluation without retrieving a selected file.

The convergence campaign also records a software-stack limitation. PIDNet-S, ROD, DDRNet, and BiSeNet ran with PyTorch 2.7.1 and CUDA 12.6, while the FPN, U-Net, and SegFormer launcher used PyTorch 2.13.0 and CUDA 13.0 because its environment contained the required SMP dependency. All used the same H100 MIG partition, data, configurations, and metric implementation. The cross-model ranking is usable as a recipe-level comparison, but framework version is not fully controlled across architecture families.

Appendix C gives commands and expected outputs. Appendix D records the artifact identities. The repository ledgers provide the per-seed architecture paths and integrity status.

7. Accuracy, Asymmetric Errors, and Qualitative Results

7.1. Reference PIDNet-S Result

Table 7.1 reports the retained checkpoint on all 544 official test images. No image subset is used for the aggregate. This artifact-level measurement is kept separate from the three-seed PIDNet mean, which comes from a different training protocol.

TABLE 7.1. Verified reference PIDNet-S result for blue+green traversability. Lower FSR and FBR are preferable; the other quality metrics are higher-is-better.

Metric	Value
mIoU	0.893941
IoU untraversable	0.908936
IoU traversable	0.878946
Traversable precision	0.941592
Traversable recall	0.929631
Traversable F1	0.935573
False-Safe Rate	0.043176 (4.32%)
False-Block Rate	0.070369 (7.04%)
Weighted cross-entropy test loss	0.149528

Untraversable IoU is 3.00 percentage points higher than traversable IoU. Traversable precision exceeds recall by 1.20 points, and FBR is 2.72 points higher than FSR. At threshold 0.50, the model is therefore more likely to remove an annotated traversable pixel than to open an annotated blocked pixel. That conservative balance is visible numerically, but it does not prevent large false-safe regions in individual images.

TABLE 7.2. Aggregated pixel confusion matrix for the reference PIDNet-S checkpoint. Rows are ground truth and columns are prediction.

		Predicted	
		Untraversable	Traversable
Ground truth	Untraversable	73,151,244 (TN)	3,300,886 (FP)
	Traversable	4,027,996 (FN)	53,213,314 (TP)

Table 7.2 exposes the four raw counts from which the reference checkpoint’s aggregate rates are calculated.

The matrix contains exactly $544 \times 640 \times 384 = 133,693,440$ pixel decisions. Substitution into Equations 6.7 and 6.8 gives

$$\text{FSR} = \frac{3,300,886}{73,151,244 + 3,300,886} = 0.043176, \quad (7.1)$$

$$\text{FBR} = \frac{4,027,996}{53,213,314 + 4,027,996} = 0.070369. \quad (7.2)$$

A fresh CPU evaluation reproduced all four counts and every derived metric exactly. This direct calculation also prevents the two denominators from being accidentally interchanged.

7.2. PIDNet–ROD Comparison

Under the matched three-seed blue+green recipe, ROD reaches 0.9138 ± 0.0003 mIoU compared with 0.8822 ± 0.0002 for PIDNet-S. The reported difference is 3.15 percentage points using the unrounded run values. ROD also reduces mean FSR from 4.98% to 3.51% and mean FBR from 7.67% to 5.62%. Under blue-only, ROD reaches 0.9211 ± 0.0010 versus 0.8832 ± 0.0016 for PIDNet-S, a 3.79-point improvement, while reducing both asymmetric rates.

The small observed ROD standard deviations and the size of the mean gaps show that its advantage over PIDNet-S is not explained by one favourable seed: ROD records the higher test mIoU in every one of the six policy–seed runs. Because ROD simultaneously changes architecture, pretraining, normalisation, internal resolution, and trainable-parameter allocation, the experiment does not show which component causes the improvement. It establishes that the complete ROD pipeline achieves higher CaT accuracy than the matched PIDNet pipeline.

The closest documented ROD recipe improves further (Table 7.3). For blue+green, mIoU rises by 0.98 points over the controlled ROD mean. FSR is 0.34 points higher, but FBR falls by 1.65 points, so much of the gain reflects better recovery of traversable pixels rather than uniformly more conservative prediction. For blue-only, both reported error rates improve relative to controlled ROD.

TABLE 7.3. Closest documented ROD recipe transferred to CaT. This is a single-seed approximation because the publication omits the epoch budget and augmentation policy.

Policy	mIoU	IoU untrav.	IoU trav.	F1	FSR	FBR
Blue+green	0.9236	0.9338	0.9134	0.9547	3.85%	3.97%
Blue-only	0.9280	0.9590	0.8970	0.9457	2.20%	5.14%

These experiments establish a repeatable advantage over PIDNet-S. Whether that advantage is competitive with other compact systems is answered by the nine-model comparison.

7.3. Fixed-Budget Blue+Green Architecture Comparison

Table 7.4 gives the nine-model result for the blue+green policy after the original common budget of 15 epochs. The FPN with an EfficientNet-B0 encoder has the highest observed mean mIoU at 93.15%; the corresponding U-Net records 93.13%. Both have a population standard deviation of 0.07 percentage points across the three runs, whereas their means differ by only 0.02 points. With only three runs, the data support a common leading performance band but not a reliable ordering of the two decoders.

TABLE 7.4. Fixed-15-epoch blue+green results over seeds 1337, 2027, and 4242 (mean $\pm$ population standard deviation). Lower FSR and FBR are preferable.

Model	mIoU	F1	FSR	FBR
FPN/EfficientNet-B0	0.9315 $\pm$ 0.0007	0.9594 $\pm$ 0.0005	0.0293 $\pm$ 0.0013	0.0420 $\pm$ 0.0021
U-Net/EfficientNet-B0	0.9313 $\pm$ 0.0007	0.9592 $\pm$ 0.0004	0.0288 $\pm$ 0.0007	0.0429 $\pm$ 0.0016
FPN/MobileNetV2	0.9255 $\pm$ 0.0001	0.9556 $\pm$ 0.0001	0.0309 $\pm$ 0.0013	0.0472 $\pm$ 0.0015
SegFormer-B0	0.9254 $\pm$ 0.0002	0.9555 $\pm$ 0.0001	0.0296 $\pm$ 0.0008	0.0491 $\pm$ 0.0009
U-Net/MobileNetV2	0.9207 $\pm$ 0.0016	0.9528 $\pm$ 0.0009	0.0374 $\pm$ 0.0029	0.0446 $\pm$ 0.0019
ROD ViT-S	0.9138 $\pm$ 0.0003	0.9482 $\pm$ 0.0003	0.0351 $\pm$ 0.0028	0.0562 $\pm$ 0.0038
BiSeNetV2	0.8868 $\pm$ 0.0007	0.9307 $\pm$ 0.0005	0.0425 $\pm$ 0.0039	0.0803 $\pm$ 0.0048
PIDNet-S	0.8822 $\pm$ 0.0002	0.9281 $\pm$ 0.0002	0.0498 $\pm$ 0.0043	0.0767 $\pm$ 0.0053
DDRNet-23-Slim	0.8746 $\pm$ 0.0063	0.9228 $\pm$ 0.0042	0.0499 $\pm$ 0.0020	0.0862 $\pm$ 0.0062

Relative to PIDNet-S, FPN/EfficientNet-B0 improves mean mIoU by 4.93 points, reduces FSR by 2.05 points, and reduces FBR by 3.47 points. The gain is not merely a threshold shift from one error direction to the other. U-Net with the same encoder has nearly identical accuracy and the lowest observed mean FSR. FPN has slightly lower FBR. With three seeds, those small differences support a shared performance tier rather than selecting one decoder universally.

The 2×2 encoder–decoder subset shows two consistent patterns. EfficientNet-B0 improves mean mIoU over MobileNetV2 with either FPN or U-Net, and FPN improves mean mIoU over U-Net with either encoder. The decoder effect is much smaller with EfficientNet-B0 than with MobileNetV2. SegFormer-B0 has nearly the same mean mIoU as FPN/MobileNetV2. ROD ranks sixth in this controlled table: substantially above PIDNet-S, BiSeNetV2, and DDRNet, but below the four generic encoder–decoder systems and SegFormer-B0.

The single-seed ROD paper recipe reaches 92.36%, which narrows the gap but still remains below the 93.15% and 93.13% three-seed means of the top controlled systems. Since its optimiser, loss, augmentation, and seed protocol differ, that observation is descriptive rather than a strict row-wise comparison.

7.4. Convergence-Aware Blue+Green Follow-Up

The fixed-budget table controls training exposure, but it does not show what the same recipes reach after their validation curves flatten. The follow-up therefore keeps the blue+green data, loss, augmentation, optimiser, initial learning rate, threshold, and three seeds fixed while extending the training horizon. The 300 epochs are a ceiling. No run reaches it: early stopping ends the 27 runs between epochs 74 and 150, and validation selects checkpoints between epochs 50 and 141.

Table 7.5 reports the resulting test metrics. The selected epochs are listed in seed order 1337, 2027, and 4242. They are part of the result because replacing them with one round number would conceal large between-model and between-seed differences.

TABLE 7.5. Convergence-aware blue+green results over seeds 1337, 2027, and 4242. Each metric cell places the mean above its $\pm$ population standard deviation. “Selected epochs” gives the validation-selected best .pt epoch for those three seeds. Lower FSR and FBR are preferable.

Model	Selected epochs	mIoU	F1	FSR	FBR
FPN/EfficientNet-B0	119/90/83	0.9423 ± 0.0005	0.9660 ± 0.0003	0.0255 ± 0.0004	0.0340 ± 0.0011
U-Net/EfficientNet-B0	102/76/56	0.9409 ± 0.0004	0.9652 ± 0.0002	0.0260 ± 0.0002	0.0349 ± 0.0003
FPN/MobileNetV2	70/99/65	0.9372 ± 0.0005	0.9629 ± 0.0003	0.0274 ± 0.0003	0.0376 ± 0.0010
SegFormer-B0	50/141/87	0.9341 ± 0.0005	0.9610 ± 0.0003	0.0304 ± 0.0004	0.0374 ± 0.0002
U-Net/MobileNetV2	74/79/57	0.9332 ± 0.0004	0.9605 ± 0.0002	0.0314 ± 0.0001	0.0372 ± 0.0005
ROD ViT-S	74/82/54	0.9331 ± 0.0004	0.9604 ± 0.0002	0.0294 ± 0.0005	0.0399 ± 0.0003
PIDNet-S	104/75/75	0.9195 ± 0.0029	0.9519 ± 0.0018	0.0335 ± 0.0013	0.0511 ± 0.0019
BiSeNetV2	125/58/123	0.9149 ± 0.0019	0.9490 ± 0.0012	0.0361 ± 0.0021	0.0536 ± 0.0020
DDRNet-23-Slim	81/68/93	0.9095 ± 0.0022	0.9456 ± 0.0014	0.0374 ± 0.0009	0.0584 ± 0.0017

FPN/EfficientNet-B0 has the highest mean mIoU and also the lowest mean FSR and FBR. It scores above U-Net/EfficientNet-B0 in each matched seed; the mean lead is 0.13 percentage points. This is a consistent first place within the 27 executed runs, although three seeds and one split do not establish a universal decoder ranking. The two MobileNetV2 decoders separate more clearly: FPN/MobileNetV2 leads U-Net/MobileNetV2 by 0.41 points. ROD and U-Net/MobileNetV2 differ by only 0.003 percentage points in mean mIoU, far below either model’s observed between-seed standard deviation, but retain different error balances.

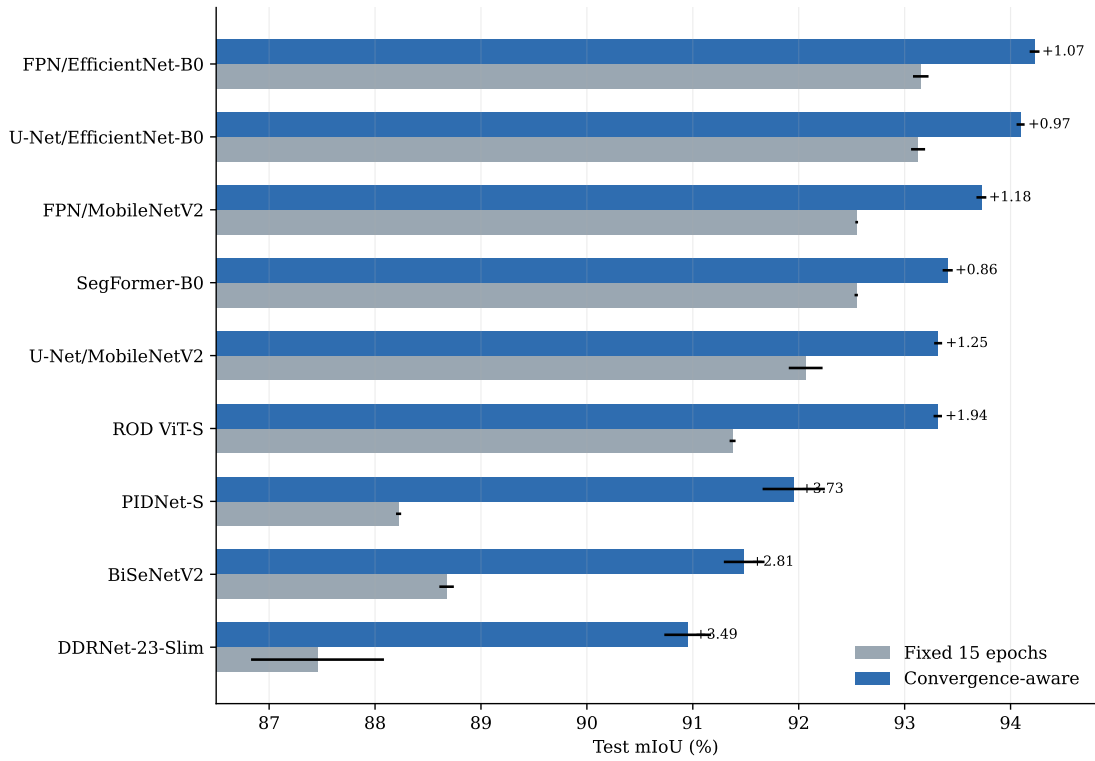


Fig. 7.1. Mean test mIoU under the fixed 15-epoch and convergence-aware blue+green protocols. Error bars are population standard deviations over three seeds; labels give the change in percentage points. Every architecture improves, but the size of the gain is strongly model-dependent.

Figure 7.1 shows that the extended protocol does not merely add a common offset. PIDNet-S gains 3.73 points, DDRNet-23-Slim 3.49, and BiSeNetV2 2.81, whereas the two EfficientNet-B0 systems gain about one point. The earlier comparison therefore penalised the slower-converging real-time models most strongly. Since the follow-up was run later and used two recorded PyTorch environments across architecture groups, the exact differences are protocol-level changes rather than pure causal estimates of additional epochs alone.

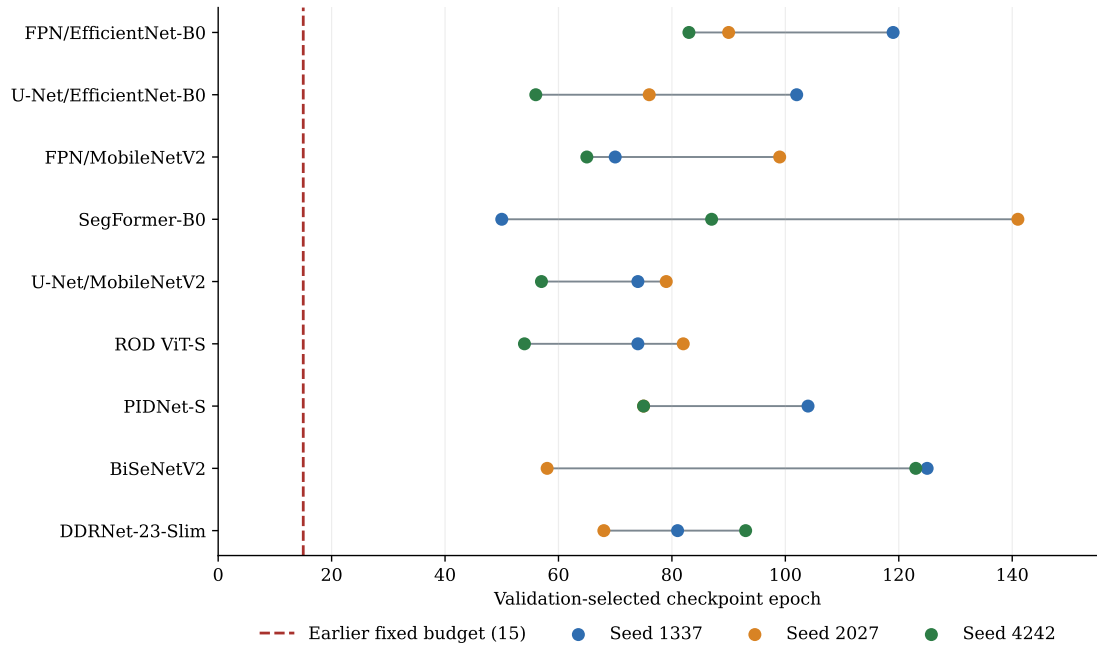


Fig. 7.2. Checkpoint epochs selected by validation mIoU. The dashed line marks the earlier 15-epoch budget. Horizontal segments span the three seeds of one architecture; none of the 27 selected checkpoints lies near epoch 15.

Figure 7.2 shows both between-model and between-seed variation in the validation-selected checkpoint epoch.

The selected epochs range from 50 for one SegFormer run to 141 for another. This spread is the direct reason not to replace the fixed budget with a claim that one epoch count, such as 60 or 300, is “optimal” for every model. The curves in Figure 7.3 add the missing temporal context: pretrained encoder–decoders rise quickly but continue to make small gains, while PIDNet, BiSeNet, and DDRNet require a longer initial climb.

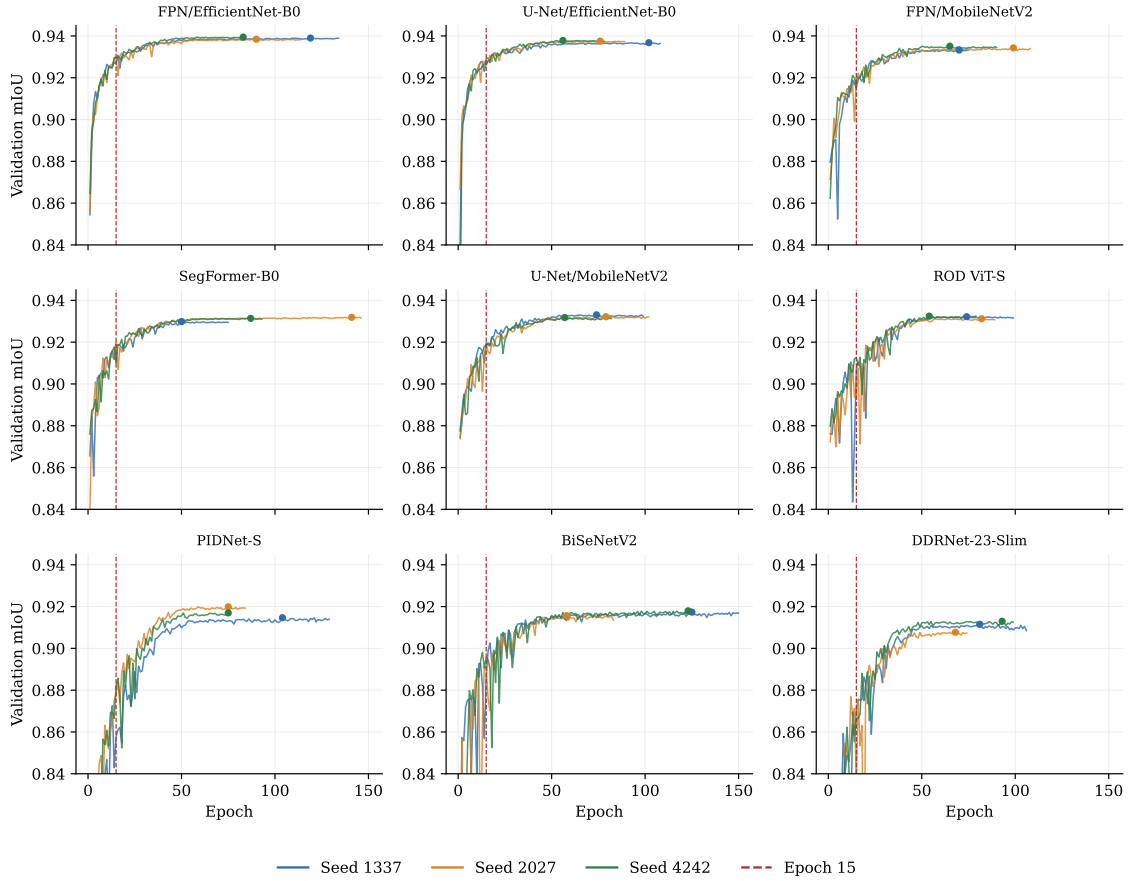


Fig. 7.3. Validation mIoU histories for all 27 convergence-aware runs. Dots mark the saved maximum for each seed and the dashed line marks epoch 15. The histories support a longer protocol, but their late fluctuations also show why validation-based checkpoint selection is preferable to reporting the final epoch.

7.5. Blue-Only Architecture Comparison

The blue-only table shows the same broad ordering. The task is different, so its higher mIoU values should not be interpreted as a direct improvement over blue+green. The EfficientNet-B0 FPN and U-Net again form the top tier: their means differ by 0.04 points, comparable to their observed 0.05–0.06-point seed variation.

Table 7.6 reports this fixed-budget blue-only matrix in full; it is not merged with the convergence-aware blue+green evidence.

TABLE 7.6. Controlled blue-only results over seeds 1337, 2027, and 4242 (mean $\pm$ population standard deviation). Lower FSR and FBR are preferable.

Model	mIoU	F1	FSR	FBR
FPN/EfficientNet-B0	0.9376 $\pm$ 0.0006	0.9531 $\pm$ 0.0005	0.0180 $\pm$ 0.0007	0.0469 $\pm$ 0.0016
U-Net/EfficientNet-B0	0.9372 $\pm$ 0.0005	0.9529 $\pm$ 0.0004	0.0192 $\pm$ 0.0013	0.0445 $\pm$ 0.0028
FPN/MobileNetV2	0.9332 $\pm$ 0.0009	0.9497 $\pm$ 0.0007	0.0193 $\pm$ 0.0015	0.0502 $\pm$ 0.0040
SegFormer-B0	0.9286 $\pm$ 0.0013	0.9461 $\pm$ 0.0010	0.0214 $\pm$ 0.0008	0.0523 $\pm$ 0.0011
U-Net/MobileNetV2	0.9283 $\pm$ 0.0001	0.9460 $\pm$ 0.0000	0.0233 $\pm$ 0.0017	0.0479 $\pm$ 0.0040
ROD ViT-S	0.9211 $\pm$ 0.0010	0.9401 $\pm$ 0.0008	0.0230 $\pm$ 0.0013	0.0597 $\pm$ 0.0019
BiSeNetV2	0.8885 $\pm$ 0.0026	0.9140 $\pm$ 0.0020	0.0363 $\pm$ 0.0028	0.0788 $\pm$ 0.0052
PIDNet-S	0.8832 $\pm$ 0.0016	0.9093 $\pm$ 0.0014	0.0353 $\pm$ 0.0016	0.0897 $\pm$ 0.0039
DDRNet-23-Slim	0.8743 $\pm$ 0.0042	0.9019 $\pm$ 0.0039	0.0384 $\pm$ 0.0022	0.0963 $\pm$ 0.0116

The best blue-only systems reach lower FSR than under blue+green because the negative class is larger and the target boundary changes; the numerical rates remain policy-specific. ROD retains its advantage over PIDNet-S but again ranks below the five strongest compact alternatives. DDRNet’s larger standard deviation shows that not every architecture is equally stable under the 15-epoch recipe.

For literature context, the reconstructed ROD blue-only traversable IoU is 89.70%, while the original CaT PSPNet/ResNet-101 sedan IoU is 91.65% [1]. The positive pixel identity is aligned, but one model is trained as binary segmentation and the other as four-class segmentation, so the 1.95-point gap is contextual rather than a controlled architecture comparison.

7.6. High-Performing PIDNet Cases

Figure 7.4 presents the four PIDNet cases with the lowest per-image sum of FSR and FBR. Each triptych shows the RGB image, ground truth, and prediction. The two open-field examples demonstrate that the model can reproduce a nearly horizontal transition between a foreground vehicle-capability region and a distant blocked field even when both are pale and textured. The wooded-trail example recovers a narrowing corridor and its asymmetric edges. The curved open-track example follows a broad bend while leaving a small interior blocked island.

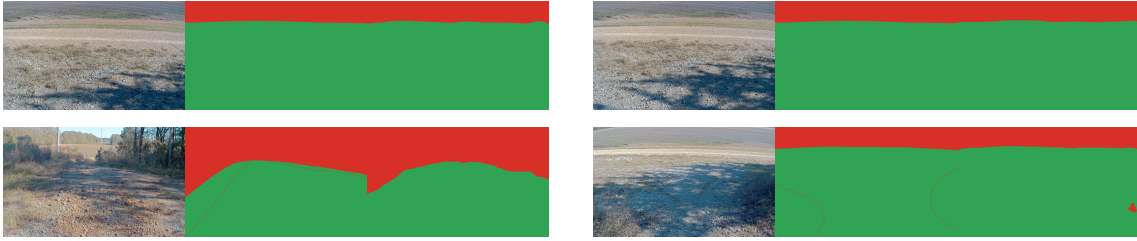


Fig. 7.4. High-performing reference PIDNet-S examples selected by the lowest per-image sum of FSR and FBR. Each triptych is RGB, binary ground truth, and prediction. Green denotes traversable; red denotes untraversable. These are reproducible successes, not a representative sample of the complete test distribution.

The similarity between the ground-truth and prediction panels in these cases shows that the model is learning a region-level task, not merely detecting a central road centreline. It predicts the full visible capability envelope and responds to distant boundary shape. Selection by low error also means these images cannot be used to infer how often such behaviour occurs.

7.7. Performance Across the Test Distribution

Figure 7.5 spans the 10th to 90th percentiles of combined per-image error. The top winding-track image recovers a broad connected corridor with small contour differences. By the 70th percentile the prediction narrows the visible region, and the 90th-percentile woodland case blocks much of a weakly defined corridor.

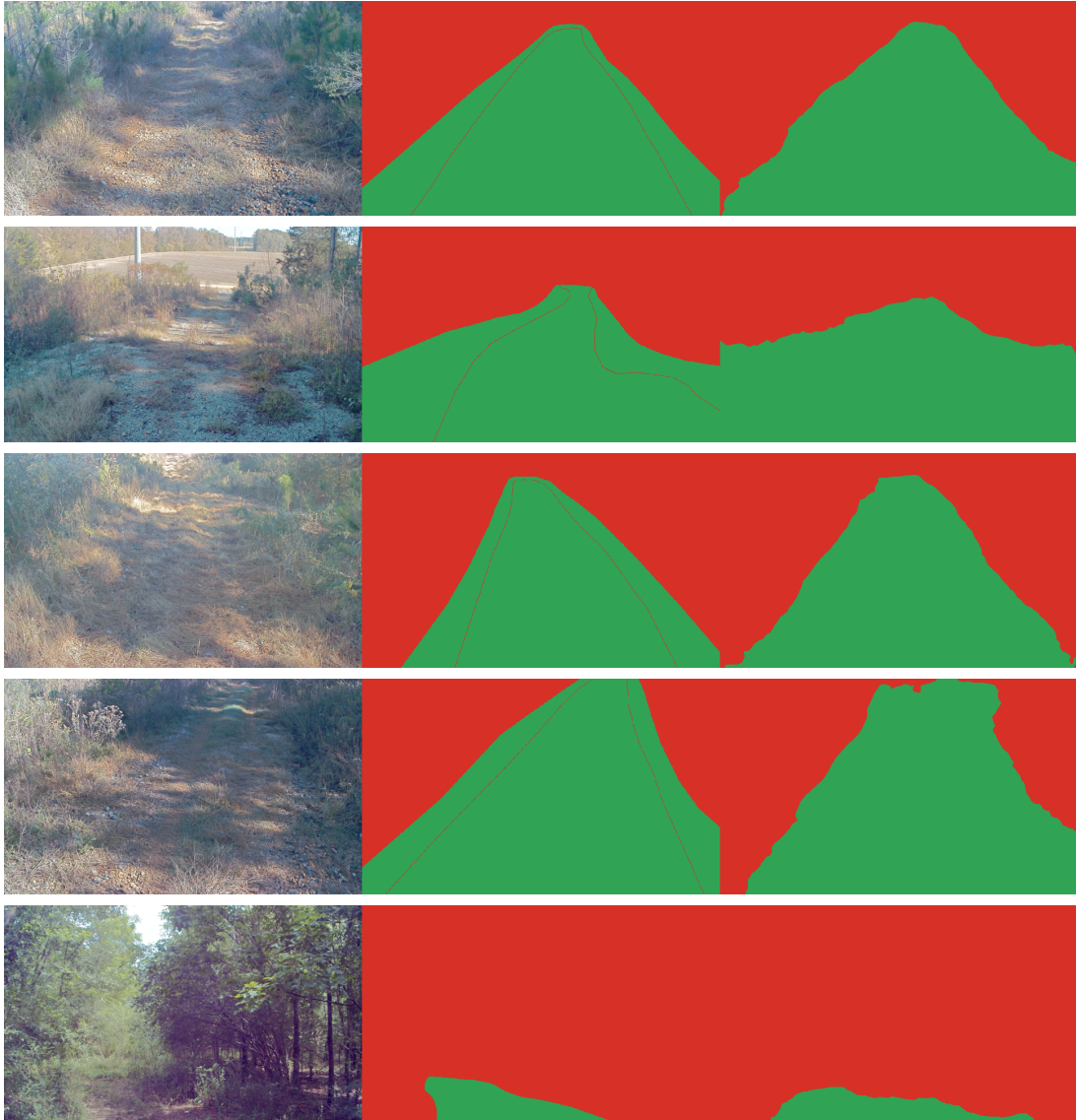


Fig. 7.5. Reference PIDNet-S predictions at the 10th, 30th, 50th, 70th, and 90th percentiles of per-image FSR + FBR, from top to bottom. Each triptych is RGB, ground truth, and prediction. The corresponding (FSR, FBR) pairs are (3.51%, 2.47%), (5.92%, 2.71%), (6.90%, 4.20%), (5.27%, 9.67%), and (1.62%, 19.75%).

The sequence explains why one per-image scalar is incomplete. The 50th-percentile image is weighted toward false-safe error, while the 90th-percentile image has a low FSR but a 19.75% FBR. Both may occupy a similar rank under a combined score while requiring different remedies.



(a) RGB input.



(b) Ground-truth mapping.



(c) PIDNet-S prediction.

Fig. 7.6. Detailed view of `mixed_pln_961`, the 10th-percentile example. The model follows the winding foreground corridor into the distance. Its remaining 3.51% FSR and 2.47% FBR are concentrated primarily along the two lateral boundaries.

In Figure 7.6, green extends along the track while vegetation and terrain outside the selected vehicle envelope remain red. The ground-truth and prediction overlays make small boundary offsets visible that are difficult to see in the binary panels alone. The image connects the low per-image rates to their spatial form.

7.8. False-Safe, False-Block, and Boundary Failures



Fig. 7.7. Asymmetric-error examples for reference PIDNet-S; each full-width triptych is RGB, ground truth, and prediction. Top: `mixed_pln_1278`, maximum per-image FSR (30.20%, FBR 0.19%). Middle: `mixed_425`, maximum FBR (52.84%, FSR 0.67%). Bottom: `mixed_459`, 49.72% boundary-band error (FSR 5.63%, FBR 10.70%).

Figure 7.7 provides the three ranked cases discussed below: maximum FSR, maximum FBR, and severe boundary disagreement.

In the top case, the prediction opens almost the entire pale cultivated field at the top and right of the image. The target admits the central and foreground corridor but blocks that field under the blue+green policy. The large connected green expansion explains the 30.20% FSR; its spatial coherence makes it more operationally important than the same number of isolated pixels. The field and track share visual cues, so the example also illustrates the annotation caveat: the model is clearly wrong relative to the mask, but RGB alone does not reveal the physical property used to separate the two capability regions.

In the middle case, the annotation marks a narrow path from the foreground toward the left distance. The prediction retains only a small lower segment and loses much of the shadowed corridor. This produces 52.84% FBR but only 0.67% FSR. It is conservative in the pixel convention, yet it could leave a planner with no route. The weak distant contrast and narrow labelled geometry connect the visible failure to the numerical rate.

The bottom case contains an irregular interface between low vegetation and track. The prediction moves the foreground contour and changes the width of the feasible strip. The global FSR and FBR are not as extreme, but nearly half of the four-pixel-wide target

boundary band is wrong. Because more than one nearby contour can look plausible at this interface, the 49.72% result should be interpreted as precise disagreement with the supplied annotation, not as a measurement of physical clearance error.

Together, the examples show three different remediation targets. False-safe expansion suggests stronger physical cues, calibration, or planner-side constraints. False blocks suggest better recall for narrow and shadowed corridors. Boundary disagreement suggests boundary-aware supervision, higher-quality annotation, or planning margins. A single loss change is unlikely to address all three.

7.9. Qualitative Transfer to ALICE

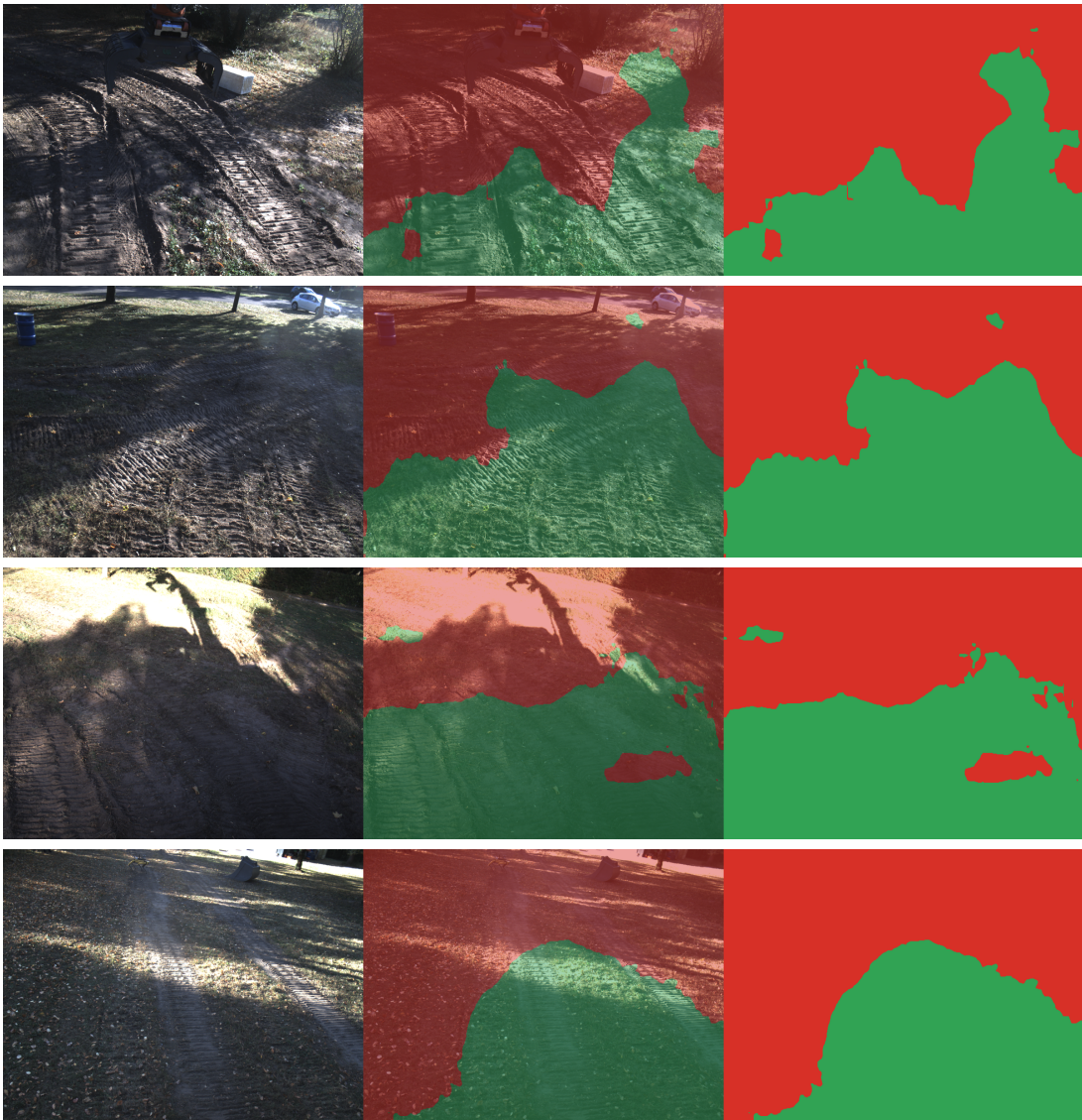


Fig. 7.8. Unscored transfer of the same reference blue+green PIDNet-S checkpoint to the external ALICE mobile-robot sequences. Each triptych shows RGB input, prediction overlay, and binary prediction. No ALICE ground truth is available, so the panels show output behaviour only.

Figure 7.8 is intentionally qualitative: it shows how the fixed checkpoint responds to external scenes but supplies no accuracy claim.

The first ALICE frame contains another robot close to the camera. The model mostly blocks the robot body and produces a narrow irregular traversable region below and around it, showing that the output responds to a foreground object rather than blindly opening the entire image. The second frame contains grass, shadows, a paved road, a parked car, and a barrel. The prediction opens a foreground grass wedge while largely blocking the road at the top. This is a warning that a CaT-trained capability prior is not a general road detector.

The third frame has severe illumination contrast. The model forms a broad foreground region but introduces holes and fragments near shadow boundaries. The fourth frame contains a visible central worn track through leaf-covered ground; the prediction forms a large connected lower region aligned only partly with that visual track. Across the four scenes the masks are structured and change with scene content, but their apparent plausibility varies.

No statement such as “the model generalises to ALICE” can be derived from these images. There is no target mask, no vehicle trial, and no calibration measurement. The panels only demonstrate that the reference pipeline executes on the new camera data and reveal concrete behaviours that a future labelled ALICE study should score.

8. Hardware and Inference-Speed Analysis

8.1. Why One FPS Number Is Insufficient

Inference speed is a property of a complete execution configuration, not only of a neural-network architecture. Processor, runtime, numerical precision, batch size, tensor shape, compiler support, memory transfer, warm-up, and the timer boundaries all affect the result. This chapter therefore keeps three questions separate:

1. how fast does the saved model execute on a consumer CPU?
2. how do all nine architectures compare on one common server GPU?
3. what throughput is obtained after compiling the two deployed checkpoints into fixed TensorRT engines and including preprocessing and mask return?

No speed value is borrowed from an architecture paper for ranking. Published Cityscapes or A6000 FPS figures use a different image size, model output, runtime, and hardware. Every table below names its own scope.

8.2. Ryzen 5 5500 CPU Results

Table 8.1 reports batch-one forward throughput with six PyTorch threads. Eager PIDNet-S reaches 21.16 FPS with 47.25 ms mean latency for the reference checkpoint. Its median is 46.45 ms and P95 is 55.63 ms. This confirms GPU-free operation at a modest perception rate, but the timer does not include camera decode or planning.

TABLE 8.1. Ryzen 5 5500 batch-one throughput at 640×384 . Values are forward and prediction only; compiled timing excludes compilation warm-up.

Model	Eager FPS	Compiled FPS	Compiled mean latency
DDRNet-23-Slim	27.74	36.50	27.40 ms
PIDNet-S	21.16	27.04	36.98 ms
BiSeNetV2	11.48	16.34	61.20 ms
FPN/MobileNetV2	7.23	14.26	70.13 ms
FPN/EfficientNet-B0	5.92	11.70	85.47 ms
U-Net/MobileNetV2	7.10	10.15	98.52 ms
U-Net/EfficientNet-B0	6.96	9.66	103.52 ms
SegFormer-B0	4.38	8.23	121.51 ms
ROD ViT-S	0.41	0.46	2,195.96 ms

Compiled DDRNet-23-Slim is the CPU throughput leader at 36.50 FPS. Its

convergence-aware blue+green mIoU, however, is the lowest in the nine-model table at 0.9095. PIDNet-S is slower but reaches 0.9195. FPN/EfficientNet-B0 reaches only 11.70 compiled FPS but gives the highest mean accuracy, 0.9423. The CPU therefore presents a clear sequence of trade-offs rather than one winner. ROD reaches 0.41 eager FPS and 0.46 compiled FPS; its frozen encoder reduces training-time optimisation but the full internally resized 1024×1024 transformer remains active during inference.

Compiler benefit is architecture-dependent. FPN/MobileNetV2 nearly doubles its throughput, and SegFormer-B0 improves substantially, while U-Net and PIDNet receive smaller gains. These differences show why floating-point-operation (FLOP) counts or parameter estimates cannot substitute for runtime measurements. Compilation raises ROD throughput by only 11.2%, from 0.4094 to 0.4554 FPS, leaving the current implementation unsuitable for CPU-rate perception loops.

8.3. Common H100 Results

The H100 protocol controls hardware and eager FP32 execution across all nine systems. PIDNet-S is fastest at 270.38 FPS. DDRNet-23-Slim follows at 245.17 FPS, while ROD is slowest at 21.79 FPS. The frozen encoder reduces ROD training gradients but does not reduce inference work; the full transformer remains on the forward path.

TABLE 8.2. Parameter count and common H100 NVL 3g.47GB MIG batch-one forward throughput. External input is 640×384 ; execution is eager FP32.

Model	Parameters	Mean FPS	Mean latency
PIDNet-S	7.62 M	270.38	3.70 ms
DDRNet-23-Slim	5.69 M	245.17	4.08 ms
BiSeNetV2	3.34 M	167.80	5.96 ms
FPN/MobileNetV2	4.22 M	147.36	6.79 ms
SegFormer-B0	3.71 M	141.23	7.08 ms
U-Net/MobileNetV2	6.63 M	131.86	7.58 ms
FPN/EfficientNet-B0	5.76 M	131.40	7.61 ms
U-Net/EfficientNet-B0	6.25 M	101.24	9.88 ms
ROD ViT-S	29.11 M	21.79	45.89 ms

Table 8.2 reports the measured H100 throughput and latency for every architecture under this common scope.

On the same H100 protocol, PIDNet-S is approximately 12.4 times faster than ROD. Under convergence-aware training, ROD’s blue+green mean is 1.36 points higher, so that accuracy improvement has a large latency price. In the nine-model table, ROD is also slower and less accurate than FPN/EfficientNet-B0 on this particular hardware and recipe. This does not make ROD universally inferior: a model-specific compiler, different training

objective, memory constraint, or another dataset can change the decision. It does mean that the controlled H100 evidence does not place ROD on the observed accuracy–speed frontier.

Parameter count again fails to determine speed. SegFormer-B0 has only 3.71 million parameters but is much slower than PIDNet-S on CPU. BiSeNetV2 is the smallest system but not the fastest H100 model. Operator mix, feature-map size, attention, memory traffic, and kernel implementation matter alongside stored weights.

8.4. Accuracy–Speed Relationship

Figure 8.1 combines convergence-aware blue+green mean mIoU with the existing H100 throughput measurements. Moving upward improves accuracy; moving right improves speed. No model occupies the upper-right corner.

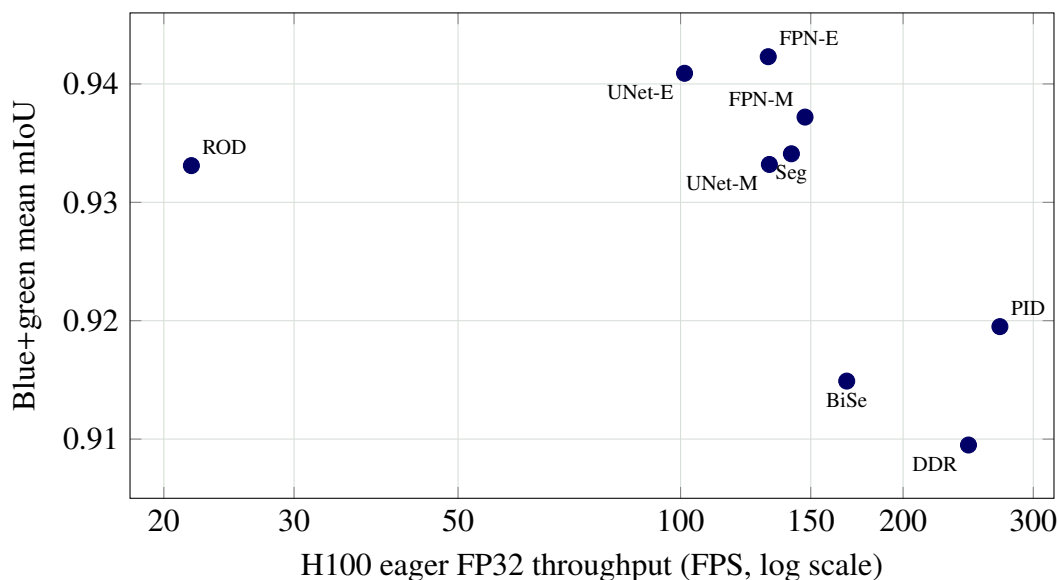


Fig. 8.1. Blue+green mean mIoU versus common H100 throughput. E and M denote EfficientNet-B0 and MobileNetV2. FPN/EfficientNet-B0 supplies the highest accuracy, while PIDNet-S provides the highest throughput. Accuracy uses the convergence-aware blue+green suite; inference speed uses the unchanged architecture graphs.

For a latency-first H100 deployment, PIDNet-S offers the most headroom. The FPN with MobileNetV2 adds 1.77 mIoU points while retaining 147.36 FPS. Replacing that encoder with EfficientNet-B0 adds another 0.50 points at 131.40 FPS and gives the highest observed accuracy mean. The EfficientNet-B0 U-Net is 0.13 points lower and also has lower throughput in this runtime. The plotted means do not include latency variance or model-specific compiler tuning, so the frontier is specific to the declared protocol.

The CPU frontier differs. Compiled DDRNet supplies the only measured rate above 30 FPS, PIDNet-S offers a stronger accuracy point at 27.04 FPS, and FPN/EfficientNet-B0

provides the highest accuracy at 11.70 FPS. A robot with a 10 Hz perception requirement may select a different model from one requiring 30 Hz. Frame rate should therefore be treated as a constraint in the model choice, not as an isolated leaderboard.

8.5. RTX 5060 TensorRT Results

TensorRT changes the deployment picture for the two exported checkpoints. Table 8.3 reports both device and measured-pipeline scopes. PIDNet-S reaches 106.43 pipeline FPS with 9.396 ms mean latency. ROD reaches 39.38 pipeline FPS with 25.392 ms mean latency. ROD therefore crosses a nominal 30 FPS threshold for the measured pipeline after compilation, although the unmeasured camera and planning stages still consume part of a real 33.3 ms frame budget.

TABLE 8.3. FP16-enabled mixed-precision TensorRT batch-one timing on the RTX 5060. Each distribution contains 320 measured samples after warm-up.

Model	Scope	Mean	P50	P95	FPS
PIDNet-S	Device	0.963 ms	0.880 ms	1.414 ms	1,037.97
	Measured pipeline	9.396 ms	9.178 ms	10.943 ms	106.43
ROD ViT-S	Device	17.401 ms	17.397 ms	17.669 ms	57.47
	Measured pipeline	25.392 ms	25.255 ms	26.580 ms	39.38

Figure 8.2 expresses the same result as a frame budget. At 30 FPS, one complete cycle may occupy at most 33.33 ms. The coloured segment is the measured TensorRT pipeline; the pale segment is only remaining time, not a measurement of the omitted camera, transport, planning, and control stages.

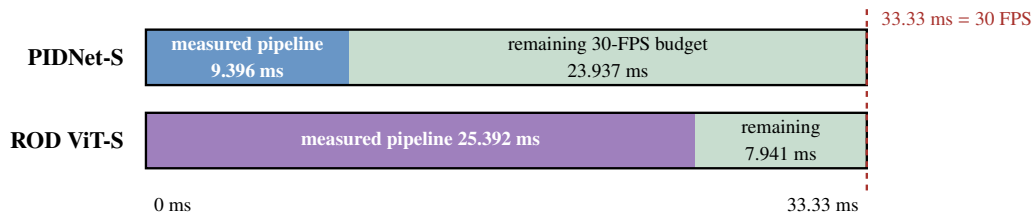


Fig. 8.2. Measured RTX 5060 TensorRT pipeline latency within a nominal 30-FPS frame budget. PIDNet-S leaves substantially more unallocated time than ROD, but neither bar measures the remaining robot stack. Original graphic drawn from the results in Table 8.3.

The gap between PIDNet’s 0.963 ms device time and 9.396 ms pipeline time shows that host preprocessing and transfers dominate after the small network is compiled. ROD still spends most of its time in the engine, because the internal 1024×1024 transformer remains expensive. Improving CPU resize or transfer would therefore benefit PIDNet proportionally more than ROD.

The TensorRT values should not be interpreted as a controlled speed-up over the H100 or Ryzen tables because the GPU, runtime, and precision all change. They answer a deployment question: what rate do the saved checkpoints achieve after hardware-specific compilation on the available local GPU?

8.6. Mixed-Precision Accuracy Verification

Full-test evaluation confirms that mixed-precision conversion does not materially change the operating points. Table 8.4 compares each engine with its source PyTorch checkpoint.

TABLE 8.4. Full 544-image comparison of each source PyTorch checkpoint with its mixed-precision TensorRT conversion.

Model/runtime	mIoU	FSR	FBR	mIoU change
PIDNet-S PyTorch	0.893941	4.3176%	7.0369%	reference
PIDNet-S TensorRT	0.893764	4.3117%	7.0667%	−0.000177
ROD PyTorch paper recipe	0.923569	3.85%	3.97%	reference
ROD TensorRT	0.923635	3.8508%	3.9574%	+0.000066

PIDNet changes by 0.0177 mIoU percentage points and ROD by 0.0066 points. Small positive or negative differences are expected when probabilities close to 0.50 cross the threshold after mixed-precision execution. The full confusion matrices, rather than a small image sample, support the claim that both conversions preserve accuracy at the precision used for architectural interpretation.

8.7. Deployment Meaning

Three distinct deployment candidates emerge:

- **CPU-constrained:** compiled DDRNet maximises frame rate, while PIDNet-S offers a more accurate near-real-time alternative. The EfficientNet systems require a lower frame-rate budget, and the current ROD implementation is not a practical CPU candidate.
- **Powerful common GPU:** PIDNet-S maximises throughput; FPN/EfficientNet-B0 provides the highest accuracy above 100 FPS on the H100 protocol, followed by U-Net/EfficientNet-B0.
- **Available RTX TensorRT path:** the reference PIDNet model exceeds 100 pipeline FPS, while the more accurate reconstructed ROD model reaches 39.4 FPS.

None of these measurements includes sensor exposure, decode, inter-process transport, projection into vehicle coordinates, temporal filtering, path planning, or control. Power

draw, thermal throttling, and embedded-GPU behaviour are also unmeasured. The results establish inference capacity and an accuracy–latency trade-off, not production readiness of a complete autonomous system.

9. Discussion and Research Outlook

9.1. Synthesis of Findings

The PIDNet-S experiments establish that the CaT capability labels can be learned by a compact network at useful accuracy and latency. The retained 7.62-million-parameter model reaches 0.893941 mIoU, and an independent CPU run reconstructs its complete confusion matrix exactly. This level of reproducibility connects the reported rates to one identifiable model rather than to an undocumented training run. It remains a CaT result, however; neither the exact score nor its operational interpretation transfers automatically to another site or vehicle.

Several seemingly reasonable interventions did not work as intended. The tested photometric augmentation reduced performance, the additional false-safe term increased both asymmetric error rates, and greater input resolution provided no benefit while halving the batch size. Neutral class weights recovered more traversable pixels but did not give the strongest combined operating point. By contrast, threshold adjustment behaved predictably: it traded false-safe openings for false blocks without eliminating the underlying errors. These outcomes argue for controlled measurements of engineering choices rather than assumptions based on their names or intent.

Pretrained feature transfer changes the attainable accuracy, but training horizon changes the comparison as well. Under the original matched 15-epoch conditions, ROD improves mean mIoU over PIDNet-S by 3.15 percentage points for blue+green and 3.79 points for blue-only. The convergence-aware blue+green follow-up raises both means to 0.9331 and 0.9195, narrowing their difference to 1.36 points. Across all nine systems, FPN/EfficientNet-B0 reaches 0.9423 ± 0.0005 and is highest in each of the three executed seeds; U-Net/EfficientNet-B0 follows at 0.9409 ± 0.0004 . ROD remains below the five pretrained encoder-decoder alternatives but nearly matches U-Net/MobileNetV2. These changes show that the earlier matrix described performance after equal early exposure, not a converged architecture ranking.

Runtime supplies a different ordering. PIDNet-S produces the highest H100 throughput, compiled DDRNet-23-Slim leads on the tested CPU, and the two EfficientNet-B0 systems provide the greatest overlap. TensorRT brings the reconstructed ROD model to 39.38 FPS for the measured local pipeline, but PIDNet-S remains much faster at 106.43 FPS. On the Ryzen CPU, ROD reaches only 0.41 eager FPS and 0.46 compiled FPS because freezing removes encoder gradients during training but not encoder computation during inference. Model selection is consequently a constraint problem involving target frame rate, runtime, and acceptable error profile, not a lookup of the largest mIoU.

9.2. Interpreting the Architecture Results

The comparison does not support a simple narrative that transformers are more accurate and CNNs are faster. ROD’s pretrained transformer improves over randomly initialised PIDNet-S, but FPN and U-Net with a compact pretrained convolutional EfficientNet-B0 encoder perform better under the common recipe. SegFormer-B0 lies between the EfficientNet and MobileNet systems. The best observed result is therefore associated with strong pretrained compact features and an effective multiscale decoder, not with one architecture category alone.

The 2×2 subset suggests that encoder choice has the larger effect: EfficientNet-B0 improves both decoders, while FPN’s advantage over U-Net is 0.13 points with EfficientNet and 0.41 points with MobileNetV2. FPN is higher for every matched encoder–seed pair in the convergence suite. With only two encoders and two decoders, this remains a local interaction, not a universal conclusion about FPN or U-Net.

ROD’s reconstructed training result shows that optimisation is architecture-dependent. CE alone, neutral weights, a larger learning rate, and polynomial decay improve ROD relative to the common CE+Dice recipe. This does not invalidate the retained PIDNet configuration. It shows that a recipe selected during one architecture’s development should not be assumed optimal for another.

The real-time architectures occupy different parts of the speed–accuracy plane. DDR-Net provides the highest compiled CPU rate but the lowest convergence-aware blue+green accuracy. PIDNet provides the highest H100 throughput, a verified deployment chain, and now a higher mean mIoU than BiSeNetV2. The phrase “real-time architecture” therefore describes design intent, not a fixed ranking on every processor.

9.3. Meaning of the Asymmetric Error Metrics

The reference PIDNet model has lower FSR than FBR, which indicates a conservative aggregate operating balance. The maximum-FSR image demonstrates why that statement is insufficient: a 4.32% global rate can coexist with a 30.20% per-image rate and one connected expansion into a large field. Conversely, the maximum-FBR example shows a visually coherent route being almost removed.

The highest-scoring models reduce both directions, so their accuracy gain is not obtained simply by opening or closing the entire mask. Even then, pixel rates do not specify whether an error intersects the planned trajectory. A false-safe pixel behind the vehicle and one under the projected front wheel contribute equally. Connectedness, distance, obstacle type, speed, and confidence are absent.

The metrics are best used as three layers of evidence:

1. mIoU and class IoU summarise balanced segmentation overlap;

2. FSR and FBR expose the direction of disagreement;
3. ranked visual review shows the spatial form of the error.

Figure 9.1 shows why these layers should not be collapsed into a single safety claim. This thesis evaluates pixel disagreement and its spatial form. Whether an error changes a route, and whether the vehicle then collides or loses progress, require a planner and physical system that are outside the present experiment.

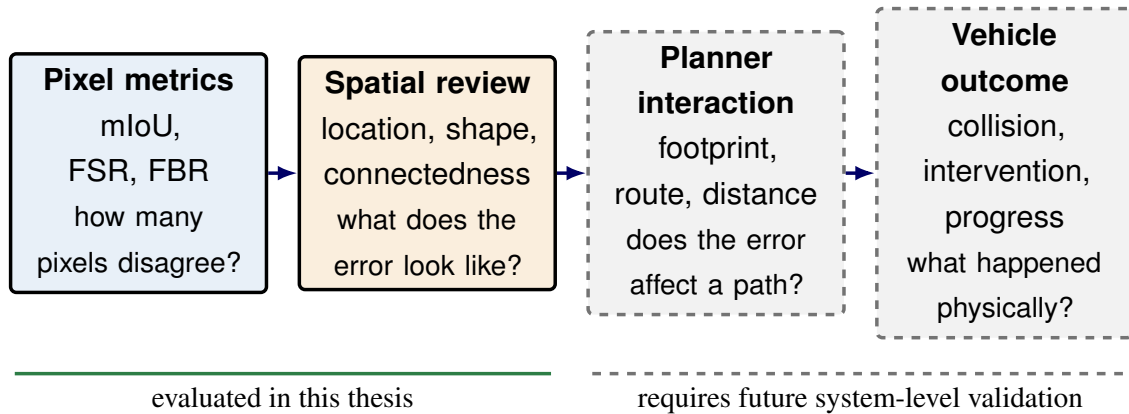


Fig. 9.1. Evidence required to move from segmentation quality to a vehicle-level safety statement. Solid stages are evaluated here; dashed stages require future planner and vehicle experiments. Original schematic drawn for this thesis.

A later planner-level study should add collision, intervention, route availability, progress, and time-to-goal measures rather than attempting to turn FSR alone into a safety certificate.

9.4. Annotation and Task Uncertainty

CaT's vehicle-capability concept is a strength because it is closer to navigation than a material label. It also introduces judgement. A sharp semantic boundary between a track and adjacent vegetation may be visible in RGB; the exact boundary between pickup-traversable and off-road-only soil may depend on slope, compaction, moisture, hidden roughness, or an annotator's vehicle model.

The selected examples make this limitation concrete without declaring the reference wrong. In `mixed_pln_1278`, a pale field and the accepted track look similar. In `mixed_118`, dense woodland and leaf-covered ground provide little visible support for the broad accepted foreground region. In `mixed_459`, several nearby vegetation-track contours appear visually possible. The masks may encode knowledge unavailable in the RGB frame, and visual ambiguity does not imply that both regions would support a vehicle equally.

Because the dataset provides one released annotation per image and no inter-annotator

agreement, the thesis cannot estimate label variance, separate model error from annotation uncertainty, or provide an upper bound on RGB-only performance. A model that reproduces the labels perfectly would reproduce their assumptions; it would not necessarily predict vehicle outcome perfectly.

9.5. Limitations

1. **One small, imbalanced dataset.** Training uses 1,002 images from three environments, with Power Line dominating. No environment-stratified result or geographically independent dataset is reported.
2. **Sample-level split.** The deterministic hash prevents direct identifier overlap, but it does not explicitly group visually adjacent or near-duplicate frames. The degree of correlation between training and validation samples is not measured.
3. **Annotation uncertainty is unquantified.** There are no repeated labels, annotator confidence scores, vehicle trials, or agreement statistics.
4. **RGB cannot observe every physical constraint.** Support, deformability, water depth, slope, clearance, and wheel slip may require geometry, temporal evidence, or interaction.
5. **Policy dependence.** Blue-only and blue+green are explicit vehicle policies. Results do not transfer numerically to blue+green+red, four-class CaT, or a different vehicle.
6. **Repeated benchmark use.** Test images are excluded from optimisation and automatic checkpoint selection, but the project reports many test runs. The benchmark is not a single-use blinded evaluation.
7. **Limited statistical scope.** The controlled ablation has one seed. The architecture table has three selected seeds on one fixed split and uses population standard deviation, not confidence intervals or hypothesis tests.
8. **Architecture and pretraining are confounded.** PIDNet-S and DDRNet start randomly, while the leading systems use pretrained encoders. The table ranks complete recipes and cannot isolate causal architecture effects.
9. **The convergence rule is shared, not model-specific optimisation.** The follow-up removes the obviously early 15-epoch stopping point for blue+green, but all models still share one objective, learning-rate schedule, and patience rule. Individually tuned recipes could change the ranking, and the blue-only matrix remains a fixed-budget result.
10. **Two training software environments were used.** The FPN, U-Net, and SegFormer runs used PyTorch 2.13.0/CUDA 13.0; PIDNet, ROD, DDRNet, and BiSeNet used PyTorch 2.7.1/CUDA 12.6. The data, configuration, hardware partition, and evaluator were fixed, but framework version remains a cross-family confound.

11. **Checkpoint availability differs from checkpoint identity.** The convergence suite records server-side SHA-256 identities for all 27 selected checkpoints, but their binaries are not present locally. The reference PIDNet checkpoint remains the fully local reproduction chain.
12. **Convergence training time is not usable evidence.** Shared-GPU workers were paused and resumed to control contention. This preserves training state but makes their recorded wall-clock duration unsuitable for comparing training cost.
13. **Qualitative comparison is centred on PIDNet.** The detailed percentile, failure, and ALICE panels use the reference PIDNet model. Equivalent ROD and top-FPN failure panels were not produced, so qualitative error differences between architectures are not established.
14. **ALICE is unlabelled.** The model output can be inspected, but no cross-domain accuracy or error rate can be calculated.
15. **Pixel metrics omit planning and time.** No temporal consistency, confidence calibration, trajectory collision, intervention, or progress metric is evaluated.
16. **Hardware is not the target embedded computer.** The Ryzen, H100 MIG partition, and desktop RTX 5060 do not measure automotive power, thermal limits, camera capture, planner latency, or end-to-end control.

9.6. Future Work

Future work should close the largest evidence gaps before adding more architectures.

First, quantify the target. A representative CaT subset should be annotated independently by several people using a written vehicle-capability guide. Agreement, boundary dispersion, and confidence should be reported. Where possible, annotations should be paired with slope, roughness, soil, and vehicle-trial evidence. This would separate visually ambiguous pixels from clear model failures.

Second, validate domain transfer. A temporally coherent ALICE subset should receive ground truth and vehicle-specific policy labels. Evaluation should include sequence-level consistency and should hold complete locations or runs out of training. ORFD, RUGD, RELLIS-3D, or WildScenes can extend appearance diversity, but their labels must be mapped carefully rather than treated as equivalent to CaT capability.

Third, connect perception to planning. The mask should be projected into vehicle coordinates and evaluated with a planner. Metrics should weight the near field, measure connected false-safe components, record whether the planned footprint intersects blocked terrain, and quantify collision-free progress. Obstacle inflation and a minimum-corridor rule can then be tested against both exposure to blocked terrain and route availability.

Fourth, represent uncertainty. Probability calibration, ensembles, test-time uncer-

tainty, and an explicit unknown/abstain region should be evaluated. Thresholds should be selected on validation data from a declared application cost rather than by maximising test mIoU. Temporal smoothing should be tested without allowing old confident errors to persist.

Fifth, isolate modelling factors. Pretrained and random initialisation should be compared within the same encoder family. The selected architectures should receive model-specific schedules and then be reevaluated under a common compute budget. More seeds and grouped or location-held-out splits would give stronger uncertainty estimates. Boundary supervision and hard-example sampling should be assessed against FSR, FBR, and annotation disagreement.

Finally, benchmark the actual robot stack. The selected checkpoint should run from camera capture through preprocessing, inference, projection, planning, and control on the intended embedded computer. Measurements should include median and tail latency, dropped frames, power, memory, temperature, and throttling. RGB should be fused with depth, LiDAR, or vehicle interaction where physical support cannot be inferred visually.

These steps would convert the present verified perception benchmark into evidence about autonomous navigation rather than merely adding another segmentation score.

10. Conclusions

This work set out to determine how far monocular RGB segmentation can support vehicle-specific traversability perception under realistic computational constraints. CaT provided a suitable test because its labels encode nested vehicle capability rather than terrain material alone. Auditing the release identified 1,812 valid RGB-label pairs. Their integer maps were converted into explicit blue-only and blue+green policies, and the official 544-image test partition was kept separate from the deterministic 1,002/266 development split.

PIDNet-S provided the first complete answer to the implementation question. The selected 7,623,522-parameter model achieves 0.893941 mIoU, with 4.32% FSR and 7.04% FBR. Reconstructing its 133,693,440-pixel confusion matrix on the CPU confirms that the saved configuration and checkpoint reproduce the reported result. Its development also produced negative findings of practical value: the evaluated augmentation, false-safe penalty, and higher-resolution setting did not improve the reference configuration. The threshold sweep showed that conservativeness can be adjusted, but only by exchanging one disagreement direction for the other.

Those disagreements acquire meaning only when their spatial form is inspected. The large false-safe field, the almost removed shadowed corridor, and the shifted vegetation boundary would lead to different planning responses despite all appearing as pixel errors. Percentile sampling shows that such behaviour is not represented adequately by a few best and worst images. ALICE sequences add evidence about output behaviour under domain shift, but without labels they cannot supply an accuracy estimate.

The comparative study demonstrates both the value and the limits of pretrained transfer. In the original fixed-15-epoch matrix, ROD raises the matched blue+green mean from PIDNet-S’s 0.8822 to 0.9138. The convergence-aware follow-up shows that neither value represented the end of learning: they rise to 0.9195 and 0.9331. Every architecture improves beyond its 15-epoch result, and validation selects checkpoints between epochs 50 and 141. The largest mean is 0.9423 ± 0.0005 for FPN/EfficientNet-B0, which ranks first in all three executed seeds; U-Net/EfficientNet-B0 follows at 0.9409 ± 0.0004 . This identifies the highest observed mean for the declared suite without claiming that one shared schedule is optimal for every architecture.

No single ordering survives the change from accuracy to runtime. PIDNet-S is fastest on the common H100 protocol at 270.38 FPS and reaches 106.43 FPS in the RTX 5060 TensorRT pipeline. DDRNet-23-Slim leads the compiled Ryzen measurement at 36.50 FPS, while the most accurate EfficientNet-B0 models are slower and ROD reaches only 0.46 compiled CPU FPS. Because training changes parameter values rather than the inference graph, these architecture-level timings remain applicable to the convergence-aware accuracy points. The TensorRT-compiled ROD model reaches 39.38 FPS with

negligible mIoU change. These measurements show why hardware, precision, runtime, and timing boundaries belong in the model specification.

The four research questions of Chapter 1 can now be answered directly:

1. PIDNet-S does provide a reproducible, real-time baseline. Trained only on the official training partition, the retained checkpoint reaches 0.893941 mIoU with 4.32% FSR and 7.04% FBR, its complete confusion matrix is reproduced exactly over 133,693,440 pixel decisions, and the same saved parameters sustain 21.16 eager CPU frames per second and a 106.43 FPS TensorRT pipeline.
2. The tested choices that materially change accuracy and error balance are augmentation (disabling the evaluated recipe improves mIoU by 2.72 points), class weighting (neutral weights lower FBR to 6.95% but raise FSR), and the decision threshold (it exchanges FSR for FBR monotonically). The false-safe loss penalty and the higher-resolution variant degrade the operating point.
3. Under matched seeds, frozen-encoder transfer outperforms the compact trained-from-scratch network: ROD leads PIDNet-S by 3.15 percentage points for blue+green and 3.79 for blue-only, with the higher score in all six policy-seed runs. Within the nine-system matrix, both rank below the ImageNet-pretrained encoder-decoder systems, and FPN/EfficientNet-B0 leads the convergence-aware suite at 0.9423 ± 0.0005 .
4. Accuracy rankings change substantially once deployment conditions enter the comparison. PIDNet-S is fastest under the common H100 protocol at 270.38 FPS and reaches 106.43 FPS in the TensorRT pipeline; compiled DDRNet-23-Slim leads the Ryzen measurement at 36.50 FPS; and the most accurate EfficientNet-B0 systems sit in the slower half of the table, confirming that parameter counts and architecture families predict measured speed poorly.

The resulting masks are useful perception outputs, not certificates of safe motion. A single RGB image cannot reveal every property that determines vehicle support, and the available CaT release contains neither repeated annotations nor agreement measurements for ambiguous capability boundaries. The study also does not test temporal consistency, planning, vehicle dynamics, or the complete embedded stack. Progress beyond the present results therefore depends less on adding another architecture row than on collecting stronger target evidence, labelling cross-domain sequences, modelling uncertainty, and evaluating the perception-planning-vehicle loop as a whole.

Bibliography

- [1] S. Sharma, L. Dabbiru, T. Hannis, G. Mason, D. W. Carruth, M. Doude, C. Goodin, C. Hudson, S. Ozier, J. E. Ball, and B. Tang, “CaT: CAVS Traversability Dataset for Off-Road Autonomous Driving,” *IEEE Access*, vol. 10, pp. 24759–24768, 2022, doi: 10.1109/ACCESS.2022.3154419.
- [2] J. Xu, Z. Xiong, and S. P. Bhattacharyya, “PIDNet: A Real-Time Semantic Segmentation Network Inspired by PID Controllers,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19529–19539, doi: 10.1109/CVPR52729.2023.01871.
- [3] F. Islam, M. M. Nabi, and J. E. Ball, “Off-Road Detection Analysis for Autonomous Ground Vehicles: A Review,” *Sensors*, vol. 22, no. 21, art. 8463, 2022, doi: 10.3390/s22218463.
- [4] C. Sevastopoulos and S. Konstantopoulos, “A Survey of Traversability Estimation for Mobile Robots,” *IEEE Access*, vol. 10, pp. 96331–96347, 2022, doi: 10.1109/ACCESS.2022.3202545.
- [5] M. Wigness, S. Eum, J. G. Rogers III, D. Han, and H. Kwon, “A RUGD Dataset for Autonomous Navigation and Visual Perception in Unstructured Outdoor Environments,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 5000–5007, doi: 10.1109/IROS40897.2019.8968283.
- [6] P. Jiang, P. Osteen, M. Wigness, and S. Saripalli, “RELLIS-3D Dataset: Data, Benchmarks and Analysis,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2021, pp. 1110–1116, doi: 10.1109/ICRA48506.2021.9561251.
- [7] S. Triest, M. Sivaprakasam, S. J. Wang, W. Wang, A. M. Johnson, and S. Scherer, “TartanDrive: A Large-Scale Dataset for Learning Off-Road Dynamics Models,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2022, pp. 2546–2552, doi: 10.1109/ICRA46639.2022.9811648.
- [8] C. Min, W. Jiang, D. Zhao, J. Xu, L. Xiao, Y. Nie, and B. Dai, “ORFD: A Dataset and Benchmark for Off-Road Freespace Detection,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2022, pp. 2532–2538, doi: 10.1109/ICRA46639.2022.9812139.
- [9] K. Vidanapathirana, J. Knights, S. Hausler, M. Cox, M. Ramezani, J. Jooste, E. Griffiths, S. Mohamed, S. Sridharan, C. Fookes, and P. Moghadam, “WildScenes: A Benchmark for 2D and 3D Semantic Segmentation in Large-Scale Natural Environments,” *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 532–549, 2025, doi: 10.1177/02783649241278369.

- [10] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440, doi: 10.1109/CVPR.2015.7298965.
- [11] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.
- [12] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [13] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder–Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” in *Proc. European Conf. Computer Vision (ECCV)*, 2018, pp. 833–851, doi: 10.1007/978-3-030-01234-2_49.
- [14] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, “Pyramid Scene Parsing Network,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6230–6239, doi: 10.1109/CVPR.2017.660.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
- [16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [17] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [18] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 12077–12090.
- [19] H. Pan, Y. Hong, W. Sun, and Y. Jia, “Deep Dual-Resolution Networks for Real-Time and Accurate Semantic Segmentation of Traffic Scenes,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 3, pp. 3448–3460, 2023, doi: 10.1109/TITS.2022.3228042.
- [20] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, and N. Sang, “BiSeNet V2: Bilateral Network with Guided Aggregation for Real-Time Semantic Segmentation,” *International Journal of Computer Vision*, vol. 129, no. 11, pp. 3051–3068, 2021, doi: 10.1007/s11263-021-01515-2.

- [21] T. Sun, H. Ye, J. Mei, L. Chen, F. Zhao, L. Zong, and Y. Hu, “ROD: RGB-Only Fast and Efficient Off-road Freespace Detection,” *arXiv preprint arXiv:2508.08697*, 2025.
- [22] Y. Xiong, B. Varadarajan, L. Wu, X. Xiang, F. Xiao, C. Zhu, X. Dai, D. Wang, F. Sun, F. Iandola, R. Krishnamoorthi, and V. Chandra, “EfficientSAM: Leveraged Masked Image Pretraining for Efficient Segment Anything,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16111–16121.
- [23] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [24] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, “Segment Anything,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 4015–4026, doi: 10.1109/ICCV51070.2023.00371.
- [25] M. H. Faykus III, A. Pickeral, E. Marquez, M. C. Smith, and J. C. Calhoun, “Efficient Vision Transformers for Autonomous Off-Road Perception Systems,” *Journal of Computer and Communications*, vol. 12, no. 9, pp. 188–207, 2024, doi: 10.4236/jcc.2024.129011.
- [26] V. V. Ramtekkar, L. Dahiya, N. Shah, K. Nishimiya, T. Kuroki, C. Song, A. Kim, and M.-H. Jeon, “Robust Depth-Aided Segmentation for Drivable Region Detection in Challenging Environments,” in *ICRA 2024 Workshop on Resilient Off-Road Autonomy*, poster, 2024. [Online]. Available: <https://openreview.net/forum?id=qzVdN0ZRt9>
- [27] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *Proc. 4th Int. Conf. 3D Vision (3DV)*, 2016, pp. 565–571, doi: 10.1109/3DV.2016.79.
- [28] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

A. Configuration Reference

A.1. PIDNet-S Configuration

The portable YAML below is the configuration used for verified evaluation. It uses repository-relative paths; the training checkpoint contains the original full configuration.

```
seed: 1337
raw_root: CAT
processed_root: data_processed_blue_green
reports_dir: reports
configs_dir: configs
outputs_dir: outputs
dataset_name: mixed
experiment_name: controlled_ablation_augment_off

preprocessing:
  val_fraction: 0.2
  traversable_palette_names: [blue, green]
  mapping_filename: discovered_mapping_blue_green.yaml

training:
  model_name: pidnet-s
  input_size: [640, 384]
  batch_size: 4
  epochs: 15
  learning_rate: 0.00025
  min_learning_rate: 0.00001
  weight_decay: 0.0001
  mixed_precision: bfloat16
  num_workers: 0
  augment: false
  untraversable_weight_bias: 2.5
  traversable_weight_bias: 0.75

postprocessing:
  traversable_threshold: 0.5
```

A.2. Reconstructed ROD Configuration

```
seed: 1337
raw_root: CAT
processed_root: data_processed_blue_green
reports_dir: reports
configs_dir: configs
outputs_dir: outputs
```

```

dataset_name: mixed
experiment_name: mixed_binary_traversability_rod_vits_paper

preprocessing:
  val_fraction: 0.2
  traversable_palette_names: [blue, green]
  mapping_filename: discovered_mapping_blue_green.yaml

training:
  model_name: rod-vits
  rod_encoder_checkpoint: weights/efficient_sam_vits.pt
  rod_encoder_image_size: 1024
  input_size: [640, 384]
  batch_size: 8
  val_batch_size: 4
  epochs: 15
  learning_rate: 0.001
  weight_decay: 0.01
  lr_scheduler: poly
  poly_power: 0.9
  mixed_precision: bf16
  num_workers: 4
  augment: false
  class_weights: [1.0, 1.0]
  ce_loss_weight: 1.0
  dice_loss_weight: 0.0

postprocessing:
  traversable_threshold: 0.5

```

A.3. Common Fixed-Budget Architecture-Comparison Template

The model name, experiment name, seed, processed root, and mapping filename change by row and policy. The remaining controlled settings follow this template:

```

training:
  model_name: <architecture-specific name>
  encoder_weights: <imagenet | null | model-specific>
  input_size: [640, 384]
  batch_size: 4
  epochs: 15
  learning_rate: 0.00025
  min_learning_rate: 0.00001
  weight_decay: 0.0001
  mixed_precision: bf16
  num_workers: 4
  augment: true
  untraversable_weight_bias: 2.5
  traversable_weight_bias: 0.75

```

```
postprocessing:
  traversable_threshold: 0.5
```

Table A.1 maps each human-readable architecture name to the exact configuration string consumed by the model factory.

TABLE A.1. Configuration model names used to construct each architecture-comparison system.

System	training.model_name
PIDNet-S	pidnet-s
ROD ViT-S	rod-vits
FPN/MobileNetV2	smp:FPN:mobilenet_v2
FPN/EfficientNet-B0	smp:FPN:efficientnet-b0
U-Net/MobileNetV2	smp:Unet:mobilenet_v2
U-Net/EfficientNet-B0	smp:Unet:efficientnet-b0
SegFormer-B0	smp:Segformer:mit_b0
DDRNet-23-Slim	ddrnet-23-slim
BiSeNetV2	bisenetv2

A.4. Convergence-Aware Follow-Up Template

The convergence suite retains the model-specific fields and common settings above, but replaces the fixed horizon with the following schedule and stopping rule:

```
training:
  epochs: 300 # ceiling, not claimed optimum
  lr_scheduler: cosine
  cosine_decay_epochs: 60 # then hold min_learning_rate
  save_last_checkpoint: false
  save_optimizer_state: false
  early_stopping:
    enabled: true
    metric: mIoU
    mode: max
    patience: 25
    min_epochs: 60
    min_delta: 0.0001
postprocessing:
  traversable_threshold: 0.5
```

The complete 27 generated YAML files are stored under `configs/generated/convergence_blue_green_e300_c60_m60_p25/blue_green/`. They should be used instead of reconstructing a run from this shortened listing.

B. Reference Evaluation Record

The following values are sufficient to check an independent checkpoint evaluation. The confusion matrix is ordered by ground-truth row and predicted column.

```
split: test
checkpoint: outputs/controlled_ablation_augment_off/checkpoints/best.pt
threshold: 0.5
confusion_matrix:
  - [73151244, 3300886]
  - [4027996, 53213314]
mIoU: 0.8939406948414585
iou_untraversable: 0.9089355053942137
iou_traversable: 0.8789458842887033
precision_traversable: 0.9415919184912819
recall_traversable: 0.9296313099752609
f1_traversable: 0.9355733889285891
false_safe_rate: 0.04317585396247299
false_block_rate: 0.07036869002473913
```

The TensorRT PIDNet result is expected to differ slightly:

```
confusion_matrix:
  - [73155734, 3296396]
  - [4045047, 53196263]
mIoU: 0.8937640836927854
false_safe_rate: 0.04311712440189698
false_block_rate: 0.07066656930108693
```

The reconstructed ROD TensorRT full-test record is:

```
confusion_matrix:
  - [73508109, 2944021]
  - [2265250, 54976060]
mIoU: 0.9236346417584864
false_safe_rate: 0.03850803110390776
false_block_rate: 0.03957369249585658
```

C. Reproduction Commands

Reproducibility repository:

<https://github.com/hax2/cat-pidnet-thesis-reproducibility>. The repository contains the code, configurations, checkpoints and evidence records, and the instructions needed for reproduction. Run the commands in this appendix from its root. The convergence evidence was generated from source commit 865cda90; later thesis-only edits do not alter the recorded training artifacts or metrics.

C.1. Audit and Preprocessing

From the repository root, the raw dataset can be audited and the main binary dataset regenerated with:

```
PYTHONPATH=src python -m ttfm.cli \
  --config configs/frozen_pidnet_cat.yaml audit

PYTHONPATH=src python -m ttfm.cli \
  --config configs/frozen_pidnet_cat.yaml preprocess
```

Preprocessing replaces the configured processed root, so the retained manifest identity should be checked before regenerating evidence.

C.2. PIDNet-S Evaluation

```
PYTHONPATH=src python -m ttfm.cli \
  --config configs/frozen_pidnet_cat.yaml \
  eval --split test
```

This command loads `best.pt`; it does not update weights. After processing the complete test manifest, it writes `test_metrics.json`.

C.3. Ranked Qualitative Review

```
PYTHONPATH=src python -m ttfm.cli \
  --config configs/frozen_pidnet_cat.yaml \
  review --split test --top-k 10
```

The ALICE panel is regenerated separately:

```
PYTHONPATH=src python scripts/alice_preview.py \
  --config configs/frozen_pidnet_cat.yaml \
  --input-dir alice \
```

```
--output-dir outputs/alice_pidnet_frozen_preview \
--sample-count 12 \
--checkpoint best.pt
```

C.4. TensorRT Benchmarks

```
PYTHONPATH=src python scripts/edge_trt_benchmark.py \
--config configs/ablations/controlled_ablation_augment_off.yaml \
--checkpoint outputs/controlled_ablation_augment_off/checkpoints/best.pt \
--input-dir data_processed_blue_green/test/images \
--precision fp16 --limit 64 --warmup 10 --repeats 5 \
--accuracy-limit 0 \
--output reports/edge_pidnet_rtx5060_trt.json
```

```
PYTHONPATH=src python scripts/edge_trt_benchmark.py \
--config rod_suite_results/configs/rod_vits_cat_paper.yaml \
--checkpoint rod_suite_results/outputs/mixed_binary_traversability_rod_vits_paper/checkpoints/best.pt \
--input-dir data_processed_blue_green/test/images \
--precision fp16 --limit 64 --warmup 10 --repeats 5 \
--accuracy-limit 0 \
--output reports/edge_rod_rtx5060_trt.json
```

Both commands require a CUDA and TensorRT environment. Engine construction is recorded but excluded from inference timing. Exact latency is hardware- and runtime-dependent even when accuracy is reproducible.

C.5. Architecture Ledgers

The readable aggregate reports are:

```
reports/blue_only_verified_summary.md
reports/blue_green_verified_summary.md
reports/convergence_blue_green_verified_summary.md
```

Their JSON ledgers contain per-seed configurations, metrics, confusion counts, artifact paths, and integrity checks. The convergence ledger additionally contains validation-selected epochs and server-recorded checkpoint identities. A selected run can be reevaluated after retrieving its checkpoint with:

```
PYTHONPATH=src python -m ttfm.cli \
--config <exact-config.yaml> eval --split test
```

D. Artifact Identities

These SHA-256 values identify the evidence package at the time of verification. A different digest denotes a different artifact, even if its filename is unchanged. Table D.1 records the retained PIDNet evidence chain.

TABLE D.1. SHA-256 identities for the PIDNet evidence chain.

Artifact	SHA-256
configs/frozen_pidnet_cat.yaml	07b07bbb1bb9e6f0586df6598d2cf982f8af845fb3e16785810615c80ebdaa74
data_processed_blue_green/ manifest.json	198fef93fc45568e1dae49ae8780906fa6ef479548214d20244cd12e51201fa2
outputs/.../checkpoints/best.pt	b9862fe49713c0622d760b94aa3fc055fda6549a67a4ee3492004d9c591f471
src/ttfm/eval.py	f83a6da0902bec7226a0a23112c97fdf549ce493642a002b535285c7d18fdd78
src/ttfm/metrics.py	84ee42f674b97ae7f14b04d68e09c9c899c89c8d528f178897ec67610d8662fc
src/ttfm/review.py	678820d64404d8a3f5ed2b9c59a54ac63596c67acb4082df53bc1a78604c73ee
outputs/.../test_metrics.json	f72f173c5c64bc2b4e5ff11bffc9184962db7f33fe2cc82136c661282bdd2263
outputs/.../test_review/ summary.json	bc0b0237acd555c4361540afa1ab0ad0b445734662877fb52fe77a37ebaca0b2
reports/cpu_pidnet_s_ryzen5500_ torch213_idle.json	4db19ce68688fcdca4a44924dfa64670d9045474448703feff3302c459f53f5b
configs/ablations/controlled_ ablation_augment_off.yaml	62869964e3fd30b3cba59d28f1799c6add73002c1a39af4dfb32641626d27928
scripts/edge_trt_benchmark.py	0a1acc1a55659384208cbf067f3c90c7c5a304b47749bada65e7be6cc624f2a3
reports/.trt_scratch/pidnet-s_ fp16.onnx	15ac7c3ea859e8776b073aaafca76a5507f501cd62be992efc54d08276d43d91
reports/.trt_scratch/pidnet-s_ fp16.engine	2458b84daa0133d0de7fea5f2c20fd1ecb3f335eaa00e81bd6f00c8f3f52da85
reports/edge_pidnet_rtx5060_ trt.json	5eb085823c892bc40d34d4997df5745206246ee8eba14cf906a7619d43107255

E. Implementation Map

Table E.1 maps the main repository paths to their role in creating or checking the experimental record.

TABLE E.1. Principal repository files and their role in the experimental record.

File or directory	Responsibility
<code>src/ttfm/audit.py</code>	Discover file counts, IDs, palette colours, dimensions, pairing, and corruption.
<code>src/ttfm/preprocess.py</code>	Apply the binary mapping, deterministic hash split, and write manifests and binary masks.
<code>src/ttfm/data.py</code>	Load manifest samples, resize RGB/masks, apply deterministic augmentation, scale tensors, and compute train-only class weights.
<code>src/ttfm/model.py</code>	Construct PIDNet, ROD, DDRNet, BiSeNet, and SMP systems behind one full-resolution logits interface.
<code>src/ttfm/vendor_pidnet/</code>	Vendored PIDNet implementation used by the baseline wrapper.
<code>src/ttfm/rod.py</code>	EfficientSAM ViT-S encoder, frozen-weight loading, intermediate feature extraction, ROD decoder, and output resizing.
<code>src/ttfm/train.py</code>	Losses, AdamW, cosine/polynomial schedules, mixed precision, epoch loops, validation, and best-checkpoint selection.
<code>src/ttfm/eval.py</code>	Read-only checkpoint evaluation and JSON metric output.
<code>src/ttfm/metrics.py</code>	Softmax threshold, confusion accumulation, mIoU, class IoUs, precision, recall, F1, FSR, and FBR.
<code>src/ttfm/review.py</code>	Per-image asymmetric-error and boundary scoring plus ranked qualitative panels.
<code>scripts/run_rod_suite.py</code>	Controlled PIDNet–ROD seeds, reconstructed training runs, common GPU timing, and aggregate reports.
<code>scripts/run_realtime_comparison_suite.py</code>	Nine-model blue-only repeat-seed comparison.
<code>scripts/run_realtime_blue_green_suite.sh</code>	Corresponding blue+green architecture suite.
<code>scripts/run_convergence_suite.py</code>	Generate, resume, evaluate, and aggregate the 27-run convergence-aware blue+green suite.

File or directory	Responsibility
scripts/build_ convergence_evidence.py	Audit convergence artifacts and regenerate the verified ledger, summary, and figures.
scripts/gpu_inference_ benchmark.py	Eager/CPU forward benchmark with warm-up and repeat controls.
reports/cpu_*_ ryzen5500*.json	Retained eager and compiled Ryzen benchmark records, including thread count, warm-up, repeats, sample count, and latency percentiles.
scripts/edge_trt_ benchmark.py	ONNX export, TensorRT engine construction, scoped latency, and full-test engine accuracy.
reports/*verified_ ledger.json	Per-seed metric, configuration, checkpoint, benchmark, and integrity records.

F. Development Time and Work Plan

The project was developed iteratively rather than from a fixed recipe. Early work compared several possible formulations of off-road perception before the binary CaT task and the final architecture study were selected. Table F.1 reconstructs the effort devoted to each principal step. The values are approximate because a formal timesheet was not maintained; they represent about 22 working days, or roughly 180 hours, of research, implementation, experimentation, and writing. Reading and revision continued across several phases, so the entries should be interpreted as an effort breakdown rather than as mutually exclusive calendar blocks.

TABLE F.1. Approximate development time devoted to the principal stages of the thesis. Automated computation is distinguished from hands-on research work where it materially affects the estimate.

Stage	Estimated effort	Work performed and reason for the effort
Problem framing, literature review, and alternative approaches	4 days (about 30 h)	Reviewed monocular, geometric, multimodal, and segmentation-based approaches; compared possible thesis directions; traced relevant CaT and ROD literature; and contacted an original author to query apparent inconsistencies in the published experimental description.
Dataset search and selection	3 days (about 23 h)	Compared candidate off-road datasets by annotation type, licence, scene diversity, task compatibility, and reproducibility before selecting CaT.
CaT audit and dataset preparation	2 days (about 16 h)	Audited RGB–mask–integer-map correspondence, identified error-prone filename pairing assumptions, reconstructed the palette mapping, defined the two binary policies, and produced deterministic development splits.
PIDNet-S baseline development	2 days (about 16 h)	Adapted PIDNet-S to two-class output, implemented the common training and evaluation path, checked tensor shapes and losses, and obtained a reproducible working baseline.
Controlled ablations and checkpoint audit	2 days (about 16 h)	Ran the controlled variants, checked validation-based checkpoint selection, recomputed strict test metrics, and generated qualitative error reviews.
ROD reconstruction and comparison	1 day (about 8 h)	Reconstructed the published frozen-encoder recipe, adapted it to the CaT policies, and established the matched PIDNet–ROD comparison.

Continued on the next page

Stage	Estimated effort	Work performed and reason for the effort
Architecture extension and reproducibility evidence	1.5 days (about 12 h)	Integrated seven further compact systems, standardised their interfaces and configurations, executed the multi-seed matrices, and assembled traceable result ledgers.
Hardware evaluation	1 day (about 8 h)	Defined timing scopes and measured CPU, H100, and TensorRT execution while checking that exported models preserved their source predictions.
Convergence-aware follow-up	1 full day (about 13 h elapsed)	Prepared, launched, monitored, and verified 27 long-horizon runs. Early stopping reduced the 300-epoch ceiling to validation-selected checkpoints from epochs 50–141.
Writing, figure production, and final revision	4.5 days (about 36 h)	Organised the argument, documented equations and protocols, produced and revised figures and tables, incorporated the convergence evidence, and checked language, citations, reproducibility, and UC3M presentation requirements.
Total	about 22 days (about 180 h)	Approximate hands-on and supervised experimental effort for the complete thesis.